

Přírodou inspirované počítání

Učební text k okruhu státní závěrečné zkoušky

SZZ Umělá inteligence, MFF UK

srpen 2026

Tento text pokrývá okruh *Přírodou inspirované počítání* magisterské SZZ oboru Umělá inteligence. Jeho členění se drží oficiálního znění okruhu: každé jeho větě odpovídá **jedna ku jedné** právě jedna kapitola, takže je vždycky vidět, kterou část zadání se čtenář zrovna učí. Přehled té vazby dává mapovací tabulka níže.

Výkladovým základem jsou slidy přednášek NAIL025 (Evoluční algoritmy I) a NAIL086 (Evoluční algoritmy II). Tam, kde slidy látku jen naznačují, ji text doplňuje o standardní podání z učebnic Eiben & Smith: *Introduction to Evolutionary Computing*, Mitchell: *An Introduction to Genetic Algorithms*, Michalewicz: *Genetic Algorithms + Data Structures = Evolution Programs* a Poli, Langdon & McPhee: *A Field Guide to Genetic Programming*. Vedle definic a pseudokódů obsahuje také odvození, číselné příklady a srovnávací tabulky. Každá kapitola pak končí shrnutím, které je psané rovnou pro ústní zkoušku.

Rozsah textu a značení

Kapitoly nejsou stejně dlouhé, a je to záměr: jejich rozsah odpovídá váze látky. Nejvíce prostoru proto dostávají genetické algoritmy spolu s genetickým programováním, teorie schémat a aplikace, tedy právě to, čemu se přednášky věnují nejpodrobněji.

Značka [♦ nad rámeček slidů] upozorňuje na látku, která **není v žádných dostupných slidech**, tj. ani v EVA I, ani v EVA II, ani ve volitelné *Evoluční robotice* (Mráz). V textu je taková látka přesto ponechána, a to ze dvou důvodů: buď ji *jmenuje oficiální znění okruhu*, nebo by bez ní okolní výklad nedával smysl. Značka tak slouží jako nabídka ke škrtnutí: co označeno není, je ve slidech doloženo. Je-li látka ve slidech zmíněna jen letmo, uvádí značka rovnou i rozsah té zmínky.

Nejnápadnější případ je **otevřená evoluce**. Okruh ji výslovně jmenuje, slidy o ní mají jedinou větu. Podobně jsou na tom mravenčí algoritmy (ACO): okruh mluví o rojových algoritmech v množném čísle, slidy ale probírají jen PSO. (Rojová robotika, kterou probírá Evoluční robotika, je něco jiného než rojová optimalizace.) Opačným případem jsou *gramatická evoluce* a *interaktivní evoluce*. Ty označeny nejsou: v EVA sice chybějí, ale Evoluční robotika je probírá.

Mapování okruhu na kapitoly

Věta oficiálního znění okruhu	Kapitola v tomto textu
Genetické algoritmy, genetické a evoluční programování.	1 Genetické algoritmy, genetické a evoluční programování
Teorie schémat, pravděpodobnostní modely jednoduchého genetického algoritmu.	2 Teorie schémat a pravděpodobnostní modely SGA
Evoluční strategie, diferenciální evoluce, koevoluce, otevřená evoluce.	3 Evoluční strategie, diferenciální evoluce, koevoluce, otevřená evoluce
Rojové optimalizační algoritmy.	4 Rojové optimalizační algoritmy
Memetické algoritmy, hill climbing, simulované žíhání.	5 Lokální prohledávání a memetické algoritmy
Aplikace evolučních algoritmů (evoluce expertních systémů, neuroevoluce, řešení kombinatorických úloh, vícekritériální optimalizace).	6 Aplikace evolučních algoritmů

Obsah

Mapování okruhu na kapitoly	1
1 Genetické algoritmy, genetické a evoluční programování	5
1.1 Obecné schéma evolučního algoritmu	5
1.1.1 Zařazení oboru	5
1.1.2 Biologická inspirace	5
1.1.3 Obecné schéma evolučního algoritmu	6
1.1.4 Složky evolučního algoritmu	6
1.1.5 Seleční mechanismy	7
1.1.6 Teoretické meze: No Free Lunch	9
1.1.7 Historické větve evolučních algoritmů	9
1.2 Genetické algoritmy	10
1.2.1 Jednoduchý genetický algoritmus (SGA)	10
1.2.2 Binární reprezentace	10
1.2.3 Operátory křížení	11
1.2.4 Mutace a inverze	11
1.2.5 Škálování fitness	12
1.3 Reprezentace jedince a genetické operátory	12
1.3.1 Celočíselná reprezentace	12
1.3.2 Reálná reprezentace	13
1.3.3 Permutační reprezentace	13
1.3.4 Další reprezentace	15
1.4 Genetické a evoluční programování	15
1.4.1 Historie	15
1.4.2 Stromové GP	16
1.4.3 Bloat	17
1.4.4 ADF a typované GP	17
1.4.5 Lineární a grafové GP	18
1.4.6 Gramatická evoluce	18
1.4.7 Evoluční programování	19
1.4.8 Je křížení k něčemu? Experiment s bezhlavým kuřetem	20
1.5 Shrnutí ke zkoušce	20
2 Teorie schémat a pravděpodobnostní modely SGA	21
2.1 Teorie schémat	21
2.1.1 Schéma, jeho řád a definující délka	21
2.1.2 Hollandova věta o schématech	22
2.1.3 Hypotéza stavebních bloků	23
2.1.4 Implicitní paralelismus a vícetuký bandita	24
2.1.5 Kritika teorie schémat	24
2.2 Pravděpodobnostní modely jednoduchého GA	25
2.2.1 Zjednodušený SGA	25
2.2.2 Formalizace populace	25
2.2.3 Operátor $\mathcal{G}$	26
2.2.4 Výsledky pro nekonečné populace	27
2.2.5 Konečné populace: Markovův řetězec	27
2.2.6 Algoritmy odhadu rozdělení (EDA)	28
2.3 Shrnutí ke zkoušce	28
3 Evoluční strategie, diferenciální evoluce, koevoluce, otevřená evoluce	29
3.1 Evoluční strategie	29

3.1.1	Motivace a základní rysy	29
3.1.2	(1 + 1)-ES a pravidlo 1/5	30
3.1.3	Samoadaptace strategických parametrů	30
3.1.4	Rekombinace v ES	31
3.1.5	CMA-ES	32
3.2	Diferenciální evoluce	32
3.2.1	Motivace	32
3.2.2	Algoritmus DE/rand/1/bin	32
3.2.3	Číselný příklad jednoho kroku	33
3.2.4	Varianty mutace	33
3.2.5	Srovnání DE s ES a GA	34
3.3	Koevoluce	34
3.3.1	Kontextová fitness	34
3.3.2	Kompetitivní koevoluce	35
3.3.3	Patologie koevoluce	35
3.3.4	Kooperativní koevoluce	35
3.3.5	Evoluce kooperace a věžňovo dilema	36
3.3.6	Diverzita, druhy a niky	37
3.3.7	Dynamické fitness krajiny	38
3.4	Otevřená evoluce	38
3.5	Shrnutí ke zkoušce	39
4	Rojové optimalizační algoritmy	40
4.1	Optimalizace rojem částic (PSO)	41
4.2	Optimalizace mravenčí kolonií (ACO)	42
4.3	Další rojové algoritmy	44
4.4	Shrnutí ke zkoušce	44
5	Lokální prohledávání a memetické algoritmy	45
5.1	Lokální prohledávání a horolezecký algoritmus	45
5.2	Simulované žhání	45
5.3	Tabu prohledávání	47
5.4	Scatter search	47
5.5	Memetické algoritmy	47
5.6	Ladění a řízení parametrů	48
5.7	Shrnutí ke zkoušce	49
6	Aplikace evolučních algoritmů	49
6.1	Evoluce pravidlových a expertních systémů	49
6.1.1	Michigan vs. Pittsburgh	49
6.1.2	Learning classifier systems (Michigan)	50
6.1.3	Pittsburghský přístup a systém GIL	51
6.1.4	Připomenutí: metriky klasifikace	52
6.2	Neuroevoluce	52
6.2.1	Evoluce vah	52
6.2.2	Evoluce architektury	53
6.2.3	NEAT	53
6.2.4	HyperNEAT a CPPN	54
6.2.5	Neuroevoluce v éře hlubokých sítí	54
6.3	Kombinatorické úlohy a práce s omezeními	54
6.3.1	Problém batohu	54
6.3.2	Tři obecné strategie pro omezení	54
6.3.3	Problém obchodního cestujícího	55

6.3.4	SAT	56
6.3.5	Rozvrhování a plánování výroby	56
6.4	Vícekriteriální optimalizace	56
6.4.1	Dominance a Pareto fronta	56
6.4.2	Skalarizace — jednoduchá řešení	57
6.4.3	Historické algoritmy	57
6.4.4	NSGA-II	57
6.4.5	Další významné algoritmy	59
6.4.6	Metriky kvality aproximace	59
6.4.7	Souhrnné srovnání MOEA	59
6.5	Shrnutí ke zkoušce	59
Souhrnné srovnání a typické zkouškové otázky		61

1 Genetické algoritmy, genetické a evoluční programování

Tohle je nejobsáhlejší věta celého okruhu a tomu odpovídá i stavba kapitoly. Nejdřív přijde obecné schéma evolučního algoritmu, protože bez něj nedává zbytek textu smysl. Na ně navazuje Hollanův jednoduchý genetický algoritmus spolu s reprezentacemi jedince. Kapitola končí genetickým a evolučním programováním, tedy evolucí spustitelných struktur.

1.1 Obecné schéma evolučního algoritmu

1.1.1 Zařazení oboru

Pod hlavičkou *přírodou inspirovaného počítání* (*nature-inspired computing*) se schází metaheuristické algoritmy, které principy svého fungování přebírají z toho, co se pozoruje v živé, a někdy i v neživé přírodě. Obor se tradičně člení na tři postupně se zužující okruhy:

- **Výpočetní inteligence** (*computational intelligence*, dříve *soft computing*) je z nich nejširší a patří do ní neuronové sítě, fuzzy systémy a evoluční výpočty. Vymezuje se proti „tvrdé“, logicky a symbolicky orientované AI.
- **Evoluční výpočty** (*evolutionary computation*, EC) jsou nadmnožinou evolučních algoritmů: vedle nich sem spadají i metody, které evoluční v úzkém smyslu nejsou, tedy rojové algoritmy, memetické algoritmy nebo tabu prohledávání.
- **Evoluční algoritmy** (*evolutionary algorithms*, EA) jsou podmnožinou EC, která se drží základních principů přírodní evoluce. Pracují s populací kandidátních řešení, mění ji stochastickými operátory (křížením a mutací) a o přežití rozhoduje selekce řízená účelovou funkcí.

Z hlediska klasifikace optimalizačních metod jsou EA **populační stochastické prohledávací algoritmy**. Sáhne se po nich tam, kde klasické numerické postupy selhávají. Typicky nemáme k dispozici gradient ani analytický tvar účelové funkce a ta se chová jako černá skříňka (tzv. *black-box optimization*). Jindy je prostor řešení diskrétní nebo strukturovaný a skládá se ze stromů, grafů či permutací. A konečně se EA hodí tam, kde nehledáme jedno řešení, ale rovnou několik vzájemně nedominovaných.

1.1.2 Biologická inspirace

Terminologie i mechanika evolučních algoritmů jsou převzaté z biologie a vyplatí se vědět, odkud která myšlenka pochází.

- **Darwin (1859, O původu druhů)**. Prostředí nabízí jen omezené zdroje, takže o ně jedinci soupeří a klíčem k přežití je reprodukce. Kdo je lépe přizpůsobený, má větší šanci se rozmnožit, a úspěšné znaky fenotypu se proto reprodukuje, modifikují a rekombinují.
- **Mendel (1866)**. Zavádí gen jako základní jednotku dědičnosti a předpokládá, že se alely předávají nezávisle. Realita je složitější: *polygenie* znamená, že jeden znak ovlivňuje více genů, *pleiotropie* naopak to, že jeden gen ovlivňuje více znaků, a mimo Mendelovo schéma stojí i mitochondriální DNA a epigenetika.
- **Watson a Crick (1953)**. Popis struktury DNA přidává molekulárně-biologický pohled na to, jak se informace ukládá a dědí. Kodon, tedy trojice nukleotidů, kóduje jednu z aminokyselin a kód je redundantní.
- **Genotyp → fenotyp**. Cesta od vnitřního zápisu k vnějším vlastnostem je jednosměrná a složitá, protože vede přes transkripci DNA→RNA a translaci RNA→protein. *Lamarckismus* oproti tomu předpokládá existenci inverzního zobrazení, tedy dědičnost získaných vlastností. Biologie ho odmítla, v memetických algoritmech se však jako technika používá (kap. 5).

Překlad mezi obojím slovníkem je přímočarý a drží se ho celý zbytek textu:

Slovník: příroda vs. algoritmus

prostředí	optimalizační úloha
jedinec / chromozom	kandidátní řešení, bod prohledávaného prostoru
gen, alela	složka řešení a její hodnota
genotyp	vnitřní reprezentace (kódování) jedince
fenotyp	„vnějšek“ jedince, na němž se hodnotí kvalita
fitness	míra kvality řešení (účelová funkce)
populace	multimnožina kandidátních řešení
generace	jedna iterace algoritmu

1.1.3 Obecné schéma evolučního algoritmu

Všechny evoluční algoritmy sdílejí jeden a týž cyklus a liší se až tím, čím jeho jednotlivé kroky naplní.

Evoluční algoritmus

Evoluční algoritmus je iterativní stochastický proces, který udržuje populaci $P(t)$ kandidátních řešení a z ní opakovaně vytváří populaci $P(t+1)$ pomocí *rodičovské selekce*, *rekombinace*, *mutace* a *environmentální selekce*, dokud není splněna ukončovací podmínka.

Algoritmus 1 Obecné schéma evolučního algoritmu

Vstup: reprezentace, fitness f , operátory, parametry

- 1: $t \leftarrow 0$; inicializuj náhodně populaci $P(0)$
- 2: ohodnoť všechny jedince v $P(0)$ funkcí f
- 3: **while** není splněna ukončovací podmínka **do**
- 4: **rodičovská selekce:** vyber z $P(t)$ množinu rodičů (pravděpodobnostně, dle fitness)
- 5: **rekombinace:** z dvojic (n -tic) rodičů vytvoř potomky s pravděpodobností p_C
- 6: **mutace:** každého potomka náhodně poruš s pravděpodobností p_M
- 7: ohodnoť potomky $P'(t+1)$
- 8: **environmentální selekce:** z $P(t) \cup P'(t+1)$ (nebo jen z $P'(t+1)$) vyber novou populaci $P(t+1)$
- 9: $t \leftarrow t + 1$
- 10: **end while**

Výstup: nejlepší nalezený jedinec

Uvedený cyklus je společný všem EA. Jednotlivé varianty, tedy GA, ES, EP, GP nebo DE, se liší až volbou reprezentace, důrazem na jednotlivé operátory a typem selekce. Uvnitř cyklu přitom proti sobě stojí dvě síly:

- **Variace**, tedy křížení a mutace, vytváří novou rozmanitost, a stará se tak o *exploraci* (*exploration*) prostoru.
- **Selekce** naopak rozmanitost odebírá a soustředí hledání do slibných oblastí, čímž zajišťuje *exploataci* (*exploitation*).

Najít mezi nimi rovnováhu je ústřední problém návrhu každého EA. Když převáží explorace, algoritmus degeneruje na náhodnou procházku. Když převáží exploatace, populace uvízne v lokálním extrému a dojde k *předčasné konvergenci* (*premature convergence*).

1.1.4 Složky evolučního algoritmu

Návrh konkrétního algoritmu se skládá z několika rozhodnutí, která na sebe navazují.

Reprezentace. Volba kódování je vůbec nejdůležitější návrhové rozhodnutí. Nabízí se binární řetězec, vektor celých čísel, vektor reálných čísel, permutace, strom, graf, matice, konečný automat i neuronová síť. Volba není kosmetická, protože reprezentace určuje, jaké operátory vůbec dávají smysl.

Fitness funkce. Kvalitu jedince měří zobrazení $f : \text{prostor fenotypů} \rightarrow \mathbb{R}$. Obvykle se maximalizuje, takže minimalizační úlohu je potřeba převrátit, například pomocí $f' = C_{\max} - f$ nebo $f' = 1/(1 + f)$. Sama fitness nemusí být pevná hodnota: bývá *deterministická*, *zašuměná*, počítá-li se simulací, *dynamická*, mění-li se v čase (odst. 3.3), nebo *kontextová*, závisí-li na ostatních jedincích, jak je tomu u koevoluce.

Populace. Populace je multimnožina jedinců, obvykle pevné velikosti n . Právě ona je nositelem *diverzity* a tu lze měřit průměrnou Hammingovou vzdáleností, entropií alel na jednotlivých pozicích nebo rozptylem fitness.

Inicializace. Standardem je náhodná uniformní inicializace, existují ale i cílenější postupy. *Seeding* vkládá do počáteční populace heuristická nebo expertní řešení, čímž ovšem riskuje předčasnou konvergenci. Dále se používají konstrukční heuristiky, například nejbližší soused pro TSP, a pro reálné vektory latinské hyperkrychle.

Ukončovací podmínka. Běh se zastaví po vyčerpání rozpočtu, tedy maximálního počtu generací či vyhodnocení fitness, nebo po dosažení cílové kvality. Dalšími rozumnými kritérii jsou stagnace nejlepší fitness po k generací a ztráta diverzity populace.

1.1.5 Selekcční mechanismy

V selekci se rozhoduje o velikosti selekčního tlaku, a proto se jí věnuje celá řada mechanismů. Liší se tím, kolik informace o fitness využívají a jak citlivé jsou na její měřítko.

Ruletová (fitness-proporcionální) selekce

Ruletová selekce (*roulette wheel selection*)

Jedinec x_i je vybrán s pravděpodobností

$$p_i = \frac{f(x_i)}{\sum_{j=1}^n f(x_j)} = \frac{f(x_i)}{n \bar{f}},$$

kde $\bar{f}$ je průměrná fitness populace. Očekávaný počet výběrů jedince při n tazích je $np_i = f(x_i)/\bar{f}$. Vybírá se *s vrácením*, takže jeden jedinec může uspět i vícekrát.

Příklad (klasický Goldbergův). Uvažme populaci čtyř 5-bitových řetězců, které se dekodují jako celá čísla a jejichž fitness je $f(x) = x^2$:

i	řetězec	x	$f = x^2$	$p_i = f / \sum f$	oček. počet $f/\bar{f}$
1	01101	13	169	0,144	0,58
2	11000	24	576	0,492	1,97
3	01000	8	64	0,055	0,22
4	10011	19	361	0,309	1,23
součet			1170	1,000	4,00
průměr $\bar{f}$			292,5		

Poslední sloupec je poměr fitness jedince k průměru populace, tedy očekávaný počet kopií v mezi-populaci. Ruleta rozdělí obvod kola na výseče o velikostech p_i a pak se n -krát roztočí. Jedinec 2 tak obsadí v mezipopulaci zhruba dvě místa, kdežto jedinec 3 z ní nejspíš vypadne.

Nevýhody ruletové selekce:

- *Předčasná dominance* „supermana“: jedinec s extrémní fitness ovládne populaci a diverzita zmizí.
- *Ztráta selekčního tlaku* na konci běhu: jakmile jsou všechny fitness podobné, jsou podobné i pravděpodobnosti výběru a selekce se blíží náhodnému výběru.
- Ruleta vyžaduje kladné fitness a je citlivá na afinní transformace, protože f a $f + c$ dávají jiné pravděpodobnosti. Obojí se řeší *škálováním* (odst. 1.2.5).
- Výběr má vysoký rozptyl, takže se skutečné počty kopií mohou od očekávaných značně lišit.

Stochastické univerzální vzorkování (SUS) Poslední zmíněnou nevýhodu odstraňuje *stochastic universal sampling*. Kolo se roztočí jen jednou, zato je po jeho obvodu rozmístěno n ukazatelů vzdálených od sebe o $1/n$. Očekávané počty výběrů zůstávají stejné jako u rulety, jenže rozptyl klesne na minimum: jedinec s očekávaným počtem e_i je vybrán $\lfloor e_i \rfloor$ nebo $\lceil e_i \rceil$ -krát. Je to tedy „spravedlivější ruleta“.

Turnajová selekce

k -turnajová selekce (*tournament selection*)

Vyber náhodně (uniformně, s vrácením) k jedinců a vrať toho nejlepšího. Parametr k (typicky 2, 3, 5) přímo řídí selekční tlak: pravděpodobnost, že bude vybrán i -tý nejlepší jedinec z n (zde $i = 1$ značí **nejlepšího**), je $((n - i + 1)^k - (n - i)^k) / n^k$ — je to pravděpodobnost, že všech k losovaných padne mezi i -tého a horší, mínus pravděpodobnost, že všech k padne ostře pod i -tého. Deterministický turnaj lze změkčit: lepší vyhraje s pravděpodobností $p < 1$.

Turnaj má hned několik výhod. Nepotřebuje globální informaci o populaci, protože nic nesčítá, a snadno se proto paralelizuje. Je invariantní vůči monotónním transformacím fitness. A funguje i tam, kde fitness není explicitně dána a porovnání dvou jedinců se získává až *souhrou*, tedy odehranou partií, simulací nebo koevolucí.

Pořadová selekce Ještě dál jde *rank-based selection*, která hodnotu fitness zahodí zcela a nahradí ji pořadím jedince. Lineární pořadová selekce přiřadí jedinci s pořadím i (*pozor, zde opačně než u turnaje*: nejhorší $i = 1$, nejlepší $i = n$) pravděpodobnost

$$p_i = \frac{1}{n} \left(s^- + (s^+ - s^-) \frac{i - 1}{n - 1} \right), \quad 1 \leq s^+ \leq 2, \quad s^- = 2 - s^+,$$

kde s^+ je očekávaný počet kopií nejlepšího jedince. Exponenciální varianta místo toho volí $p_i \propto (1 - c)^{c^{n-i}}$. Protože pořadí na absolutních hodnotách nezávisí, udržuje pořadová selekce konstantní selekční tlak během celého běhu a neublíží jí ani „superman“, ani změna měřítko fitness.

Boltzmannova selekce Boltzmannova selekce vybírá jedince s pravděpodobností $p_i \propto e^{f(x_i)/T}$, kde T je *teplota* klesající v čase. Na začátku je T vysoká a selekce se blíží uniformnímu výběru, takže převládá *explorace*; na konci je T nízká a selekce se stává téměř deterministickou, tedy *exploatační*. Stejnou myšlenku používá *Boltzmannův turnaj*, v němž horší jedinec vyhraje s pravděpodobností závislou exponenciálně na poměru rozdílu fitness a teploty. Souvislost se simulovaným žíháním je přímá (kap. 5).

Environmentální selekce, elitismus, modely populace Rodičovská selekce rozhoduje o tom, kdo se bude rozmnožovat; environmentální selekce o tom, kdo se dožije další generace. Podle toho, jak velkou část populace vymění, se rozlišují tyto modely:

- **Generační model** (*generational*) nahradí celou populaci potomky naráz, takže $|P'| = |P| = n$.
- **Model s ustáleným stavem** (*steady-state*) mění populaci po kouscích: v každém kroku vznikne jen λ potomků, často 1–2, a ti nahradí vybrané jedince. Nahradit lze nejhoršího, náhodně zvoleného, nebo nejpodobnějšího, čemuž se říká *crowding*. Podíl nahrazované populace udává parametr *generation gap* $G = \lambda/n$.

- **Elitismus** (*elitism*) bezpodmínečně kopíruje k nejlepších jedinců do další generace. Zaručuje tím monotonii nejlepší nalezené fitness a je nutnou podmínkou pro řadu konvergenčních vět; příliš silný elitismus však snižuje diverzitu.
- **Deterministická selekce ES** nenechává nic náhodě: $(\mu + \lambda)$ vybírá μ nejlepších z rodičů i potomků dohromady, a má tedy implicitní elitismus, kdežto (μ, λ) vybírá jen z potomků (kap. 3).

Selekční tlak a doba převzetí

Selekční tlak (*selection pressure*) je míra zvýhodnění lepších jedinců. Kvantifikuje se *dobou převzetí* (*takeover time*) τ^* , tedy počtem generací, za který jediná kopie nejlepšího jedince zaplní bez variace celou populaci. Pro turnaj velikosti k platí zhruba $\tau^* \approx \ln n / \ln k$. Pro fitness-proporcionální selekci je doba převzetí delší a závisí na rozložení fitness.

1.1.6 Teoretické meze: No Free Lunch

Nabízí se otázka, jestli nejde navrhnout algoritmus, který bude lepší než ostatní na čemkoli. Odpověď je záporná a má podobu věty.

Věta 1.1 (No Free Lunch, Wolpert & Macready 1995–1997). *Zprůměrováno přes všechny možné účelové funkce na konečné doméně mají libovolné dva deterministické neopakující se prohledávací algoritmy stejný očekávaný výkon. Neexistuje tedy univerzálně nejlepší optimalizační algoritmus.*

Praktický důsledek je jasný: vyplácí se stavět **doménově specifické varianty** EA, tedy volit reprezentaci a operátory tak, aby zohledňovaly strukturu úlohy. Co věta naopak neříká, je to, že by si na *reálných* úlohách byly všechny algoritmy rovny. Reálné funkce tvoří velmi malou a strukturovanou podmnožinu všech funkcí a právě té struktury se dá využít.

1.1.7 Historické větve evolučních algoritmů

Evoluční algoritmy nevyrostly z jediného kořene. Nezávisle na sobě vznikly čtyři školy, které se lišily reprezentací i tím, který operátor považovaly za hlavní:

	GA	ES	EP	GP
autor, rok	Holland, 1975	Rechenberg, Schwefel, 1964	L. Fogel, Owens, Walsh, 1965	Koza, 1989–92
reprezentace	binární řetězec	vektor $\mathbb{R}^n$ + strategické parametry	konečný automat (genotyp = fenotyp)	syntaktický strom (LISP)
hlavní operátor	křížení	mutace	mutace	křížení podstromů
křížení	1-bodové	rekombinace více rodičů (lokální/globální)	<i>není</i>	výměna podstromů
mutace	bit-flip p_M	gaussovská, samoadaptivní σ	„chytré“ mutace automatu	více typů
rodičovská selekce	ruleta	náhodná	každý jednou	turnaj
env. selekce	generační náhrada	deterministická (μ, λ) , $(\mu + \lambda)$	turnaj (rodiče + potomci)	generační
teorie	schémata	konvergence, 1/5 pravidlo	—	bloat, schémata

Dnes se hranice mezi větvemi stírají; EP a ES třeba prakticky splynuly a mluví se jednotně o evolučních algoritmech.

1.2 Genetické algoritmy

1.2.1 Jednoduchý genetický algoritmus (SGA)

Původnímu Hollandovu návrhu z roku 1975 se dnes říká *simple genetic algorithm* (SGA). Vystačí si s minimální sadou operátorů a s nejjednodušším možným kódováním, a to záměrně: jednoduchost byla podmínkou toho, aby šel algoritmus teoreticky analyzovat (teorie schémat, kap. 2).

SGA

SGA udržuje populaci n binárních řetězců délky m . Rodiče vybírá fitness-proporcionální (ruletovou) selekcí, kříží je jednobodovým křížením s pravděpodobností p_C a na každý bit potomka aplikuje bitovou mutaci s pravděpodobností p_M . Populace se nahrazuje generačně: $P(t+1)$ zcela nahradí $P(t)$.

Algoritmus 2 Jednoduchý genetický algoritmus (SGA)

```
1:  $t \leftarrow 0$ ; vygeneruj náhodně  $P(0) = n$  řetězců délky  $m$ 
2: while není splněna ukončovací podmínka do
3:   spočítej  $f(x)$  pro každé  $x \in P(t)$ 
4:    $P(t+1) \leftarrow \emptyset$ 
5:   for  $n/2$  krát do
6:     vyber ruletou dvojici rodičů  $x, y$  z  $P(t)$ 
7:     s pravděpodobností  $p_C$  zkříž  $x, y$  jednobodovým křížením
8:     každý bit  $x$  i  $y$  převrať s pravděpodobností  $p_M$ 
9:     vlož  $x, y$  do  $P(t+1)$ 
10:  end for
11:   $t \leftarrow t+1$ 
12: end while
```

Parametry se v praxi volí takto: $n \in [50, 500]$, $p_C \in [0,6; 0,9]$ a $p_M \approx 1/m$, což znamená, že se v průměru změní jeden bit na jedince.

1.2.2 Binární reprezentace

SGA pracuje s binárními řetězci, takže cokoli jiného než bity je nutné do bitů zakódovat.

Kódování celých a reálných čísel. Reálné číslo $x \in [a, b]$ zakódované na k bitů jako celé číslo $z \in \{0, \dots, 2^k - 1\}$ dekódujeme vztahem

$$x = a + z \cdot \frac{b - a}{2^k - 1}.$$

Dosažitelná přesnost je $(b - a)/(2^k - 1)$. Výhodou je explicitní kontrola nad přesností a komprese prohledávaného prostoru. Nevýhodou jsou *Hammingovy útesy* (*Hamming cliffs*): dvě sousední hodnoty se mohou lišit ve všech bitech naráz, třeba $0111 \rightarrow 1000$. Malá změna fenotypu pak vyžaduje velkou změnu genotypu, což mutaci po jednotlivých bitech znesnadňuje.

Grayův kód. Hammingovy útesy odstraňuje Grayův kód, v němž se sousední celá čísla liší právě v jednom bitu. Z binárního zápisu b na Grayův g se přechází předpisem $g_1 = b_1$, $g_i = b_{i-1} \oplus b_i$; zpět pak $b_1 = g_1$, $b_i = b_{i-1} \oplus g_i$.

Hollandův argument pro binární abecedu. Holland zdůvodňoval volbu dvouprvkové abecedy počtem schémat. Binární řetězce délky 100 kódují zhruba stejnou informaci jako desítkové délky 30, obsahují ale více schémat (3^{100} oproti 11^{30}), takže GA „zpracovává“ více informací naráz. Dnes víme, že schémata nejsou tak zásadní, jak se Holland domníval, a že rozhodující je **přirozenost**

kódování pro danou úlohu. Pro reálnou optimalizaci se proto používají reálné vektory, pro TSP permutace atd.

1.2.3 Operátory křížení

Genetický algoritmus stojí a padá s křížením, takže se vyplatí projít jeho základní varianty a všimnout si přitom, jak moc která rozbíjí to, co už populace našla.

Křížení (*crossover, recombination*)

Operátor, který ze dvou (nebo více) rodičů vytvoří potomka kombinací jejich genetického materiálu. V GA je hlavním operátorem; vyjadřuje naději, že rekombinací dobrých částí (*stavebních bloků*) vzniknou ještě lepší jedinci. Aplikuje se s pravděpodobností p_C , jinak potomci = kopie rodičů.

Jednobodové křížení. Náhodně se zvolí bod řezu $k \in \{1, \dots, m-1\}$ a rodiče si vymění koncové části za ním.

Příklad. Rodiče $P_1 = 1011|0110$, $P_2 = 0010|1101$, bod řezu $k = 4$:

$$O_1 = \mathbf{1011} \ 1101, \quad O_2 = \mathbf{0010} \ 0110.$$

Dvoubodové křížení. Zvolí se dva body $k_1 < k_2$ a vymění se úsek mezi nimi. Proti jednobodové variantě se řetězec chová jako kruh, takže geny na okrajích nejsou znevýhodněné: u jednobodového křížení je totiž pravděpodobnost rozbití úseku úměrná jeho *definující délce* (viz kap. 2).

Příklad. $P_1 = 10|1101|10$, $P_2 = 00|1011|01 \Rightarrow O_1 = 10 \ 1011 \ 10$, $O_2 = 00 \ 1101 \ 01$.

Uniformní křížení. Pro každou pozici j se zvlášť hodí mince: s pravděpodobností 0,5 pochází gen od prvního rodiče, jinak od druhého, přičemž druhý potomek se sestaví komplementárně. Sousedící pozice tedy nejsou nijak zvýhodněné, zato uniformní křížení silně narušuje schémata s velkou definující délkou, takže má vysokou *disruptivitu*.

Příklad. $P_1 = 11111111$, $P_2 = 00000000$, maska 10110010: $O_1 = 10110010$, $O_2 = 01001101$.

Jak moc který operátor rozbíjí schémata, shrnuje následující tabulka:

operátor	p-st rozbití schématu délky d	poznámka
jednobodové	$d/(m-1)$	poziční bias, chrání krátká schémata
dvoubodové	$\frac{2d(m-d)}{m(m-1)}$	řetězec jako kruh, odstraňuje bias konců
k -bodové	roste s k	
uniformní	$1 - (1/2)^{o(S)-1}$	distribuční bias, největší disruptivita

Poznámka k dvoubodovému křížení. Vzorec ve druhém řádku stojí za odvození. Chápeme-li řetězec jako kruh, vybíráme dva ze m řezů. Schéma je rozbito, padne-li dovnitř jeho definujícího úseku o d mezerách právě jeden z nich, tj. s pravděpodobností $d(m-d)/\binom{m}{2} = 2d(m-d)/(m(m-1))$. Pro schéma sahající od prvního k poslednímu bitu, tedy pro $d = m-1$, klesne rozbití z jistoty (jednobodové: $d/(m-1) = 1$) na pouhé $2/m$, a to je přesně ono „odstranění okrajového biasu“. U krátkých schémat je tomu naopak: dvoubodové křížení je asi dvakrát destruktivnější než jednobodové.

1.2.4 Mutace a inverze

Bitová mutace. Každý bit se nezávisle na ostatních překlopí s pravděpodobností p_M . V SGA má mutace roli *sekundárního* operátoru a slouží jako pojistka: brání uvíznutí v lokálním extrému a trvalé ztrátě alely z populace. V EP a v raných ES je tomu jinak, tam je mutace jediným zdrojem variability. Doporučenou hodnotou je $p_M \approx 1/m$.

Inverze. Původní Hollandův návrh počítal ještě s operátorem *inverze*. Ten obrátí pořadí části řetězce, ale *zachová význam bitů*, protože každý gen si s sebou nese identifikátor své pozice. Cílem bylo nechat evolovat samotné uspořádání genů tak, aby spolupracující geny ležely blízko sebe a nebyly rozbíjeny křížením. Technicky je inverze náročná a její přínos se neprokázal. U permutační reprezentace se však používá jako velmi účinná mutace (odst. 1.3).

1.2.5 Škálování fitness

Ruletová selekce reaguje na absolutní hodnoty fitness, a to jí dělá potíže na obou koncích běhu. Zpočátku bývá rozptyl fitness velký a populaci ovládne několik „supermanů“. Ke konci je rozptyl naopak malý a selekce ztrácí tlak. Oboje se dá napravit tím, že se fitness před selekcí transformuje předpisem $f \mapsto f'$:

- **Lineární škálování** počítá $f' = af + b$, kde se a, b volí tak, aby $\bar{f}' = \bar{f}$ a $f'_{\max} = C_{\text{mult}} \cdot \bar{f}$, typicky s $C_{\text{mult}} \in [1, 2]$. Nutná je pojistka proti záporným hodnotám u nejhorších jedinců.
- **Sigma truncation** počítá $f' = \max(0, f - (\bar{f} - c\sigma_f))$, kde σ_f je směrodatná odchylka fitness a $c \in [1, 3]$. Odstraní odlehlé nízké hodnoty a automaticky se přizpůsobuje rozptylu.
- **Mocninné škálování** umocňuje, $f' = f^k$, přičemž k se v průběhu běhu zvyšuje, takže tlak roste.
- **Pořadové škálování** nahradí fitness pořadím (viz odst. 1.1.5). Je nejrobustnější a dnes nejobvyklejší.
- **Boltzmannovo škálování** používá $f' = e^{f/T}$ s klesající teplotou.

Příklad. Vezměme populaci s fitness (169, 576, 64, 361), kde $\bar{f} = 292,5$ a $f_{\max} = 576$, a použijme lineární škálování s $C_{\text{mult}} = 1,5$. Chceme $f'_{\max} = 438,75$ a $\bar{f}' = 292,5$. Z podmínek $af_{\max} + b = 438,75$ a $a\bar{f} + b = 292,5$ dostáváme $a = 146,25/283,5 = 0,516$ a $b = 292,5 - 0,516 \cdot 292,5 = 141,6$. Nové fitness jsou (228,8; 438,8; 174,6; 327,9) a podíl nejlepšího jedince klesl z 49,2 % na 37,5 %. Selektční tlak se tím zmírnil, a přitom ani nejhorší jedinec neztrácí šanci.

1.3 Reprezentace jedince a genetické operátory

Volba reprezentace rozhoduje o tom, které operátory mají vůbec smysl, a tím zároveň o tom, jak vypadá *fitness krajina* (*fitness landscape*), tedy jak snadné bude prohledávání. Při návrhu evolučního algoritmu je to nejdůležitější praktické rozhodnutí. Následující oddíly proto procházejí obvyklé reprezentace jednu po druhé a u každé ukazují, jaké mutace a jaké křížení k ní patří.

1.3.1 Celočíselná reprezentace

Jedinec je vektor $(z_1, \dots, z_m) \in \prod_j D_j$, kde D_j jsou konečné domény. Záleží ale na tom, jakou povahu hodnoty v doméně mají, protože od ní se odvíjí volba mutace. Rozlišujeme dva případy:

- **Kardinální (ordinální) atributy** mají smysluplné uspořádání a metriku, takže se dá říci, které dvě hodnoty jsou si blízké. Patří sem třeba počet strojů nebo věk.
- **Nominální atributy** jsou pouhé kódy bez vnitřního uspořádání, například barva nebo typ komponenty. O „blízkých“ hodnotách tu nemá smysl mluvit.

Mutace. Podle povahy atributu se používá jeden ze dvou operátorů:

- *Nezaujatá* mutace (*unbiased, random resetting*) losuje novou hodnotu náhodně z celé domény. Pro nominální atributy je to jediná korektní volba.
- *Zaujatá* mutace (*biased, creep mutation*) ponechá původní hodnotu a posune ji o malý náhodný krok, například z diskretizovaného normálního rozdělení. Vhodná je jen pro ordinální atributy, protože jinde by „malý krok“ nic neznamenal.

Křížení. S křížením je to jednodušší, protože se na rozdíl od mutace nemusí ohlížet na povahu hodnot. Používají se stejné operátory jako u binární reprezentace, tedy jednobodové, vícebodové i uniformní. Jediný rozdíl je v tom, že se místo jednotlivých bitů kopírují celá čísla.

1.3.2 Reálná reprezentace

Jedinec je $\vec{x} = (x_1, \dots, x_n) \in \mathbb{R}^n$. Historicky se reálná čísla kódovala do bitových řetězců a pracovalo se s nimi binárními operátory. Dnes se pracuje přímo s reálnými hodnotami a operátory se tomu přizpůsobují.

Mutace.

- **Gaussovská (normální) mutace** přičte ke složce normálně rozdělený šum: $x'_i = x_i + \sigma_i \cdot N(0, 1)$. Klíčovým parametrem je šířka kroku σ . Může být pevná, deterministicky klesající podle předem daného předpisu (např. $\sigma(t) = 1 - 0,9t/T$), adaptivní (pravidlo 1/5), nebo samoadaptivní, kdy se stane součástí genomu a evoluje spolu s jedincem (viz kap. 3).
- **Cauchyho mutace** má těžší chvosty, takže velké skoky nastávají častěji a jedinec snáze unikne z lokálního extrému. Používá se ve *fast EP*.
- **Uniformní mutace** losuje x'_i náhodně z intervalu $[a_i, b_i]$, a je tedy nezaujatá.
- Zvlášť je potřeba ošetřit meze prohledávaného prostoru. Nabízí se oříznutí (*clipping*), odraz (*reflection*), periodické zabalení (*wrapping*) nebo převzorkování, dokud výsledek nepadne dovnitř.

Křížení. Rozlišujeme dva typy:

- **Strukturální (diskrétní) křížení**, tedy 1-bodové a uniformní, hodnoty jen přeskupuje mezi rodiči. Žádné nové číslo nevznikne a potomek proto leží ve vrcholu hyperkvádrů daného rodiče.
- **Aritmetické křížení** hodnoty kombinuje, takže potomek může nést čísla, která ani jeden rodič neměl.

Aritmetické křížení (*arithmetic crossover*)

Pro rodiče $\vec{x}, \vec{y}$ a parametr $\alpha \in (0, 1)$ vzniká potomek jako

$$\vec{z} = \alpha \vec{x} + (1 - \alpha) \vec{y} \quad (\text{konvexní kombinace}).$$

Varianty se liší tím, kolika složek se kombinace týká. *Jednoduché* křížení kombinuje všechny složky a je nejběžnější, *jednoduché aritmetické* vybere náhodně jedinou složku a *celé/single* se kombinuje s jednobodovým křížením, takže aritmeticky se míchá teprve za bodem řezu. Pro $\alpha = 1/2$ vyjde prostý průměr rodičů.

Příklad. $\vec{x} = (2,0; -1,0; 4,0)$, $\vec{y} = (0,0; 3,0; 1,0)$, $\alpha = 0,25$:

$$\vec{z} = 0,25 \vec{x} + 0,75 \vec{y} = (0,5; 2,0; 1,75).$$

Čistě aritmetické křížení má ovšem podstatnou nevýhodu. Potomci leží uvnitř konvexního obalu rodičů, takže rozptýl populace nutně klesá a populace se „smršťuje“. Proto se používají rozšířené varianty, které dovolí potomkovi vyjít i ven:

- **BLX- α** (*blend crossover*) losuje z_i uniformně z intervalu $[\min - \alpha I, \max + \alpha I]$, kde $I = |x_i - y_i|$; typicky se volí $\alpha = 0,5$. Interval je tedy oproti rodičům rozšířený na obě strany a potomek smí ležet i vně nich.
- **SBX** (*simulated binary crossover*) napodobuje chování jednobodového binárního křížení v reálném prostoru: potomci jsou rozděleni symetricky kolem rodičů s hustotou, kterou řídí parametr η . Je to standardní operátor v NSGA-II.

1.3.3 Permutační reprezentace

Jedinec je permutace π na $\{1, \dots, n\}$, což je přirozený zápis pro obchodního cestujícího (TSP), rozvrhování, přiřazovací úlohy nebo uspořádání operací. Klasické operátory zde ale selhávají, protože výsledek jednobodového křížení dvou permutací obecně permutace není. Potřebujeme operátory,

které zachovávají přípustnost a zároveň dědí něco smysluplného. Co přesně se má dědit, závisí na úloze:

operátor	co zachovává	typické použití
PMX	absolutní pozice měst	úlohy s významnou pozicí
OX	relativní pořadí	TSP (cyklické cesty)
CX	absolutní pozice (cykly)	úlohy s významnou pozicí
ER	hrany (sousednosti)	TSP — nejlepší kvalita

PMX — částečně mapované křížení

PMX (*partially mapped crossover*, Goldberg)

Dvoubodový operátor. (1) Vyber úsek mezi dvěma body řezu a vyměň jej mezi rodiči. (2) Z vyměněných úseků odvoď *mapování* odpovídajících si hodnot, tedy dvojic, které se v úsecích ocitly nad sebou. (3) Zbylé pozice zkopíruj z původního rodiče; kde by vznikl konflikt, protože hodnota už v potomkovi je, použij mapování, a to případně i opakovaně.

Příklad. $P_1 = (123|4567|89)$, $P_2 = (452|1876|93)$.

1. Výměna úseků: $O_1 = (\dots|1876|\dots)$, $O_2 = (\dots|4567|\dots)$.
2. Mapování z úseků: $1 \leftrightarrow 4$, $8 \leftrightarrow 5$, $7 \leftrightarrow 6$, $6 \leftrightarrow 7$.
3. Doplnění bezkonfliktních pozic z P_1 do O_1 : $O_1 = (.23|1876|.9)$ (hodnoty 1 a 8 by kolidovaly).
4. Konflikty vyřešíme mapováním: $1 \rightarrow 4$, $8 \rightarrow 5$, tedy $O_1 = (423|1876|59)$. Analogicky $O_2 = (182|4567|93)$.

OX — křížení zachovávající pořadí

OX (*order crossover*, Davis)

Dvoubodový operátor zachovávající *relativní pořadí*. (1) Zkopíruj úsek mezi body řezu z prvního rodiče. (2) Ze druhého rodiče čti hodnoty cyklicky počínaje pozicí za druhým bodem řezu a vynech ty, které už v potomkovi jsou. (3) Zbylé hodnoty vkládej do volných pozic potomka, opět cyklicky od pozice za druhým bodem řezu.

Příklad. $P_1 = (123|4567|89)$, $P_2 = (452|1876|93)$.

1. O_1 převezme úsek z P_1 : $O_1 = (\dots|4567|\dots)$.
2. Z P_2 čteme cyklicky od pozice 8: 9, 3, 4, 5, 2, 1, 8, 7, 6.
3. Vyškrtneme hodnoty už obsažené (4, 5, 6, 7): zbývá 9, 3, 2, 1, 8.
4. Doplnujeme od pozice 8 cyklicky (pozice 8, 9, 1, 2, 3): $O_1 = (218|4567|93)$.
5. Symetricky (úsek z P_2 , pořadí z P_1): $O_2 = (345|1876|92)$.

Princip je tedy jednoduchý: vnitřní úsek se převezme beze změny a zbytek se doplní v relativním pořadí druhého rodiče, jen se vynechají duplicity. Pro TSP je OX vhodnější než PMX, protože u okružní jízdy nezáleží na absolutní pozici města, ale na pořadí, v jakém se města projíždějí.

CX — cyklické křížení

CX (*cyclic crossover*, Oliver)

Zachovává *absolutní pozici*: každý gen potomka pochází ze stejné pozice u některého z rodičů. Nejprve se sestrojí cykly. Začni na pozici 1 a vezmi hodnotu z P_1 ; podívej se, jaká hodnota je na téže pozici v P_2 , najdi ji v P_1 a pokračuj stejným způsobem, dokud se cyklus neuzavře. Sudé cykly potom ber z jednoho rodiče, liché z druhého.

Příklad. $P_1 = (123456789)$, $P_2 = (412876935)$. Začneme hodnotou 1 z P_1 na pozici 1. Pod ní je v P_2 hodnota 4, tu najdeme v P_1 na pozici 4 $\Rightarrow$ přidáme; pod ní je 8, ta je v P_1 na pozici 8 $\Rightarrow$ přidáme; pod ní je 3 (pozice 3), pod ní 2 (pozice 2) a pod 2 je 1, čímž se cyklus uzavřel. Máme $O_1 = (1\ 2\ 3\ 4\ _ _ _ 8\ _)$; zbytek doplníme z P_2 : $O_1 = (123476985)$, analogicky $O_2 = (412856739)$.

ER — rekombinace hran Předchozí tři operátory spojuje jedno pozorování: PMX, OX i CX zachovávají jen asi 60 % hran rodičů. Pro TSP je ale právě *hrana* nositelem kvality, protože délka cesty je součtem délek hran. *Edge recombination* (Whitley) proto pracuje rovnou s nimi:

1. Pro každé město sestaví *seznam sousedů* z **obou** rodičů.
2. Začne v náhodném (nebo prvním) městě.
3. Dalším městem je ten dosud nenavštívený soused, který má *nejméně* zbývajících sousedů; heuristika za tím je, aby žádné město nezůstalo „osamocené“. Při shodě rozhodne náhoda.
4. Pokud seznam sousedů dojde prázdný, pokračuje se náhodným nenavštíveným městem. Nastává to jen v 1–1,5 % případů.

Vylepšená varianta **ER2** navíc značí symbolem # hrany, které se vyskytují v *obou* rodičích, a při výběru je upřednostňuje.

Příklad. $P_1 = (123456789)$, $P_2 = (412876935)$ (cyklické cesty). Seznamy sousedů: 1: 9, 2, 4; 2: 1, 3, 8; 3: 2, 4, 9, 5; 4: 3, 5, 1; 5: 4, 6, 3; 6: 5, 7, 9; 7: 6, 8; 8: 7, 9, 2; 9: 8, 1, 6, 3. Startujeme v 1. Kandidáty jsou 9 (4 sousedi), 2 (3) a 4 (3), takže 9 vypadává a mezi 2 a 4 losujeme; vyjde 4. Sousedé 4 jsou 3 a 5, přičemž 3 má ještě 3 volné sousedy a 5 jen 2, bereme tedy 5, a tak dále. Výsledkem je např. (145678239).

Technická poznámka: korektně se navštívené město nejprve odstraní ze *všech* seznamů a teprve pak se počítají zbývající sousedé. Výše jsou u startu uvedeny počty ještě *před* odebráním jedničky (po odebrání by bylo 9:3, 2:2, 4:2). Na pořadí kandidátů to zde nic nemění, ale při vlastním počítání na papíře je třeba postupovat takto, jinak vycházejí jiné remízy.

Mutace permutací Mutace musí rovněž zachovat přípustnost, takže se omezuje na přeuspořádání již přítomných hodnot:

- **Swap** prohodí dvě náhodně vybraná města.
- **Insert** vyjme jedno město a vloží je jinam, čímž se zbytek posune.
- **Inverze (2-opt krok)** obrátí souvislý úsek cesty. Pro TSP je nejúčinnější, protože mění jen dvě hrany.
- **Scramble** náhodně přeháze souvislý úsek.
- Používá se také **výměna podcest** a heuristické mutace, tedy 2-opt, 3-opt a Lin-Kernighan.

1.3.4 Další reprezentace

Výčtem výše repertoár nekončí. Jedincem může být strom (odst. 1.4), graf nebo DAG, jak je tomu v Cartesian GP a v neuroevoluci, případně matice, kterou se popisují rozvrhy i TSP zapsaný jako matice předchůdců. Dále se používají konečné automaty v evolučním programování, seznamy pravidel v systémech LCS (odst. 6.1) a celí agenti umělého života.

1.4 Genetické a evoluční programování

1.4.1 Historie

Evoluci lze pustit i na struktury, které samy něco počítají, a ten nápad je podstatně starší než většina evolučních algoritmů. Myšlenku evoluce programů zmínil už Alan Turing v 50. letech. Forsyth ji v roce 1980 aplikoval v systému BEAGLE na rozpoznávání vzorů, Michael Cramer (1985) poprvé popsal stromové jedince a John Koza formuloval v letech 1989–1992 stromové genetické programování v podobě, v jaké je známe dnes; nechal si je také patentovat.

Genetické programování (*genetic programming*, GP)

Evoluční algoritmus, jehož jedinci jsou **spustitelné struktury**, tedy programy, výrazy, obvody nebo modely. Fitness se určuje *spuštěním* jedince na testovacích datech. Velikost ani tvar jedince nejsou pevné a mění se v průběhu evoluce.

Algoritmus 3 Obecné schéma GP

- 1: vygeneruj počáteční populaci náhodných programů
 - 2: **while** není nalezeno dost dobré řešení **do**
 - 3: ohodnot programy jejich **spuštěním** na trénovacích datech
 - 4: selekce podle fitness (typicky turnaj)
 - 5: křížení dvou programů, mutace programů
 - 6: vytvoř novou populaci
 - 7: **end while**
-

1.4.2 Stromové GP

Reprezentace. Program je *syntaktický strom*. Abychom takové stromy mohli generovat, definujeme dvě množiny:

- **Množina terminálů** T obsazuje listy stromu. Patří do ní proměnné, konstanty a funkce bez argumentů, například čtení senzoru.
- **Množina neterminálů (funkcí)** F obsazuje vnitřní uzly a u každého prvku je určena jeho *arita*. Bývají to aritmetické operace, logické spojky, podmínky nebo cykly.

Na obě množiny se kladou dva požadavky. *Uzavřenost (closure)* žádá, aby každá funkce uměla zpracovat libovolný výstup libovolné jiné; odtud pochází „chráněné dělení“ %, které pro nulového dělitele vrací 1. *Dostatečnost (sufficiency)* pak žádá, aby z $T \cup F$ bylo řešení vůbec vyjádřitelné.

Příklad: strom $(+ (* x x) (\sin y))$ reprezentuje výraz $x^2 + \sin y$.

Inicializace. Počáteční populaci lze vygenerovat několika způsoby, které se liší tvarem výsledných stromů:

- **Grow** losuje uzly z $T \cup F$, dokud se nedosáhne limitu hloubky nebo počtu uzlů. Vznikají tak nepravidelné stromy různé velikosti.
- **Full** losuje do dané hloubky jen z F a na poslední úrovni jen z T , takže vznikají plné, pravidelné stromy.
- **Ramped half-and-half** (Koza) vytvoří polovinu populace metodou grow a polovinu metodou full, přičemž hloubky rozloží rovnoměrně (např. 2–6). Je to standardní volba. Vytváří spíše „košaté“ pravidelné tvary, což vyhovuje úlohám se symetriemi, kde jsou všechny vstupy podobně důležité.
- **Uniformní vzorkování** všech možných stromů dává nepravidelnější tvary, protože se některé terminály ocitnou blíže ke kořeni.
- **Seeding** vkládá do počáteční populace částečná či expertní řešení. Evoluci to zrychlí, hrozí ale předčasná konvergence.

Křížení.

- **Výměna podstromů** je standardní Kozovo křížení: v každém rodiči se zvolí náhodný uzel a podstromy pod zvolenými uzly se vymění. Neterminály mívají vyšší pravděpodobnost výběru, typicky 90 % neterminál a 10 % terminál, protože jinak by se často vyměňovaly jen listy.
- **Homologní křížení** se snaží zachovat pozici genetického materiálu. Nejprve se najde *společná oblast* C , a to procházením od kořene, dokud se shoduje arita uzlů:
 - *Jednobodové* křížení volí bod křížení ze společné oblasti.
 - *Uniformní* (GPUD, Poli a Langdon) vymění každý uzel společné oblasti s konstantní pravděpodobností. Uzly *uvnitř* C se vymění bez dopadu na jejich podstromy, zatímco uzly na *hranici* C se vymění i s podstromy. V raných fázích se tak vyměňují velké podstromy a s konvergencí populace se operátor stává stále lokálnější.
- **Křížení zachovávající kontext** (*context preserving*) identifikuje uzel jeho *souřadnicemi*, tedy posloupností odboček na cestě od kořene (např. (2, 1, 3, 1)). *Silná* varianta povoluje křížení jen

mezi uzly se *stejnými* souřadnicemi, *slabá* je volnější.

Mutace. V GP je nutné (nikoli jen užitečné) používat *více typů* mutací, protože každý typ zasahuje strom jinak:

- **Podstromová mutace** nahradí náhodný podstrom novým náhodným.
- **Size-fair** podstromová mutace navíc losuje velikost nového podstromu tak, aby odpovídala odstraněnému.
- **Mutace uzlu** (*node replacement*) zamění uzel za jiný *stejně arity*.
- **Hoist** nahradí jedince jeho vlastním podstromem, a tím ho zmenší.
- **Shrink** nahradí podstrom terminálem, což jedince zmenší také.
- **Permutační mutace** prohodí podstromy jednoho neterminálu.
- **Mutace konstant** řeší to, že GP tradičně špatně dolaďuje číselné hodnoty. Pomáhá aritmetická mutace konstant nebo rovnou lokální optimalizace *všech* konstant ve stromu, ať už hill climbingem, nebo gradientní metodou; jde tedy o memetický přístup.

Fitness a selekce. Fitness se odvozuje od toho, co má program dělat: u symbolické regrese je to chyba na trénovacích datech, jinde počet správných odpovědí nebo kvalita chování v simulaci. Selektce bývá turnajová. Protože se program učí z dat, je riziko přeučení stejné jako u strojového učení a řeší se stejně, tedy validační množinou a penalizací složitosti.

1.4.3 Bloat

Se stromovou reprezentací je spojený jeden charakteristický problém, který u pevně dlouhého genomu vůbec nemůže nastat.

Bloat

Bloat je tendence programů v GP růst do velikosti bez odpovídajícího zlepšení fitness. Typicky se v programech hromadí *introny*, tedy části kódu, které nemají vliv na výstup; příkladem je `(+ x 0)` nebo `(if false ...)`.

Důvody jsou dva a oba jsou docela pochopitelné. V prostoru programů je mnohem víc velkých programů se stejným chováním než malých, takže už samotný neutrální drift tlačí k růstu. Navíc introny chrání jedince před destruktivním křížením, a tak se „svezou“ s dobrým kódem.

V přírodě růst naráží na fyzikální a energetické limity, které ho samy zastaví, v GP ale žádná taková brzda není a s bloatem se musí bojovat explicitně:

- **Tvrdé limity** na hloubku či počet uzlů. Potomek, který limit překročí, se zahodí nebo nahradí rodičem.
- **Penalizace ve fitness** (*parsimony pressure*) odečte od fitness člen úměrný velikosti: $f' = f - \alpha \cdot \text{velikost}$. *Lexikografická* varianta velikost nezapočítává, jen při shodné fitness nechá vyhrát menšího jedince.
- **Anti-bloat operátory** hlídají, aby křížení a mutace jedince příliš nezvětšovaly (to zajišťují *size-fair* varianty), a doplňují je speciální zmenšující mutace *hoist* a *shrink*.
- **Vícekritériální přístup** bere velikost jako druhé optimalizované kritérium (odst. 6.4).

1.4.4 ADF a typované GP

Stromové GP se dá výrazně posílit tím, že se programu dovolí mít vlastní podprogramy.

ADF (*automatically defined functions*)

Podprogramy evolvované společně s hlavním programem. Jsou nutnou podmínkou **modularity**, vyšší složitosti a schopnosti zachytit symetrie úlohy. Jedinec se skládá z větve **main** a jedné či více větví ADF; každá ADF je charakterizována svou *aritou* a může mít omezenou (jinou) množinu

terminálů a neterminálů. Volání ADF se v hlavním programu chová jako **nový neterminál** dané arity. Genetické operátory pracují na větvích **odděleně**, takže se main kříží s mainem a ADF_1 s ADF_1 .

Myšlenku lze rozšířit na automaticky definované smyčky, iterace a rekurzi. Alternativou je *ko-evoluce* dvou populací, hlavních programů a podprogramů (odst. 3.3).

Vynucení struktury. Prohledávaný prostor se dá také omezit tím, že se zakážou nesmyslné stromy. *Silně typované GP* dává každému uzlu typ vstupů a výstupů a operátory smějí vytvářet jen typově korektní stromy. *Gramatické GP* místo toho popíše přípustné tvary stromů bezkontextovou gramatikou. Obojí redukuje prohledávaný prostor na smysluplné programy.

1.4.5 Lineární a grafové GP

Strom není jediná struktura, do které se program dá zapsat. Následující varianty GP se liší právě tím, jak jedince reprezentují, a s tím se mění i celá sada operátorů.

Lineární GP. Program je posloupnost instrukcí, často ve strojovém nebo byte kódu, které pracují nad registry. Výhod je několik: operátory jsou jednoduché, protože se kříží a mutuje jako u vektorů, interpretace je rychlá a celá reprezentace je přirozeně blízka skutečnému hardwaru. Nevýhodou je vysoké riziko vzniku nesmyslných programů. Lineární GP je oblíbené v umělém životě a při evoluci herních botů. Slavným příkladem je **Tierra** (T. Ray), simulace umělého ekosystému, ve které jsou jedinci sebekopírující programy ve strojovém kódu soutěžící o paměť a procesorový čas; emergentně v ní vznikli paraziti, hyperparaziti a obrana proti nim. Následovníky jsou Avida a Avida-ED.

Grafové GP. Program je obecný, často acyklický graf. Původně šlo o rozšíření stromového GP na paralelní programy, ale ukázalo se, že grafy dobře popisují také elektrické obvody, konečné automaty, neuronové sítě a plány. Problémem jsou složité operátory, protože není zřejmé, jak křížit obecné grafy.

Kartézské GP (CGP). [♦ nad rámec slidů] Elegantní obchvat: DAG se reprezentuje svým rovinným obrazem v 2D mřížce se souřadnicemi uzlů (fenotyp), zatímco genotyp je **lineární** posloupnost celých čísel, která pro každý uzel popisuje jeho funkci (index do tabulky funkcí) a jeho vstupy a nakonec vyjmenuje výstupní uzly grafu. Z toho plyne několik vlastností:

- Mutace je bodová mutace lineárního genomu. Musí se přitom hlídat, aby vznikaly jen platné funkce a platná propojení, což znamená omezení hloubky spojů.
- Malá změna genotypu může mít velký dopad na fenotyp. Naopak řada mutací je *neutrální*, protože uzly nepřipojené k výstupu tvoří přirozené introny, což CGP prospívá.
- Křížení je málo důležité. Naivní jednobodové je příliš destruktivní, a tak se používá aritmetická varianta a řezy na hranicích uzlů.
- Selektce je obvykle $(1 + \lambda)$ -ES s $\lambda = 4$.
- Genom má konstantní délku, takže bloat je omezen už samotnou mřížkou.

Vývojové (developmental) GP. Místo přímé evoluce grafu se evoluuje *strom-program*, který graf postupně staví. Začíná se „embryem“ a aplikují se operace pro strukturu (sériové či paralelní zapojení, třicestné dělení) a pro elementy (vložit rezistor, vložit kondenzátor). Takto postupoval Koza při evoluci elektrických obvodů, kde se podařilo znovuobjevit patentovaná zapojení, Gruau ve svém *cellular encoding* pro neuronové sítě a AutoML při evoluci modelů scikit-learn.

1.4.6 Gramatická evoluce

Gramatiku lze posunout ještě dál než u gramatického GP: nemusí jen filtrovat přípustné stromy, může být přímo tím, co z genomu program odvozuje.

Gramatická evoluce (*grammatical evolution*, GE)

Genotyp je **lineární seznam celých čísel** proměnné délky, fenotyp je program odvozený z bez-kontextové gramatiky. Číslo z genomu určuje, které pravidlo se použije pro expanzi nejlevějšího neterminálu:

$$\text{pravidlo} = (\text{gen} \bmod \text{počet pravidel pro tento neterminál}).$$

Použití modula činí kódování **robustním**, protože každý genom je interpretovatelný. A protože je genom lineární, lze používat **standardní operátory GA**, což je hlavní praktická výhoda celého přístupu. Gramatika navíc zaručuje syntaktickou správnost výsledku.

Nevýhodou je nelokálnost: malá změna genotypu, zejména na jeho začátku, může zcela změnit fenotyp (*ripple effect*). Nedokončí-li se *derivate*, čte se genom znovu od začátku (*wrapping*), a to s limitem na počet obalení.

1.4.7 Evoluční programování

K evoluci programů se dá dojít i úplně jinou cestou, než jakou šel Koza, a historicky se tak stalo dříve.

Evoluční programování (*evolutionary programming*, EP)

Větev EA (L. Fogel, 1965, tedy starší než GA), jejímž cílem bylo evolvovat „umělou inteligenci“: agenta, který predikuje stav prostředí a volí vhodnou akci. Agent je reprezentován **konečným automatem** a platí **genotyp = fenotyp**, žádné zvláštní kódování se tedy nepoužívá. **Křížení se nepoužívá** vůbec a veškerá variabilita pochází z mutací.

EP nad automaty. Prostředí je posloupnost symbolů konečné abecedy. Automat dostává symboly na vstup a jeho výstup se porovnává s následujícím symbolem posloupnosti; fitness je úspěšnost predikce, měřená absolutní nebo střední kvadratickou chybou, často s penalizací za velikost automatu. Mutovat se dá pěti způsoby: změnou výstupního symbolu, změnou cílového stavu přechodu, přidáním stavu, odebráním stavu a změnou počátečního stavu. Novými automaty se nahrazuje polovina populace.

Slavný příklad — predikce prvočísel. Cílem je predikovat posloupnost

$$111010100010100010100010000010\dots,$$

kde 1 znamená „toto číslo je prvočíslo“; automat se tedy má naučit Eratosthenovo síto. Postupuje se iterativně: nejprve se učí krátká posloupnost, pak se prodlužuje o jeden symbol. Fitness dává bod za každý uhodnutý symbol a odečítá $0,01 \cdot N$ za počet stavů. Výsledek je poučný. Automaty se postupně zjednodušovaly, až nakonec vznikl jednostavový automat generující stále 0. Prvočísla jsou totiž řídká, takže konstantní odpověď „není prvočíslo“ má vysokou úspěšnost. Je to klasická ukázka toho, jak evoluce zneužije špatně navrženou fitness.

EP dnes. Z původní podoby zůstalo několik zásad:

- Kódování má být *přirozené* pro úlohu, obvykle tedy genotyp = fenotyp.
- Používá se sada „chytrých“, doménově specifických mutací a žádné křížení.
- Rodičovská selekce je nastavena tak, že každý jedinec je právě jednou rodičem.
- Environmentální selekce probíhá turnajem mezi rodiči a potomky.
- EP nad reálnými vektory obsahuje, stejně jako ES, rozptyly mutací přímo v genomu ($\sigma'_j = \sigma_j(1 + \alpha N(0, 1))$, poté $x'_j = x_j + \sigma'_j N(0, 1)$). EP a ES tak prakticky splynuly.

1.4.8 Je křížení k něčemu? Experiment s bezhlavým kuřetem

Tradiční obhajoba křížení stojí na tom, že si rodiče vyměňují stavební bloky. Kritika, typická pro EP, oponuje, že křížení je jen podivná velká náhodná mutace, která navíc nerespektuje kódování. Spor se dá rozhodnout experimentem.

Headless chicken test (Jones, 1995)

Porovnáme standardní křížení s *náhodným* křížením, kde se jedinec kříží s **náhodně vygenerovaným** řetězcem. Je-li výsledek stejný nebo lepší, pak zkoumaný operátor ve skutečnosti nefunguje jako křížení, ale jako makromutace: „nenazývejme bezhlavé kuře kuřetem, i když má mnoho kuřecích rysů“.

Interpretace je přímočará. Existují-li v úloze jasné stavební bloky, je právě křížení lepší. Vyhraje-li náhodné křížení, znamená to, že v dané úloze nebo v daném kódování stavební bloky nejsou; na vině je pak špatné kódování, špatný operátor, nebo prostě úloha, pro kterou se křížení nehodí.

1.5 Shrnutí ke zkoušce

- Evoluční algoritmus je populační stochastické prohledávání a běží v cyklu *inicializace* $\rightarrow$ *ohodnocení* $\rightarrow$ *rodičovská selekce* $\rightarrow$ *rekombinace* $\rightarrow$ *mutace* $\rightarrow$ *ohodnocení* $\rightarrow$ *environmentální selekce*.
- Variace, tedy křížení a mutace, vytvářejí v populaci diverzitu, kdežto selekce ji odebírá a tím směřuje hledání. Navrhnout evoluční algoritmus proto znamená najít rovnováhu mezi explorací a exploatací.
- Seleční schémata se liší vyvíjeným tlakem a rozptylem počtu potomků. **Ruleta** vybírá rodiče s pravděpodobnostmi $p_i = f_i / \sum f_j$, má vysoký rozptyl a je citlivá na škálování fitness. **SUS** zachovává stejné střední hodnoty, ale rozptyl srazí na minimum. **Turnaj** řídí tlak velikostí k , je invariantní k transformacím fitness a vystačí si bez explicitní fitness. **Pořadová** selekce drží tlak konstantní, zatímco u **Boltzmannovy** tlak s časem roste.
- Elitismus zaručuje, že nejlepší dosažená fitness už nikdy neklesne, tedy její monotonii. Generační a steady-state model se od sebe liší tím, jak velkou část populace v jednom kroku nahradí.
- Věta No Free Lunch říká, že univerzálně nejlepší algoritmus neexistuje. Důsledkem je specializace reprezentace i operátorů na konkrétní doménu.
- Historické větve GA, ES, EP a GP i jejich charakteristické volby shrnuje srovnávací tabulka a je dobré umět je vyjmenovat z paměti.
- SGA kombinuje binární řetězce, ruletovou selekci, 1-bodové křížení, bit-flip mutaci a generační náhradu. Nastavují se u něj parametry n , $p_C \approx 0,7$ a $p_M \approx 1/m$.
- Reálná čísla se do binárního řetězce kódují vztahem $x = a + z(b - a)/(2^k - 1)$. Sousední hodnoty se přitom mohou lišit v mnoha bitech naráz a tyto Hammingovy útesy odstraňuje Grayův kód.
- Operátory křížení se liší tím, jak silně narušují schémata. Jednobodové rozbije schéma s pravděpodobnostmi $d(S)/(m - 1)$, dvoubodové nemá okrajový bias a uniformní je nejvíce disruptivní, zato bez pozičního biasu.
- Mutace má v GA sekundární roli: brání nevratné ztrátě alel. Inverze je historický operátor, kterým se evolvovalo uspořádání genů.
- Škálování fitness (lineární, sigma truncation, mocninné, pořadové a Boltzmannovo) opravuje dvě slabiny fitness-proporcionální selekce: dominanci supermanů na začátku běhu a ztrátu tlaku na jeho konci.
- Reprezentace spolu s operátory určuje fitness krajinu. Pro nominální atributy přicházejí v úvahu jen nezaujaté mutace, pro ordinální i zaujaté (creep).
- V reálné reprezentaci se mutuje gaussovsky, $x' = x + \sigma N(0, 1)$. Aritmetické křížení $\alpha \vec{x} + (1 - \alpha) \vec{y}$ zmenšuje rozptyl populace, a proto se místo něj používá BLX- α a SBX.
- Permutační křížení se rozlišují podle toho, co po rodičích dědí: PMX přenáší pozice a zbytek doplní mapováním, OX relativní pořadí, CX absolutní pozice pomocí cyklů a ER hrany, což je pro TSP nejlepší volba. PMX a OX je nutné umět předvést na konkrétním osmiprvkovém příkladu!

- Permutace se mutují operátory swap, insert, inverze (2-opt) a scramble.
- Stromové GP podle Kozy staví programy z terminálů a neterminálů, které musí splňovat uživatelskou a dostatečnost. Populace se inicializuje metodou grow, full nebo ramped half-and-half, křížením je výměna podstromů (neterminály se vybírají s vyšší pravděpodobností), mutací existuje více typů včetně mutace konstant a fitness se získá spuštěním programu.
- Bloat je růst velikosti jedinců bez zlepšení fitness, na kterém se podílejí introny. Vysvětlují ho tři teorie: replication accuracy, removal bias a crossover bias. Bránit se proti němu dá limity velikosti, penalizací (parsimony pressure), anti-bloat operátory (hoist, shrink, size-fair) a vícekritériálním přístupem.
- ADF jsou evolvované podprogramy, které do GP vnášejí modularitu; operátory pak pracují na jednotlivých větvích odděleně. Typované GP a gramatické GP omezují prohledávaný prostor na smysluplné programy.
- Z variant stojí za zapamatování lineární GP (byte kód, Tierra), grafové GP, Cartesian GP (lineární genotyp $\rightarrow$ 2D DAG, bodová mutace, (1 + 4)-ES) a vývojové GP, kde strom teprve staví výsledný graf, tedy obvod nebo síť.
- Gramatická evoluce pracuje s lineárním genomem celých čísel, jimiž se pomocí modula vybírají pravidla gramatiky, a vzniklý derivační strom dává fenotyp. Kódování je robustní a operátory standardní jako v GA, mapování je ale nelokální.
- EP evoluuje konečné automaty, genotyp splývá s fenotypem, používá se jen mutace a turnajová selekce z rodičů i potomků. Učebnicovým příkladem je predikce prvočísel, na které se ukáže degenerace fitness.
- Headless chicken experiment ověřuje, zda křížení opravdu kombinuje stavební bloky, nebo zda funguje pouze jako makromutace.

2 Teorie schémat a pravděpodobnostní modely SGA

Tahle kapitola odpovídá na otázku, *proč* genetické algoritmy vlastně fungují. Nejprve přijde klasická Hollandova teorie schémat i s kritikou, která k ní neodmyslitelně patří, a potom exaktní Voseho model, který teorii schémat nahradil.

2.1 Teorie schémat

Teorie schémat je vůbec první pokus vysvětlit úspěch genetických algoritmů formálně, a ne jen odkazem na to, že se v praxi osvědčily. Zformuloval ji Holland (1975) a do širokého povědomí ji dostal Goldberg (1989). Dnešní pohled na ni je smířlivý, ale střízlivý: historicky je to zásadní krok, jako vysvětlení ale nestačí. U zkoušky se přesto probírá standardně a její kritika váží stejně jako věta sama.

2.1.1 Schéma, jeho řád a definující délka

Základním pojmem je schéma. Je to způsob, jak mluvit najednou o celé skupině jedinců, kteří se shodují v tom podstatném a liší se jen v pozicích, na nichž zrovna nezáleží.

Schéma (*schema*)

Nechť jedinec je slovo délky m nad abecedou $\{0, 1\}$. **Schéma** S je slovo délky m nad abecedou $\{0, 1, *\}$, kde $*$ znamená „na hodnotě nezáleží“ (*don't care*). Schéma reprezentuje množinu všech jedinců, kteří se s ním shodují na všech pevných pozicích. Obsahuje-li r hvězdiček, reprezentuje 2^r jedinců.

Například schéma $1 * 0 *$ zastupuje čtveřici $\{1000, 1001, 1100, 1101\}$, zatímco schéma $* * \dots *$ ze samých hvězdiček zastupuje rovnou celý prostor.

Řád, definující délka, fitness schématu

Pro schéma S délky m definujeme:

- **Řád** $o(S)$ je počet pevných pozic, tedy počet nul a jedniček ve schématu.
- **Definující délka** $d(S)$ je vzdálenost mezi první a poslední pevnou pozicí; pro schéma s jedinou pevnou pozicí vychází $d(S) = 0$.
- **Fitness schématu** $F(S, t)$ je průměrná fitness těch jedinců v populaci $P(t)$, kteří schéma S reprezentují. *Pozor:* tato hodnota závisí na aktuální populaci, nikoli jen na S a f .

Příklad. Schéma $S = 1**0*1$ délky $m = 6$ má pevné pozice 1, 4 a 6, takže jeho řád je $o(S) = 3$; první a poslední z nich dělí $d(S) = 6 - 1 = 5$ míst.

Kombinatorika schémat. Než přijde věta samotná, vyplatí se spočítat, s kolika schématy vlastně máme co do činění.

- Schémat délky m je celkem 3^m , protože na každé pozici může stát nula, jednička, nebo hvězdička.
- Jeden jedinec délky m je reprezentantem 2^m schémat: na každé pozici lze buď ponechat jeho bit, nebo tam napsat $*$.
- Populace n jedinců reprezentuje mezi 2^m a $n \cdot 2^m$ schématy. Kde přesně v tomto rozmezí leží, závisí na míře podobnosti jedinců.

2.1.2 Hollandova věta o schématech

Věta o schématech odhaduje zdola, kolik reprezentantů daného schématu bude v populaci o generaci později.

Věta 2.1 (Věta o schématech (TST), Holland 1975). *V jednoduchém genetickém algoritmu s fitness-proporcionální selekcí, jednobodovým křížením s pravděpodobností p_C a bitovou mutací s pravděpodobností p_M platí pro očekávaný počet reprezentantů schématu S v následující generaci:*

$$\mathbb{E}[C(S, t+1)] \geq C(S, t) \frac{F(S, t)}{\bar{f}(t)} \left[1 - p_C \frac{d(S)}{m-1} - p_M o(S) \right].$$

*Slovně řečeno: schémata, která jsou **krátká** (mají malou definující délku), **nízkého řádu** a **nadprůměrná**, se v po sobě jdoucích generacích množí exponenciálně.*

Odvození Označme $C(S, t)$ počet jedinců populace $P(t)$, kteří reprezentují S , dále $n = |P(t)|$ velikost populace a $\bar{f}(t)$ její průměrnou fitness. Odvození postupuje ve třech krocích, na každý operátor jeden.

Krok 1 — selekce. Pravděpodobnost, že ruleta vybere jedince v , je $p_s(v) = f(v)/F(t)$, kde $F(t) = \sum_{u \in P(t)} f(u)$. Sečtením přes všechny reprezentanty schématu dostaneme pravděpodobnost, že vybraný jedinec reprezentuje S :

$$p_s(S) = \sum_{v \in S \cap P(t)} \frac{f(v)}{F(t)} = \frac{C(S, t) F(S, t)}{F(t)}.$$

Nová populace vzniká n nezávislými tahy, takže

$$\mathbb{E}[C(S, t+1)] = n p_s(S) = C(S, t) \frac{F(S, t)}{\bar{f}(t)}, \quad \bar{f}(t) = F(t)/n.$$

Je-li schéma nadprůměrné o ε , tj. $F(S, t) = \bar{f}(t) (1 + \varepsilon)$, a drží-li se tento poměr v čase, dostáváme

$$C(S, t+1) = C(S, t)(1 + \varepsilon) \implies C(S, t) = C(S, 0) (1 + \varepsilon)^t,$$

tedy **exponenciální růst**, případně exponenciální pokles pro $\varepsilon < 0$. Samotná selekce tedy schéma rozmnožuje, ostatní operátory mu ubližují.

Krok 2 — křížení. Jednobodové křížení vybírá bod řezu z $m - 1$ možností. Schéma se *jistě* rozpadne, padne-li řez mezi jeho první a poslední pevnou pozici, což nastane s pravděpodobností nejvýše $d(S)/(m - 1)$. Slovo „nejvýše“ je podstatné: i při řezu uvnitř schématu může být schéma obnoveno, pokud druhý rodič nese na týchž pozicích tytéž hodnoty. Ke křížení navíc dochází jen s pravděpodobností p_C , takže schéma přežije s pravděpodobností

$$p_{\text{surv}}^{\text{cross}}(S) \geq 1 - p_C \frac{d(S)}{m - 1}.$$

Krok 3 — mutace. Mutace schématu neublíží, dokud se nezmění žádná z jeho $o(S)$ pevných pozic; co dělá na hvězdičkových pozicích, je schématu jedno:

$$p_{\text{surv}}^{\text{mut}}(S) = (1 - p_M)^{o(S)} \approx 1 - p_M o(S) \quad \text{pro } p_M \ll 1.$$

Složení. Zbývá obě přežití složit. Předpokládáme, že jsou oba destruktivní jevy (přibližně) nezávislé, a jejich součin zanedbáme:

$$\mathbb{E}[C(S, t + 1)] \geq C(S, t) \frac{F(S, t)}{\bar{f}(t)} \left[1 - p_C \frac{d(S)}{m - 1} - p_M o(S) \right]. \quad \blacksquare$$

Číselný příklad Na číslech je vidět, co věta vlastně tvrdí. Vezměme Goldbergovu populaci ze str. 7, tedy $m = 5$ a $n = 4$: jedinci 01101 (169), 11000 (576), 01000 (64) a 10011 (361) dávají $\bar{f} = 292,5$. Operátory nastavíme na $p_C = 0,7$ a $p_M = 0,001$.

schéma S	reprezentanti	$F(S)$	$o(S), d(S)$	přežití	$\mathbb{E}[C(S, t+1)]$
1****	11000, 10011	468,5	1; 0	$1 - 0 - 0,001 = 0,999$	$2 \cdot 1,602 \cdot 0,999 = 3,2$
1***1	10011	361	2; 4	$1 - 0,7 - 0,002 = 0,298$	$1 \cdot 1,234 \cdot 0,298 = 0,37$
0****	01101, 01000	116,5	1; 0	0,999	$2 \cdot 0,398 \cdot 0,999 = 0,80$

Tabulka ukazuje všechny tři možné osudy schématu. Krátké nadprůměrné schéma 1**** se rozšíří ze dvou zástupců zhruba na 3,2. Schéma 1***1 je sice také nadprůměrné ($F/\bar{f} = 1,23$), jenže jeho velká definující délka je fatální: křížení ho rozbije a očekáváme, že vymizí. A podprůměrné 0**** prostě ubývá. To je přesně obsah věty.

2.1.3 Hypotéza stavebních bloků

Věta o schématech říká, které kusy řešení v populaci přežívají. Hypotéza stavebních bloků z toho dělá tvrzení o tom, jak genetický algoritmus hledá.

Hypotéza stavebních bloků (*building blocks hypothesis*)

Genetický algoritmus hledá dobré řešení tak, že **rekombinuje krátká, nízkého řádu a nadprůměrná schémata**, tzv. *stavební bloky*, do stále lepších řešení. Goldberg to popsal metaforou: „tak jako dítě staví hrad z jednoduchých dřevěných kostek, tak GA hledá téměř optimální řešení skládáním stavebních bloků“.

Z hypotézy plynou tři praktické důsledky:

- **Na kódování záleží.** Geny, které spolu interagují, mají v řetězci ležet blízko sebe (*linkage*), protože jinak je křížení rozbíjí dřív, než se stihnou osvědčit. Odtud pocházejí snahy o inverzi, *messy GA*, *linkage learning* a algoritmy odhadu rozdělení (EDA).
- **Na velikosti populace záleží.** Populace musí obsahovat dostatek vzorků od každého stavebního bloku, jinak není z čeho skládat; tomu se věnuje teorie *population sizing*.
- **Předčasná konvergence škodí.** Se ztrátou diverzity zmizí z populace materiál, z něhož by šly chybějící bloky objevit.

2.1.4 Implicitní paralelismus a víceruký bandita

Populace vypadá zvenčí jako pouhých n řetězců. Kombinatorika schémat ale ukázala, že každý z nich je zároveň svědectvím o 2^m schématech, takže algoritmus s jedním vyhodnocením fitness posouvá odhad u obrovského množství schémat najednou. Právě tomu se říká implicitní paralelismus.

Implicitní paralelismus (*implicit parallelism*)

GA pracuje explicitně s n jedinci, ale implicitně přitom „zpracovává“ mnohem více schémat: mezi 2^m a $n \cdot 2^m$. Holland odhadl, že těch schémat, se kterými algoritmus zachází *užitečně* (rostou exponenciálně a nejsou příliš rozbíjena), je řádu n^3 .

Bývá to komentováno jako jediný případ, kdy kombinatorická exploze pracuje v náš prospěch, a ne proti nám.

Souvislost s úlohou o banditovi. Hollandova původní motivace mířila na „adaptivní plán“, který hledá rovnováhu mezi explorací a exploatací. Formálně se tahle úvaha dá postavit takto:

- **Dvouruký bandita.** Máme N mincí a automat se dvěma pákami, jejichž střední výplaty μ_1, μ_2 ani rozptyly neznáme. Alokujeme n mincí horší a $N - n$ lepší páce. Analytické řešení říká, že optimální strategie přiděluje momentálně vítězné páce *exponenciálně* více pokusů: $N - n^* = O(e^{cn^*})$.
- **GA jako bandita.** Věta o schématech tvrdí o genetickém algoritmu totéž: úspěšnějším schématům přiděluje exponenciálně více „slotů v populaci“, a řeší tedy dilema explorace a exploatace (téměř) optimálně.
- Původně se soudilo, že SGA hraje jedinou hru s 3^m pákami. Ve skutečnosti hraje mnoho her paralelně: schémata spolu soutěží vždy o konkrétní pevné pozice, takže schémata řádu k hrají hru s 2^k pákami.

2.1.5 Kritika teorie schémat

Věta o schématech je pravdivá, jenže se z ní dlouho vyvozovalo víc, než z ní plyne. Kritika míří na tři místa: na úlohy, kde stavební bloky vedou špatným směrem, na to, co vlastně GA o schématech měří, a na samotný tvar věty.

Kdy GA selhává — deceptivní úlohy Uvažujme funkci, v níž jsou nadprůměrná schémata

$$S_1 = (111* \dots *), \quad S_2 = (* \dots *11),$$

ale zároveň

$$f(111*****11) \ll f(000*****00).$$

Selekce vede algoritmus k jedničkovým blokům, přestože optimum leží úplně jinde, u samých nul. Takovým úlohám se říká **deceptivní** (*deceptive*, Goldberg 1987): dílčí stavební bloky nevedou po složení ke globálnímu optimu. Deception bývá navrhována jako míra obtížnosti úlohy pro EA, i když ne všichni s tím souhlasí.

Opačným testem jsou **Royal Road funkce** (Mitchell, Forrest, Holland). Fitness je tu součtem příspěvků za kompletní „bloky“ jedniček, například osm bloků po osmi bitech. Krajina je tedy ušitá přesně na míru hypotéze stavebních bloků a GA by na ní měl excelovat. Experiment dopadl opačně: jednoduchý GA prohrál s náhodně mutujícím horolezeckým algoritmem (RMHC). Na vině je *hitchhiking* („stopování“): spolu s nalezeným dobrým blokem se selekcí šíří i nahodilé „parazitní“ bity na ostatních pozicích a ničí diverzitu, kterou by algoritmus potřeboval k nalezení zbývajících bloků. Royal Road funkce jsou proto standardním empirickým protipříkladem k naivnímu čtení BBH.

Statické průměrné fitness — statická BBH Druhá výtka pochází od Grefenstetta (1991) a týká se toho, co vlastně GA o schématu ví. Běžný výklad BBH tvrdí, že GA konverguje k řešení s dobrou *skutečnou statickou* průměrnou fitness schématu, tedy s průměrem přes celý prostor. GA však odhaduje fitness schématu jen ze vzorků, které má právě v populaci, a ten odhad je vychýlený, a to hned ze dvou důvodů:

- **Kolaterální konvergence** (*collateral convergence*). Jakmile GA někam konverguje, přestává vzorkovat schémata rovnoměrně. Je-li například schéma $111* \dots *$ dobré, má po několika generacích skoro každý jedinec tento prefix. Pak je ale téměř každý vzorek schématu $***000 \dots$ zároveň vzorkem $111000 \dots$ a fitness prvního schématu vyjde zcela zkresleně.
- **Velký rozptyl fitness schématu**. Uvažujme funkci, pro kterou je $f(x) = 2$ pro $x \in 111* \dots *$, $f(x) = 1$ pro $x \in 0* \dots *$ a 0 jinak. Statická průměrná fitness schématu $1* \dots *$ vychází $\approx 1/2$, zatímco $F(0* \dots *) = 1$. GA však odhadne $F(1* \dots *) \approx 2$, protože v populaci převáží jedinci s prefixem 111. Vysoký rozptyl tedy vede k systematickému zkreslení a k tomu, že GA nevzorkuje schémata nezávisle.

Souhrn slabín Poslední skupina výhrad se netýká konkrétních úloh, ale samotného tvaru věty a předpokladů, za nichž byla odvozena:

- Věta je pouze **nerovnost** a navíc **jednokrokový** výsledek. Iterovat ji přes více generací lze jen za předpokladu, že poměr $F(S, t)/\bar{f}(t)$ zůstává konstantní, což typicky neplatí: mění se dynamicky obě jeho složky.
- Populace jsou **konečné a malé**, takže se uplatní výběrové chyby; exponenciální růst je jen střední hodnota, ne záruka.
- Věta zohledňuje jen **destruktivní** účinek operátorů a pomíjí jejich *konstruktivní* roli, přestože křížení může schéma i vytvořit.
- Soutěž schémat je **silně závislá**, zatímco páky bandity jsou nezávislé.
- Analýza je „on-line“: SGA maximalizuje průběžný výkon, hodí se tedy spíše pro adaptivní úlohy. Nejběžnější použití je přitom nalezení jednoho nejlepšího řešení, což je kritérium off-line.

Přes všechny výhrady zůstává TST užitečnou intuicí: *co je krátké, jednoduché a dobré, to se šíří*.

2.2 Pravděpodobnostní modely jednoduchého GA

Tam, kde teorie schémat nabízí jen nerovnost pro jediný krok, chtěli mít lidé rovnost. V 90. letech proto vznikly *exaktní* modely SGA (Vose, Liepins, Nix, Whitley), které přibližnou teorii schémat nahradily. Program byl jasný: přesně popsat, jak vypadá populace, najít zobrazení, které jednu populaci převádí na následující, prozkoumat jeho vlastnosti a z nich odvodit asymptotické chování SGA. Přístupy jsou dva a liší se v tom, jak velkou populaci připustí. **Nekonečné populace** dávají deterministický dynamický systém, **konečné populace** Markovův řetězec.

2.2.1 Zjednodušený SGA

Aby byl SGA analyzovatelný, ještě se zjednoduší. V každém kroku se vyberou dva rodiče, s pravděpodobností p_C se zkříží a **jeden z potomků se zahodí**; ten zbylý zmutuje a vloží se do nové populace. Zahození jednoho potomka není libovůle: díky němu vzniká každý jedinec nové populace nezávisle na ostatních.

2.2.2 Formalizace populace

Každý binární řetězec délky l ztotožníme s číslem $i \in \{0, 1, \dots, 2^l - 1\}$, například $00000111 \equiv 7$. Celou populaci v čase t pak popíšou dva vektory délky 2^l :

Vektor populace a vektor selekce

- $p(t) = (p_0(t), \dots, p_{2^l-1}(t))$, kde $p_i(t)$ je *relativní četnost* řetězce i v populaci. Platí $p_i \geq 0$ a $\sum_i p_i = 1$, takže $p(t)$ leží v simplexu Λ .
 - $s(t) = (s_0(t), \dots, s_{2^l-1}(t))$, kde $s_i(t)$ je pravděpodobnost, že selekce vybere řetězec i .
- Vektor p popisuje *složení* populace, vektor s *selekční pravděpodobnosti*.

2.2.3 Operátor $\mathcal{G}$

Cílem je najít operátor $\mathcal{G} : \Lambda \rightarrow \Lambda$ takový, že

$$p(t+1) = \mathcal{G}(p(t)),$$

a rozložit ho na dvě části, $\mathcal{G} = \mathcal{M} \circ \mathcal{F}$, kde $\mathcal{F}$ odpovídá selekci (fitness) a $\mathcal{M}$ tzv. míchání (*mixing*), tedy křížení a mutaci dohromady. Jakmile takový operátor máme, stačí ho iterovat z počáteční populace a známe úplné chování SGA.

Část $\mathcal{F}$ — selekce. Definujme diagonální matici $\mathbf{F}$ s $\mathbf{F}_{ii} = f(i)$ a $\mathbf{F}_{ij} = 0$ pro $i \neq j$. Fitness-proporcionální selekce je pak přesně

$$s(t) = \frac{\mathbf{F} p(t)}{\sum_j \mathbf{F}_{jj} p_j(t)}.$$

Čitatel $(\mathbf{F}p)_i = f(i)p_i$ je „váha“ řetězce i v populaci, jmenovatel je normalizační konstanta, totiž průměrná fitness populace.

Vypustíme-li nyní křížení a mutaci, je nová populace prostě n nezávislých tahů podle rozdělení $s(t)$, takže $\mathbb{E}[p(t+1)] = s(t)$. Dosazením do definice selekce dostáváme $\mathbb{E}[s(t+1)] \propto \mathbf{F} s(t)$, takže iterování $\mathcal{F}$ není nic jiného než (až na normalizaci) opakované násobení maticí $\mathbf{F}$, tedy *mocninná metoda*. Ta konverguje k vlastnímu vektoru příslušnému největší diagonální hodnotě, čili k populaci složené výhradně z kopií nejlepšího jedince *přítomného v počáteční populaci*. U konečných populací způsobují statistické chyby odchylky od střední hodnoty, a čím větší populace, tím menší odchylka. Nekonečné populace jsou exaktní, byť nepraktické.

Mini-příklad. Pro $l = 2$ máme čtyři řetězce 00, 01, 10, 11, tedy indexy 0, 1, 2, 3. Nechť $f = (1, 2, 3, 4)$ a populace osmi jedinců obsahuje 2×00 , 4×01 , 1×10 , 1×11 , tj. $p = (0,25; 0,5; 0,125; 0,125)$. Pak $\mathbf{F}p = (0,25; 1,0; 0,375; 0,5)$, součet je 2,125 (= průměrná fitness populace) a

$$s = (0,118; 0,471; 0,176; 0,235).$$

Podíl nejlepšího řetězce 11 vzrostl z 12,5 % na 23,5 % a podíl nejhoršího 00 klesl z 25 % na 11,8 %, a to přesně o faktor $f(i)/\bar{f}$, jak víme z věty o schématech.

Část $\mathcal{M}$ — míchání. Zavedeme funkci

$$r(i, j, k) = \Pr[\text{potomek } k \text{ vznikne z rodičů } i, j],$$

která v sobě zahrnuje jak křížení (přes všechny body řezu), tak mutaci. Pak

$$\mathbb{E}[p_k(t+1)] = \sum_i \sum_j s_i(t) s_j(t) r(i, j, k).$$

Míchání je tedy *kvadratický* operátor na simplexu. Odvodit $r(i, j, k)$ je technicky náročné, ale možné. Klíčové zjednodušení plyne z toho, že jak jednobodové křížení, tak bitová mutace „nevidí“ absolutní hodnoty bitů, nýbrž jen jejich shody a neshody. Formálně to znamená, že pro libovolné k platí

$$r(i, j, k) = r(i \oplus k, j \oplus k, 0),$$

kde $\oplus$ je bitový XOR. Stačí tedy znát jedinou *míchací matici* $\mathbf{M}$ s prvky $\mathbf{M}_{ij} = r(i, j, 0)$ (rozměru $2^l \times 2^l$) a zbytek se dopočte permutacemi $\sigma_k : x \mapsto x \oplus k$: $\mathcal{M}(s)_k = (\sigma_k s)^\top \mathbf{M} (\sigma_k s)$. V tom je hlavní technický přínos Voseho formalismu: úloha se smrskne z 2^{3l} čísel na 2^{2l} .

2.2.4 Výsledky pro nekonečné populace

Co z modelu plyne, se dá shrnout do čtyř bodů:

- SGA je **diskrétní dynamický systém** na simplexu a posloupnost $p(0), p(1), \dots$ je trajektorií tohoto systému.
- **Pevné body** $\mathcal{F}$ jsou populace tvořené pouze kopiemi nejlepšího jedince z počáteční populace; selekce tedy „zaostřuje“.
- **Pevný bod** $\mathcal{M}$ je jediný, totiž rovnoměrné rozdělení. Míchání naopak „rozostřuje“ a maximalizuje entropii.
- Pevné body složeného $\mathcal{G}$ obecně známy nejsou a jejich hledání je hlavní otevřený problém. Chování je kombinací zaostřování a rozostřování a projevuje se jako **přerušované rovnováhy** (*punctuated equilibria*), tedy dlouhá období metastability střídaná rychlými přechody. Tentýž jev je znám i z biologie.

2.2.5 Konečné populace: Markovův řetězec

Skutečný běh SGA má populaci konečnou, takže se v něm uplatní náhoda a deterministická trajektorie nestačí. Zato je celý proces bezpaměťový: nová populace závisí výhradně na té současné, a to je přesně situace pro Markovův řetězec.

Markovův řetězec

Stochastický proces v diskrétním čase se stavy, kde pravděpodobnost přechodu do dalšího stavu závisí *jen* na aktuálním stavu, nikoli na historii. Úplným popisem takového procesu je matice přechodových pravděpodobností.

Modelujeme SGA s populací velikosti n nad řetězcí délky l . Potřebujeme tři věci:

- **Stavy** jsou jednotlivé možné populace, tedy multimnožiny n řetězců. Jejich počet je

$$N = \binom{n + 2^l - 1}{2^l - 1},$$

což je počet multimnožin velikosti n z 2^l typů.

- **Matice $\mathbf{Z}$** , jejíž sloupce odpovídají populacím a jejíž prvek $\mathbf{Z}(y, i)$ udává počet výskytů jedince y v populaci i . Sloupce matice $\mathbf{Z}$ jsou tedy přímo stavy řetězce.
- **Matice přechodů $\mathbf{Q}$** rozměru $N \times N$, kterou lze explicitně odvodit z $\mathcal{G}$, protože přechod není nic jiného než multinomické losování n jedinců podle rozdělení $\mathcal{G}(p)$:

$$\mathbf{Q}_{i,j} = n! \prod_{y=0}^{2^l-1} \frac{(\mathcal{G}(p_i)_y)^{\mathbf{Z}(y,j)}}{\mathbf{Z}(y,j)!}.$$

Problém velikosti. Už pro $n = l = 8$ má $\mathbf{Q}$ řádově 10^{29} prvků. Model je tedy exaktní, ale k počítání prakticky nepoužitelný; jeho hodnota je teoretická.

Kvalitativní důsledky.

- Je-li $p_M > 0$, je řetězec **ireducibilní a aperiodický**, protože z libovolné populace lze mutacemi dosáhnout libovolné jiné. Existuje tedy jediné stacionární rozdělení a SGA navštíví každou populaci, tu s globálním optem nevyjímaje, nekonečně mnohokrát s pravděpodobností 1.
- Bez elitismu ale algoritmus ke globálnímu optimu **nekonverguje**: optimum se objeví a zase zmizí. *Elitistický* GA, který si nejlepší nalezené řešení uchovává, konverguje k globálnímu optimu s pravděpodobností 1 při $t \rightarrow \infty$. To je standardní konvergenční výsledek, neříká však nic o *rychlosti*.
- Mezi modelem nekonečné a konečné populace existuje korespondence: pro $n \rightarrow \infty$ se chování konečného modelu blíží deterministické trajektorii $\mathcal{G}$.

2.2.6 Algoritmy odhadu rozdělení (EDA)

♦ nad rámec slidů: ve slidech EVA není; je to ale přímé pokračování Voseho pohledu]

EDA — obecné schéma

Voseho model ukazuje, že SGA je ve skutečnosti stroj na transformaci pravděpodobnostního rozdělení nad prostorem řetězců. *Algoritmy odhadu rozdělení* berou toto pozorování doslova: zahodí křížení i mutaci a místo nich udržují **explicitní model** $p_\theta(x)$. Jedna iterace vypadá tak, že se z p_θ navzorkuje populace, vybere se z ní lepší část, na té se model *přeučí*, a znovu. Model θ je jediná informace, která se přenáší mezi generacemi.

Jednotlivé algoritmy se liší tím, jak bohaté závislosti model zachytí: **žádné** (UMDA, PBIL), **párové** (MIMIC), **obecné** (BOA s Bayesovskou sítí) nebo **spojité** (CMA-ES). PBIL například posouvá vektor pravděpodobností k nejlepšímu jedinci: $p_i \leftarrow (1 - \alpha) p_i + \alpha x_i^{\text{best}}$.

Univariátní model selže, jakmile mezi geny existuje *epistáze*, a právě proto vznikl BOA, který se strukturu závislostí učí. Tím EDA řeší problém, kvůli němuž Holland navrhoval inverzi: kde mají spolupracující geny ležet, si algoritmus zjistí sám.

2.3 Shrnutí ke zkoušce

- Schéma je slovo nad abecedou $\{0, 1, *\}$. Řád $o(S)$ je počet pevných pozic, definující délka $d(S)$ je vzdálenost první a poslední pevné pozice a $F(S)$ je průměrná fitness reprezentantů v dané populaci. Celkem existuje 3^m schémat a jeden jedinec je reprezentantem 2^m z nich.
- Věta o schématech zní $\mathbb{E}[C(S, t + 1)] \geq C(S, t) \frac{F(S)}{f} [1 - p_C \frac{d(S)}{m-1} - p_M o(S)]$ a odvozuje se ve třech krocích: selekce dává exponenciální růst, křížení přispívá členem $d(S)/(m-1)$ a mutace členem $(1 - p_M)^{o(S)}$.
- Hypotéza stavebních bloků říká, že GA skládá krátká, nízkého řádu a nadprůměrná schémata, tzv. stavební bloky. Z toho plyne, že záleží na kódování i na velikosti populace a že škodí předčasná konvergence.
- Implicitní paralelismus znamená, že se užitečně zpracovává $\sim n^3$ schémat. Analogie s banditou je v tom, že schémata řádu k soupeří o tytéž k pevných pozic, hrají tedy hru s 2^k pákami; analytické řešení přiděluje vítězi exponenciálně více pokusů, $N - n^* = O(e^{cn^*})$.
- Kritika má šest bodů: věta je nerovnost pro jediný krok, populace jsou konečné, existují deceptivní úlohy, uplatňuje se kolaterální konvergence a velký rozptyl fitness schématu, a model ignoruje konstruktivní efekty i vzájemnou závislost schémat.
- Voseho model ztotožní řetězec s čísly $0..2^l - 1$. Populaci pak popisuje vektor relativních četností $p(t)$ ležící v simplexu spolu s vektorem selekčních pravděpodobností $s(t)$.
- Hledá se $\mathcal{G} = \mathcal{M} \circ \mathcal{F}$ s $p(t + 1) = \mathcal{G}(p(t))$. Selekcí část $\mathcal{F}$ je dána diagonální maticí fitness, $s = \mathbf{F}p / \sum_j \mathbf{F}_{jj}p_j$, kdežto míchací část $\mathcal{M}$ (křížení a mutace) je kvadratický operátor $\mathbb{E}[p'_k] = \sum_{i,j} s_i s_j r(i, j, k)$. Díky symetrii $r(i, j, k) = r(i \oplus k, j \oplus k, 0)$ stačí jediná míchací matice.
- Pevné body $\mathcal{F}$ jsou populace kopií nejlepšího jedince, pevným bodem $\mathcal{M}$ je rovnoměrné rozdělení. Jejich kombinace vede k přerušovaným rovnováhám a pevné body $\mathcal{G}$ známy nejsou.
- Pro konečné populace se staví Markovův řetězec, jehož stavy jsou populace. Stavů je $N = \binom{n+2^l-1}{2^l-1}$ a matice $\mathbf{Q}$ je sice exaktní, ale astronomicky velká. Při $p_M > 0$ je řetězec ireducibilní a elitistický GA konverguje k optimu s pravděpodobností 1.
- Nekonečné populace dávají deterministický model, který je exaktní, ale nepraktický; konečné populace model stochastický, tedy realistický, zato nezvládnutelný. Limitní přechod obě varianty spojuje.
- EDA** berou pohled „GA je transformace rozdělení“ doslova: místo operátorů se učí model p_θ . UMDA a PBIL pracují s nezávislými marginálami, MIMIC a BMMA s párovými závislostmi, BOA s Bayesovskou sítí a CMA-ES s gaussovskou. Cyklus je pokaždé stejný: navzorkuj, vyber lepší část, přeuč model. PBIL konkrétně počítá $p_i \leftarrow (1 - \alpha)p_i + \alpha x_i^{\text{best}}$.

3 Evoluční strategie, diferenciální evoluce, koevoluce, otevřená evoluce

Kapitola spojuje čtyři témata, jejichž společným jmenovatelem je, že jdou za klasický genetický algoritmus. První dvě, evoluční strategie a diferenciální evoluce, jsou metody pro spojitou optimalizaci: pracují s vektory reálných čísel a těžiště hledání přesouvají z křížení na mutaci.

Zbývá dvě témata mění samotnou fitness. U koevoluce přestává být fitness absolutní veličinou a závisí na ostatních jedincích, u otevřené evoluce není fitness vůbec pevně dána.

3.1 Evoluční strategie

3.1.1 Motivace a základní rysy

Evoluční strategie (*evolution strategies*, ES) navrhli Rechenberg a Schwefel v Berlíně v 60. letech, a to kvůli velmi konkrétní potřebě: optimalizovat reálné parametry v obtížných technických úlohách, jako je tvar trysky nebo ohyb potrubí. Úloha byla od začátku spojitá a té je přizpůsobená celá stavba algoritmu.

Od genetického algoritmu se ES liší především ve čtyřech rysech.

- Jedinec je vektor reálných čísel a **mutace je hlavní operátor** — právě ona nese tíhu prohledávání prostoru.
- Kromě *genetických parametrů*, tedy vlastního řešení, nese jedinec i *strategické, endogenní parametry* (*endogenous parameters*), které řídí samotnou evoluci; typicky to jsou směrodatné odchylky mutací. Vytváří se tak nejen řešení, ale i způsob, jakým se hledá, a odtud pochází slogan „evoluce evoluce“.
- Rodičovská selekce je **náhodná** a na fitness vůbec nebere ohled, zatímco environmentální selekce je **deterministická**: přežívá μ nejlepších. Veškerý selekční tlak je tedy soustředěn do jediného místa cyklu.
- Nejmodernějším a nejúspěšnějším zástupcem celé rodiny je CMA-ES, kterému je níže věnován samostatný oddíl.

Notace ES

Symbol μ značí velikost populace, tedy počet rodičů, λ počet vytvořených potomků a ρ počet rodičů, kteří se podílejí na jednom potomkovi.

- V $(\mu + \lambda)$ -ES se nová populace vybírá z μ rodičů i λ potomků; schéma je tedy elitistické a dosud nejlepší jedinec z populace nikdy nezmizí.
- V (μ, λ) -ES se vybírá jen z λ potomků a rodiče vždy vymřou. Schéma je neelitistické a nutně vyžaduje $\lambda > \mu$, jinak by nebylo z čeho vybírat.
- Zápis $(\mu/\rho \dagger \lambda)$ -ES je explicitní varianta obojího, která navíc uvádí, že potomek vzniká re-kombinací ρ rodičů.

Volba mezi čárkou a plusem není kosmetická. Obě schémata se liší téměř ve všem podstatném, od schopnosti uniknout z lokálního optima až po to, jestli se samoadaptace vůbec chytne:

	(μ, λ)	$(\mu + \lambda)$
pool pro selekci	jen potomci	rodiče + potomci
elitismus	ne (fitness může klesnout)	ano
únik z lokálních optim	lepší	horší
konvergence	pomalejší	rychlejší
doporučení	spojité domény, zašuměná či dynamická fitness	diskrétní domény, drahá fitness
samoadaptace σ	funguje dobře (špatné σ vymře)	může uváznout (starý rodič s malým σ přežívá)
poměr	typicky $\lambda/\mu \approx 7$	i $\mu \geq \lambda$; $\lambda = 1 = \text{steady-state ES}$

3.1.2 $(1 + 1)$ -ES a pravidlo 1/5

Nejjednodušší evoluční strategie si vystačí s jediným jedincem. V každé iteraci z něj gaussovská mutace vytvoří jediného potomka a ten se přijme, jen pokud je lepší než rodič. Zůstává tím jediná otevřená otázka: jak velké kroky má mutace dělat. Odpověď na ni dává pravidlo 1/5.

Algoritmus 4 $(1 + 1)$ -ES s pravidlem 1/5 (minimalizace $f : \mathbb{R}^n \rightarrow \mathbb{R}$)

```

1: inicializuj  $\vec{x}(0)$  náhodně,  $\sigma > 0$ ,  $t \leftarrow 0$ 
2: repeat
3:    $\vec{y}(t) \leftarrow \vec{x}(t) + \sigma \cdot N(\vec{0}, \mathbf{I})$ 
4:   if  $f(\vec{y}(t)) < f(\vec{x}(t))$  then  $\vec{x}(t+1) \leftarrow \vec{y}(t)$ 
5:   else  $\vec{x}(t+1) \leftarrow \vec{x}(t)$ 
6:   end if
7:   sleduj  $p$  = podíl úspěšných mutací za posledních  $k$  iterací
8:   každých  $k$  iterací uprav  $\sigma$ :
9:     if  $p > 1/5$  then  $\sigma \leftarrow \sigma/c$                                 ▷ daří se: dělej větší kroky, více explorační
10:    if  $p < 1/5$  then  $\sigma \leftarrow \sigma \cdot c$                             ▷ nedaří se: zmenšuj kroky, více exploatační
11:    if  $p = 1/5$  then  $\sigma$  beze změny                                    ▷ typicky  $0,8 \leq c \leq 1$ 
12:     $t \leftarrow t + 1$ 
13: until  $\vec{x}(t)$  je dost dobré

```

Pravidlo 1/5 úspěchu (*1/5 success rule*)

Rechenberg odvodil z analýzy dvou modelových funkcí, kulové a koridorové, heuristiku, podle níž je *optimální poměr úspěšných mutací přibližně 1/5*. Je-li podíl úspěchů vyšší, postupujeme příliš opatrně: krok je malý a σ se zvětší. Je-li nižší, je krok naopak příliš velký a σ se zmenší. Řízení parametru je zde *adaptivní*, nikoli *samoadaptivní*. Algoritmus sice využívá zpětnou vazbu z vlastního běhu, ale σ není součástí genomu a selekce na ně nepůsobí.

3.1.3 Samoadaptace strategických parametrů

Pravidlo 1/5 řídí jediný parametr zvětčí a pro celý prostor stejně. Samoadaptace jde ještě o krok dál: nastavení mutace svěří evoluci samotné, takže se velikost i tvar kroku vyladí souběžně s řešením.

Samoadaptace (*self-adaptation*)

Strategické parametry, tedy odchylky σ a případně úhly natočení, jsou **součástí genomu** a podléhají mutaci i selekci společně s řešením. Selektce na ně působí jen nepřímě: jedinci s vhodným σ produkují lepší potomky, a jejich σ se tím šíří populací.

Na pořadí obou mutací přitom záleží. **Nejprve se mutují strategické parametry, teprve pak se s nimi mutují genetické parametry**; při opačném pořadí by selekce hodnotila staré σ .

Jedinec má tvar $C = [\vec{x}, \vec{\sigma}]$, kde $\vec{x} \in \mathbb{R}^n$ je vlastní řešení a $\vec{\sigma}$ má k složek. Počet složek rozhoduje o tom, jaké tvary dokáže mutace v prostoru opsat:

k	název	geometrie mutace
1	jedno globální σ	izotropní — vrstevnice hustoty jsou koule
n	nekorelované mutace	elipsoid s osami rovnoběžnými se souřadnicemi
$n(n+1)/2$	korelované mutace	obecný elipsoid (n odchylek + $n(n-1)/2$ úhlů natočení α_{ij}), odpovídá plné kovarianční matici

V nejobecnějším případě je kovarianční matice $\mathbf{C}$ součinem rotační matice a její prvky vycházejí jako $c_{ii} = \sigma_i^2$ a $c_{ij} = \frac{1}{2}(\sigma_i^2 - \sigma_j^2) \tan(2\alpha_{ij})$.

Standardní pravidla mutace (Schwefel). Konkrétní podobu mutace obou skupin parametrů zavedl Schwefel a používá se dodnes:

$$\sigma'_i = \sigma_i \cdot \exp(\tau' N(0, 1) + \tau N_i(0, 1)), \quad x'_i = x_i + \sigma'_i \cdot N_i(0, 1),$$

kde $\tau' \propto 1/\sqrt{2n}$ je společná porucha pro celého jedince a $\tau \propto 1/\sqrt{2\sqrt{n}}$ individuální porucha jednotlivé složky. Lognormální tvar plní dva úkoly najednou: udržuje σ kladné a činí zvětšení a zmenšení symetrickými. Aby σ nezdegenerovalo na nulu a evoluce se nezastavila, zavádí se dolní mez $\sigma_{\min}$. Úhly natočení se na rozdíl od odchylek mutují aditivně, tedy $\alpha'_{ij} = \alpha_{ij} + \beta \cdot N(0, 1)$.

3.1.4 Rekombinace v ES

Rekombinace v ES má oproti křížení jednu zvláštnost: z ρ rodičů vzniká *jeden* potomek. Podle počtu zapojených rodičů se rozlišuje rekombinace **lokální**, kde $\rho = 2$, a **globální**, kde $\rho = \mu$ a potomek tedy čerpá z celé populace. Druhé dělení se týká toho, jak se rodičovské hodnoty skládají dohromady.

- **Diskrétní (dominantní) rekombinace** vybírá hodnotu na každé pozici náhodně od jednoho z rodičů, takže potomek je mozaikou nezměněných rodičovských hodnot.
- **Intermediární (aritmetická) rekombinace** bere na každé pozici průměr hodnot rodičů; potomek tedy nepřebírá beze změny ani jednu z nich.

V praxi se obě varianty kombinují. Genetické parametry se rekombinují diskrétně, protože to zachovává rozmanitost, kdežto strategické parametry intermediárně, protože průměrování stabilizuje adaptaci σ .

Algoritmus 5 Obecný cyklus ES

```

1:  $t \leftarrow 0$ ; inicializuj náhodně populaci  $P_t$  o  $\mu$  jedincích  $[\vec{x}, \vec{\sigma}]$ 
2: ohodnoť jedince v  $P_t$ 
3: while není splněna ukončovací podmínka do
4:    $C_{t+1} \leftarrow \emptyset$ 
5:   for  $\lambda$  krát do
6:     vyber náhodně  $\rho$  rodičů z  $P_t$ 
7:     rekombinuj je do jednoho potomka  $[\vec{x}, \vec{\sigma}]$ 
8:     mutuj strategické parametry  $\vec{\sigma}$ 
9:     mutuj genetické parametry  $\vec{x}$  pomocí nových  $\vec{\sigma}$ 
10:    ohodnoť potomka a vlož do  $C_{t+1}$ 
11:   end for
12:    $(\mu, \lambda)$ :  $P_{t+1} \leftarrow \mu$  nejlepších z  $C_{t+1}$ 
13:    $(\mu + \lambda)$ :  $P_{t+1} \leftarrow \mu$  nejlepších z  $P_t \cup C_{t+1}$ 
14:    $t \leftarrow t + 1$ 
15: end while

```

3.1.5 CMA-ES

♦ nad rámec slidů: slidy uvádějí jednou větou jako „dnes nejúspěšnější a nejsložitější“ ES

CMA-ES — princip

Covariance Matrix Adaptation ES (Hansen, Ostermeier) je dnes referenční algoritmus pro spojitou black-box optimalizaci. Samoadaptaci jednotlivých σ v genomu nahrazuje jiným nápadem: udržuje **jediné rozdělení pro celou populaci** $\mathcal{N}(\vec{m}, \sigma^2 \mathbf{C})$ a jeho kovarianční matici se učí z historie úspěšných kroků.

Jedna iterace vypadá takto. Navzorkuje se λ bodů a vybere se μ nejlepších *podle pořadí*. Střed $\vec{m}$ se posune do jejich váženého průměru a matice $\mathbf{C}$ spolu s délkou kroku σ se aktualizují podle *evolučních cest*, tedy exponenciálně vyhlazených záznamů o tom, kudy se střed pohyboval. Míří-li po sobě jdoucí kroky souhlasným směrem, σ se zvětší; jsou-li protichůdné, zmenší se.

Výsledkem je algoritmus, který se sám přizpůsobí anizotropii a korelacím úlohy; v podstatě se bez gradientu učí aproximaci inverzní Hessovy matice. K tomu je invariantní vůči monotónním transformacím fitness a má velmi málo laditelných parametrů, takže se s ním dá pracovat i na úloze, o které se nic neví.

3.2 Diferenciální evoluce

3.2.1 Motivace

Diferenciální evoluce (*differential evolution*, DE; Storn a Price, 1997) je jednoduchý a překvapivě silný algoritmus pro black-box optimalizaci funkcí $f : \mathbb{R}^D \rightarrow \mathbb{R}$. Klíčová myšlenka je **geometrická**: velikost ani směr mutačního kroku se neodhadují z parametru σ , nýbrž se přebírají z *rozdílu dvojic jedinců v populaci*.

Populace tak sama kóduje měřítko a orientaci úlohy. Na počátku je rozptýlená a kroky jsou proto velké; jakmile začne konvergovat, rozdíly se zmenší a mutace přejde k jemnému doladění. Algoritmus tedy dostává samoadaptaci zdarma, aniž by se cokoli muselo evolvovat nebo ladit.

Aplikace sahají od strojového učení až po plánování optimálních trajektorií kosmických sond v Evropské kosmické agentuře.

3.2.2 Algoritmus DE/rand/1/bin

Populace je N vektorů $\vec{x}_1, \dots, \vec{x}_N \in \mathbb{R}^D$. Rodičovská selekce je triviální, protože rodičem se postupně stává **každý** jedinec $\vec{x}_i$; pro každého z nich proběhnou tři kroky.

1. **Mutace.** Vyberou se tři navzájem různé jedince $\vec{x}_a, \vec{x}_b, \vec{x}_c$, všichni odlišní od $\vec{x}_i$, a vznikne z nich *dárce* (*donor vector*)

$$\vec{v}_i = \vec{x}_a + F(\vec{x}_b - \vec{x}_c), \quad F \in [0, 2], \text{ typicky } F \approx 0,8.$$

Rozdíl $\vec{x}_b - \vec{x}_c$ dodává směr i délku kroku, koeficient F jej škáluje.

2. **Křížení (binomické, uniformní „s pojistkou“).** Z dárce a rodiče se poskládá *zkušební vektor* (*trial vector*) $\vec{u}_i$:

$$u_{j,i} = \begin{cases} v_{j,i} & \text{pokud } \text{rand}_j \leq C \text{ nebo } j = I_{\text{rand}}, \\ x_{j,i} & \text{jinak,} \end{cases}$$

kde $\text{rand}_j \sim U(0, 1)$, $C \in [0, 1]$ je práh křížení a I_{rand} náhodný index z $\{1, \dots, D\}$. Podmínka $j = I_{\text{rand}}$ je zmíněná „pojistka“: zaručuje, že alespoň jedna složka pochází od dárce, takže potomek nikdy není identickou kopií rodiče.

3. **Environmentální selekce (souboj 1:1).** Zkušební vektor se utká přímo se svým rodičem a lepší z dvojice postupuje dál:

$$\vec{x}_i(t+1) = \begin{cases} \vec{u}_i & \text{pokud } f(\vec{u}_i) \leq f(\vec{x}_i(t)), \\ \vec{x}_i(t) & \text{jinak.} \end{cases}$$

Selekce je tedy elitistická na úrovni jednotlivce a populace se nikdy nezhorší.

Algoritmus 6 DE/rand/1/bin

Vstup: velikost populace N , měřítko F , práh křížení C

```

1: inicializuj populaci  $\{\vec{x}_i\}_{i=1}^N$  náhodně v mezích úlohy
2: while není splněna ukončovací podmínka do
3:   for  $i \leftarrow 1$  to  $N$  do
4:     vyber náhodně různé indexy  $a, b, c \neq i$ 
5:      $\vec{v} \leftarrow \vec{x}_a + F(\vec{x}_b - \vec{x}_c)$ 
6:      $I_{\text{rand}} \leftarrow$  náhodný index z  $\{1, \dots, D\}$ 
7:     for  $j \leftarrow 1$  to  $D$  do
8:        $u_j \leftarrow v_j$  pokud  $\text{rand}() \leq C$  nebo  $j = I_{\text{rand}}$ , jinak  $u_j \leftarrow x_{j,i}$ 
9:     end for
10:    if  $f(\vec{u}) \leq f(\vec{x}_i)$  then  $\vec{x}'_i \leftarrow \vec{u}$ 
11:    else  $\vec{x}'_i \leftarrow \vec{x}_i$ 
12:    end if
13:  end for
14:   $\{\vec{x}_i\} \leftarrow \{\vec{x}'_i\}$ 
15: end while
```

3.2.3 Číselný příklad jednoho kroku

Celý cyklus se dá spočítat na papíře. Nechť $D = 3$ a minimalizuje se $f(\vec{x}) = \sum_j x_j^2$ s parametry $F = 0,8$ a $C = 0,9$. Aktuálním jedincem je $\vec{x}_i = (4; -3; 2)$ s hodnotou $f(\vec{x}_i) = 16 + 9 + 4 = 29$ a náhodně vybráni byli jedinci

$$\vec{x}_a = (2; -1; 0,5), \quad \vec{x}_b = (1; 0; -1), \quad \vec{x}_c = (-1; 2; 1).$$

Mutace: nejprve se spočítá rozdílový vektor a z něj dárce.

$$\vec{x}_b - \vec{x}_c = (2; -2; -2), \quad \vec{v} = (2; -1; 0,5) + 0,8(2; -2; -2) = (3,6; -2,6; -1,1).$$

Křížení: nechť $I_{\text{rand}} = 1$ a náhodná čísla vyšla $\text{rand} = (0,40; 0,95; 0,20)$. Složku po složce potom platí:

- pro $j = 1$ je $0,40 \leq 0,9$, a navíc jde o pojistkový index $j = I_{\text{rand}}$, takže $u_1 = v_1 = 3,6$;
- pro $j = 2$ je $0,95 > 0,9$ a zároveň $j \neq I_{\text{rand}}$, hodnota tedy zůstává rodičovská, $u_2 = x_{2,i} = -3$;
- pro $j = 3$ je $0,20 \leq 0,9$, a proto $u_3 = v_3 = -1,1$.

Zkušební vektor je $\vec{u} = (3,6; -3; -1,1)$.

Selekce: platí $f(\vec{u}) = 12,96 + 9 + 1,21 = 23,17 < 29 = f(\vec{x}_i)$, takže $\vec{u}$ nahrazuje $\vec{x}_i$ v nové populaci.

3.2.4 Varianty mutace

Varianty DE se označují zápisem DE/ $x/y/z$, v němž x říká, jak se volí základní vektor, y udává počet rozdílových vektorů a z určuje typ křížení: **bin** znamená binomické uniformní křížení, **exp** exponenciální, které přebírá souvislý úsek složek.

varianta	dárce $\vec{v}$
DE/rand/1	$\vec{x}_a + F(\vec{x}_b - \vec{x}_c)$
DE/rand/2	$\vec{x}_a + F(\vec{x}_b - \vec{x}_c + \vec{x}_d - \vec{x}_e)$
DE/best/1	$\vec{x}_{\text{best}} + F(\vec{x}_b - \vec{x}_c)$
DE/best/2	$\vec{x}_{\text{best}} + F(\vec{x}_b - \vec{x}_c + \vec{x}_d - \vec{x}_e)$
DE/current-to-best/1	$\vec{x}_i + F(\vec{x}_{\text{best}} - \vec{x}_i) + F(\vec{x}_b - \vec{x}_c)$

Volba základního vektoru rozhoduje o kompromisu mezi rychlostí a robustností. Varianty s **best** konvergují rychleji, ale snáze uvíznou v lokálním optimu, kdežto **rand** je robustnější. Moderní varianty jDE, SaDE, JADE, SHADE a L-SHADE adaptují F a C za běhu a patří ke špičce v soutěžích CEC.

3.2.5 Srovnání DE s ES a GA

Tři algoritmy je nejnázornější postavit vedle sebe podle toho, odkud berou velikost mutačního kroku a kde v nich působí selekční tlak.

	GA (reálný)	ES	DE
zdroj kroku mutace	pevné σ	evolvované σ	rozdíly jedinců
rodičovská selekce	podle fitness	náhodná	každý jedinec jednou
env. selekce	generační / elitismus	μ nejlepších	souboj rodič vs. potomek
hlavní operátor	křížení	mutace	diferenční mutace
adaptace měřítka	žádná	explicitní (v genomu)	implicitní (z populace)

3.3 Koevoluce

3.3.1 Kontextová fitness

Koevoluce (*coevolution*)

Evoluční algoritmus, v němž fitness jedince **není absolutní**, ale závisí na ostatních jedincích, ať už na jeho vlastní populaci, nebo na jiné, souběžně se vyvíjející populaci. Fitness je tedy *kontextová* a fitness krajina **dynamická**: mění se tím, jak se mění protivníci či partneři.

Důvod, proč se ke koevoluci sahá, je zpravidla praktický. U mnoha úloh neumíme napsat absolutní účelovou funkci, protože na otázku „jak dobrá je strategie pro dámu?“ se přímo odpovědět nedá. Dva jedince spolu ale *porovnat* umíme: stačí je nechat zahrát partii.

Druhý důvod je ambicióznější. Koevoluce slibuje *evoluční závody ve zbrojení*, v nichž se protivníci navzájem tlačí ke stále lepším řešením. Podle toho, zda jsou si jedinci soupeři, nebo spoluhráči, se rozlišují dva typy koevoluce:

	kompetitivní	kooperativní
vztah	fitness jednoho roste na úkor druhého	jedinci se doplňují do společného řešení
příklad	predátor–kořist, host–parazit, hráč–protihráč, řadicí sítě vs. posloupnosti čísel	dekompozice problému na podproblemy, moduly a plány (CoDeep-NEAT), hlavní programy a ADF
fitness	výsledek souboje (turnaje)	kvalita složeného řešení, na němž se jedinec podílel
riziko	cyklení, odpojení, přespecializace	kredit assignment, líní partneři

3.3.2 Kompetitivní koevoluce

Klasický příklad: Hillisovy řadicí sítě (1990). Populace řadicích sítí zde koevolvuje proti populaci testovacích posloupností. Fitness sítě je podíl správně seřazených testů, fitness testu naopak to, kolik sítí na něm selhalo. Parazitické testy se proto specializují na slabiny sítí a sítím nezbyvá než se zlepšovat.

Hillis takto našel síť pro 16 prvků s **61** komparátory. *Pozor na častý omyl:* nejlepší známý lidský návrh (Green, 1969) má 60 komparátorů, koevoluce ho tedy nepřekonala. Podstatné je srovnání uvnitř experimentu, a to vypadá jinak: týž genetický algoritmus *bez* koevolvujících parazitů dosáhl jen 65 komparátorů. Koevoluce zde prokazatelně zlepšila optimalizační schopnost EA a přiblížila se lidskému výsledku na jediný komparátor.

Predátor–kořist je druhý klasický vzorec a je doma v evoluční robotice: roboti-lovci a roboti-uprchlíci se navzájem zdokonalují, protože každé zlepšení jedné strany je pro druhou novým zadáním. Dvojice **host–parazit** se objevuje v modelech umělého života (Tierra) a při testování software.

Turnajové hodnocení. Fitness se získává soubojem a způsobů, jak souboje uspořádat, je několik. Nejpoctivější je turnaj každý s každým, ten je ale drahý. Levnější a zároveň zašuměnou variantou je souboj s náhodným vzorkem protivníků. Metoda *hall of fame* nechává jedince utkat se s archivem historicky nejlepších, a brání tak zapomínání. *Turnaj se sdílenou fitness* (*competitive fitness sharing*) navíc odstupňuje odměnu: za poražení protivníka, kterého poráží málokdo, je odměna vyšší.

3.3.3 Patologie koevoluce

Koevoluce má pověst vrtkavého nástroje a důvodem je řada dobře popsanych patologií, které se u ní objevují znovu a znovu.

- **Cyklení** (*cycling*, intranzitivita) nastává, když populace obíhají dokola jako v kámen–nůžky–papír. Pokrok je pak jen zdánlivý a dynamika koevoluce může být až deterministicky chaotická.
- **Odpojení** (*disengagement*) znamená, že jedna populace zcela přeroste druhou. Všechny souboje pak končí stejně, fitness ztrácí rozlišovací schopnost a selekce přestane fungovat, protože zmizel gradient.
- **Efekt Červené královny** (*Red Queen effect*) nastane, když se všichni zlepšují, ale relativní fitness zůstává konstantní. Z naměřených hodnot pak nepoznáme, jestli je pokrok skutečný; řešením je měřit proti pevné externí sadě protivníků (*master tournament*).
- **Průměrné stabilní stavy** (*mediocre stable states*) vznikají, když se populace ustálí ve vzájemné rovnováze na nízké úrovni a už nic nenutí ani jednu z nich se zlepšovat.
- **Přespecializace** (*overspecialization, focusing*) postihuje jedince, který se vyladí na aktuálního protivníka a ztratí obecnost.
- **Zapomínání** (*forgetting*) je ztráta schopnosti porazit staré protivníky. Pomáhají proti němu archivy: *hall of fame*, Pareto-koevoluční archivy a *Nash memory*.

3.3.4 Kooperativní koevoluce

CCGA (*cooperative coevolutionary GA*, Potter & De Jong)

Problém se rozdělí na k komponent, jimiž mohou být například souřadnice, moduly nebo pravidla, a pro každou komponentu se udržuje **samostatná populace**. Fitness jedince z populace i se určí tak, že se jedinec *spojí* se zástupci ostatních populací (typicky s jejich aktuálně nejlepšími jedinci, případně i s náhodnými) a ohodnotí se výsledné složené řešení.

Výhody jsou nasnadě: velký prostor se rozloží na menší, dekompozice se přirozeně paralelizuje a pro modulární úlohy je takové rozdělení věcně správné.

Problémy jsou tři. *Přiřazení zásluh* (*credit assignment*) řeší, jak rozdělit kvalitu složeného řešení mezi jednotlivé komponenty. *Provázanost* brání dobrému oddělení tam, kde spolu komponenty silně interagují. A *relativní přespecializace* znamená, že se populace přizpůsobí konkrétním partnerům místo úloze jako celku.

Praktických příkladů je řada: koevoluce hlavních programů a ADF v GP, dále CoDeepNEAT s populační modulů a populací „blueprintů“ (odst. 6.2) a konečně meta-lamarckovské memetické algoritmy, kde vedle sebe žije populace memů a populace řešení (kap. 5).

3.3.5 Evoluce kooperace a věžňovo dilema

Koevoluce úzce souvisí se základní otázkou evoluční biologie: *jak může vzniknout altruismus*, když z definice snižuje fitness jedince? Darwinismus je ve své podstatě soutěživý, jak napovídá heslo „přežítí nejzdatnějšího“, a přesto je v přírodě i ve společnosti spolupráce všude kolem. Vysvětlení nabízí několik teorií.

- **Skupinová selekce** předpokládá, že evoluce působí i na skupiny (Darwin). Naráží ale na otázku, jak vysvětlit podvodníky uvnitř skupiny.
- **Příbuzenská selekce** (*kin selection*) vysvětluje altruismus zachováním téměř identických genů u příbuzných. Nevysvětlí ovšem altruismus vůči cizím jedincům a jiným druhům.
- **Sobecký gen** (Dawkins) posouvá jednotku evoluce od jedince ke genu. (Wilson to shrnuje větou „organismus je jen způsob, jak DNA vyrábí další DNA“.)
- **Reciproční altruismus** (Trivers, 1971) staví na vzájemném prospěchu a *stínu budoucnosti*; jeho formálním modelem je iterované věžňovo dilema.

Věžňovo dilema (*prisoner's dilemma*)

Hra dvou hráčů s akcemi C (kooperace) a D (zrada) a výplatami

	C	D
C	R/R	S/T
D	T/S	P/P

$T > R > P > S$ (a pro iterovanou verzi $2R > T + S$).

Typicky se volí $R = -1$, $T = 0$, $P = -2$ a $S = -3$. Protože $T > R$ a zároveň $P > S$, je zrada **dominantní strategií** obou hráčů a dvojice DD je **Nashovým ekvilibriem**. Jako jediný z možných výsledků přitom DD **není Pareto-optimální**: společný zisk maximalizuje CC .

Racionální hráči tedy dospějí k výsledku, který je pro oba horší než dosažitelná spolupráce. Tentýž mechanismus v n -hráčské podobě se nazývá *tragédie obecní pastviny* (*tragedy of the commons*): každému se individuálně vyplatí čerpat sdílený zdroj o něco víc, a zdroj proto zanikne. Jádrem celé diskuse kolem věžňova dilematu je právě otázka, co je vlastně racionální a zda se tak lidé opravdu chovají.

Dominantní strategie, Nashovo ekvilibrium, Pareto optimalita

Strategie s je *dominantní* pro hráče i , dává-li stejný nebo lepší výsledek než kterákoli jiná strategie i proti *všem* strategiím protihráče. Dvojice (s_i, s_j) je v *Nashově ekvilibriu*, jsou-li obě strategie vzájemně nejlepšími odpověďmi, takže žádný hráč nemá důvod jednostranně změnit svou volbu. Řešení je *Pareto-optimální*, pokud nelze zlepšit výsledek jednoho hráče, aniž by se zhoršil výsledek jiného.

Nashova věta říká, že každá hra s konečně mnoha strategiemi má ekvilibrium ve *smíšených* strategiích, tedy v náhodné volbě mezi čistými; příkladem je kámen–nůžky–papír.

Iterovaná verze a Axelrodovy turnaje. Hraje-li se opakovaně a hráči si pamatují historii, může se spolupráce vyplatit, protože nad hrou visí *stín budoucnosti*. Pozor na jednu výjimku: zná-li

se předem přesný počet kol N , plyne indukcí od konce, že optimální je stále zrazovat. V praxi ale hráči konec neznačí. Axelrod (1980) uspořádal turnaje strategií a vyšly z nich tři experimenty:

- 1. turnaj sestával ze 14 strategií a RANDOM, hrálo se 200 her, každý s každým (a i sám se sebou) v 5 opakováních. Vyhrála **TFT** (*tit for tat*): začni kooperací, pak opakuj poslední tah soupeře.
- 2. turnaj měl 62 strategií a všichni účastníci znali výsledky prvního; TFT vyhrála znovu.
- 3. „ekologický“ turnaj byl simulací po vzoru GA: počet jedinců dané strategie v další generaci byl úměrný počtu vítězství a běželo se 1000 generací. TFT opět zvítězila.

Čtyři vlastnosti úspěšných strategií: *slušnost* (niceness) znamená nezrazovat první; *dráždivost* (provocability/retaliation) žádá potrestat zradu okamžitě; *odpouštění* (forgiveness) velí se zase rychle uklidnit. Nejméně samozřejmá je *nezávislost* (non-enviousness): úspěšná strategie neusiluje o to porazit *konkrétního* soupeře. TFT nikdy nezískala víc bodů než její protihráč, a přesto turnaj vyhrála. Jako pátou vlastnost Axelrod uvádí *srozumitelnost* (clarity): buď jednoduchý, aby s tebou ostatní vůbec dokázali navázat spolupráci. Žádná strategie nevyhraje proti všem, takže úspěch znamená obstát proti velmi rozmanitým soupeřům (ALL-D, TFTT, RANDOM, TRIGGER) a umět dobře hrát i sám se sebou.

Poučení a pozdější vývoj. V prostředích, kde výplaty přejí spolupráci a kde je vysoká pravděpodobnost opakování, se spolupráce obvykle vyvine. Není k tomu potřeba racionalita ani vědomí, stačí vyšší fitness.

Obrázek se ovšem mění, jakmile se do hry vloží šum. V zašuměném prostředí, kde se tah může provést chybně, je úspěšnější strategie **Pavlov** (*win-stay, lose-shift*): dostal-li jsem R nebo P , zopakuj tah (C); dostal-li jsem T nebo S , změň jej (D). Turnaj opakovaný po 20 letech pak vyhrál *tým* strategií z University of Southampton. Prvních zhruba 10 tahů sloužilo k rozpoznání spoluhráče a poté všichni členové týmu obětavě pomáhali jedné „otcovské“ strategii získat vysoké skóre.

3.3.6 Diverzita, druhy a niky

Malý selekční tlak stačí k tomu, aby diverzita populace vymizela. (Podobnou úvahu rozvíjí Flegrova *zamrzlá evoluce*: geneticky proměnlivý druh se vůči selekci chová pružně jen krátce, poté „zamrzne“ a na změny prostředí přestane reagovat.) Existuje proto řada mechanismů, jak diverzitu udržet; hodí se stejně pro koevoluci jako pro dynamickou fitness a vícekritériální optimalizaci.

- **Fitness sharing** dělí fitness počtem podobných jedinců, $f'(i) = f(i) / \sum_j sh(d(i, j))$, kde $sh(d) = 1 - (d/\sigma_{share})^\alpha$ pro $d < \sigma_{share}$ a jinak 0. Vytváří se tím tlak na obsazení různých nik.
- **Crowding** nechává nového jedince nahradit *nejpodobnějšího* jedince populace; patří sem deterministic crowding a RTS.
- **Nenáhodné páření** (*non-random mating*) povoluje křížení jen mezi podobnými jedinci, čemuž se říká *speciace*, případně je řídí topologií: jedinci žijí na 2D/3D mřížce a kříží se jen se sousedy.
- **Ostrovní model** (*island model*) rozdělí populaci na několik subpopulací s občasnou migrací; podrobněji o něm níže.
- **Explicitní druhy** se zavádějí značkovacími bity (*tag bits*) nebo se odvozují ze strukturální podobnosti genomů (NEAT, odst. 6.2); páření je pak povoleno jen uvnitř druhu.

Slabinou všech těchto technik je, že se musí definovat podobnost, ať už jako Hammingova vzdálenost, vzdálenost v prostoru fenotypů, nebo počet shodných inovačních čísel, a k tomu zvolit práh niky.

Paralelní modely EA. Struktura populace úzce souvisí s paralelizací a rozlišují se tři modely.

- **Master–slave** neboli globální paralelizace pracuje s jedinou populací a mezi uzly rozděljuje pouze *vyhodnocení fitness*. Algoritmus je matematicky totožný se sekvenčním a vyplatí se, je-li fitness drahá.

- **Ostrovní neboli hrubozrný model** (*coarse-grained*) udržuje několik nezávislých subpopulací, mezi nimiž občas *migruje* několik jedinců. Ladí se u něj topologie ostrovů, *migrační interval* a *migrační míra* a také výběr migrantů a nahrazovaných jedinců. Vzácná migrace udržuje diverzitu, protože ostrovy konvergují jinam; příliš častá migrace model degeneruje na jedinou populaci.
- **Buněčný neboli jemnozrný model** (*fine-grained*, difuzní) usadí jedince na 2D/3D mřížku a kříží je jen se sousedy. Dobré řešení se pak šíří populací jako vlna, což silně zpomaluje předčasnou konvergenci; svou podstatou je to implicitní forma nenáhodného páření.

Ostrovní i buněčný model tedy nejsou jen implementační trik. Mění samotnou dynamiku algoritmu a patří mezi standardní nástroje udržení diverzity.

3.3.7 Dynamické fitness krajiny

Fitness krajina (*fitness landscape*)

Pojem zavedl Sewall Wright (1932). Jde o grafické znázornění závislosti fitness na genotypu, případně na frekvenci alel či na rysu fenotypu. Je to teoretický nástroj pro uvažování o EA; ve vyšších dimenzích ovšem přestává být intuitivní.

Fitness se často mění v čase. Robot pracuje v místnosti, kde někdo rozsvítí; začne pršet; na burze propukne krize; agenti se utkávají v turnajích, tedy koevolví. Klasické GA si v takových podmínkách vedou špatně, protože jsou „přeučené“ na statické úlohy: rychle konvergují a ztratí přitom právě tu diverzitu, kterou by pro reakci na změnu potřebovaly. De Jong to doložil srovnáním: (1 + 10)-ES reaguje na náhlou změnu pomalu, protože si samoadaptací zmenšila krok, kdežto generační GA s ruletovou selekcí a bez elitismu si udrží více diverzity a zotaví se rychleji.

Pro dynamické krajiny se proto EA upravují několika způsoby.

- **Diploidní reprezentace** (Goldberg, Smith) používá dvě sady chromozomů s dominancí, které při oscilující fitness fungují jako dlouhodobá paměť.
- **Hypermutace** (Cobb, Grefenstette) sleduje průměrnou fitness populace; klesne-li (pravděpodobně se změnilo prostředí), výrazně se zvýší pravděpodobnost mutace.
- **Náhodná migrace** vloží do populace dávku nových náhodných jedinců.
- **Udržování diverzity** zahrnuje nižší selekční tlak, crowding, niching a subpopulace, ale i volbu čárkové ES (μ, λ): rodiče v ní nepřežívají, takže se populace snáz odpoutá od zastaralého optima.

Změny samotné se dělí na malé plynulé (opotřebení hardwaru), morfologické (vznikají a zanikají kopce), cyklické (roční období) a katastrofické nespojitě. Mění-li se fitness příliš rychle, žádné řešení není trvale dobré (srov. No Free Lunch).

3.4 Otevřená evoluce

♦ nad rámec slidů: celý oddíl kromě interaktivní evoluce] Otevřenou evoluci okruh výslovně jmenuje, slidy EVA I ani EVA II o ní ale nemají jedinou stránku. Jediná zmínka se najde v *Evoluční robotice*, a to v jedné větě („evolúcia robotov je kontinuálny proces s otvoreným koncom“). Následující oddíl je proto zpracovaný stručně, na úrovni, která na zkoušce obstojí. Výjimkou je *interaktivní evoluce* na konci oddílu: ta ve slidech Evoluční robotiky opravdu je, včetně subjektivní fitness a Biomorphs.

Otevřená evoluce (*open-ended evolution*)

Evoluční proces, který **nemá pevný cíl** a který neustále produkuje novou složitost a nové „druhy“ chování, aniž by konvergoval. Vzorem je pozemská biosféra: žádnou účelovou funkci nemá, a přesto trvale generuje novinky.

Znaky otevřenosti. O konkrétním systému je těžké rozhodnout, jestli je otevřený, nebo jen dlouho běží. Proto se formulují *znaky* (*hallmarks*), podle kterých se to dá posoudit. Prvním z nich je **trvalá**

produkce novosti: nové chování se objevuje i po libovolně dlouhém běhu, ne jenom v úvodní fázi. Druhým je **neomezený růst složitosti**, a to jak u samotných jedinců, tak u vztahů mezi nimi. Třetím je **vznik nových tříd entit** a nových úrovní organizace (v biologii *major transitions*: replikátor → buňka → mnohobuněčnost → společenství). Čtvrtým znakem je **evoluce evolvability**, protože se mění i samotný způsob, jakým systém novinky generuje.

Systémy a proč se zasekávají. Pokusů postavit otevřený systém proběhla celá řada. *Tierra* (Ray) a *Avida* pracují se sebekopírujícími programy ve strojovém kódu a emergentně v nich vznikli paraziti a hyperparaziti. *Polyworld* osazuje 2D svět agenty s neuronovými sítěmi, *ECHO* (Holland) agenty, kteří si nesou „tagy“ a soupeří o zdroje. Prakticky každý z těchto systémů ale po čase dospěje do stavu, kdy se už nic kvalitativně nového neobjevuje: prostředí je konečné, prostor možných „vynálezů“ se vyčerpá a dynamika zamrzne.

Proč se to děje, se zatím spíš odhaduje. Nabízejí se tři hypotézy. Chybí dostatečně bohaté a samo se rozšiřující prostředí; chybí skutečně otevřená reprezentace; a chybí mechanismus, který by průběžně zvyšoval nároky kladené na jedince. Dosažení skutečně otevřené evoluce tak zůstává otevřeným výzkumným problémem.

Generativní (nepřímé) reprezentace. S otevřenou evolucí souvisí otázka, jak vůbec zakódovat složitost, která má neomezeně růst. Přímý popis výsledku by musel růst spolu s ním. Odpovědi jsou proto kódování, v nichž genotyp není popisem výsledku, ale **návodem, jak výsledek vypěstovat**. Patří sem *L-systémy*, tedy prepisovací gramatiky generující větvící se struktury, dále *celulární automaty*, *buněčné kódování* (Gruau) a pro neuronové sítě *CPPN/HyperNEAT* (odst. 6.2). Přínos mají všechna společný: malý genom, velký fenotyp a přirozené vyjádření symetrií i opakujících se motivů.

Novelty search a řídkost

Radikální myšlenku přinesli Lehman & Stanley (2011) v článku *Abandoning Objectives*: u deceptivních úloh je účelová funkce zavádějící, protože vede do lokálních optim. Nabízí se tedy postup přesně opačný, totiž **zahodíme cíl a odměňujeme pouze novost**.

Definuje se *prostor chování* (např. koncová pozice robota v bludišti) a novost jedince x se měří řídkostí jeho okolí

$$\rho(x) = \frac{1}{k} \sum_{i=1}^k \text{dist}(x, \mu_i),$$

kde μ_i je k nejbližších sousedů x v aktuální populaci **a v permanentním archivu**. Jako fitness slouží přímo ρ a původní účelová funkce se *nepoužívá vůbec*. Na úlohách typu „robot v bludišti“ bývá takový postup úspěšnější než cílená optimalizace. Navazuje na něj **quality-diversity**, konkrétně MAP-Elites: prostor chování se pokryje mřížkou buněk indexovanou deskriptory chování a v každé buňce se udržuje nejlepší nalezené řešení.

Interaktivní evoluce. Ještě jedna cesta vede k evoluci bez zapsaného cíle: fitness funkci nahradí **člověk**. Uživatel se předloží populace kandidátů a on vybere ty, které se mu líbí. Uplatní se to tam, kde kritérium kvality neumíme zapsat, tedy u estetiky, designu nebo hudby. Klasickou ukázkou jsou Dawkinsovy *Biomorphs*, dále **Picbreeder** a **EndlessForms**, kde se pomocí CPPN vyvíjejí obrázky a 3D tvary. Omezením je pomalost a únava uživatele, takže se nutně pracuje s malými populacemi (9–20 jedinců na obrazovku) a s malým počtem generací. Právě zkušenost z Picbreederu byla přímou motivací pro novelty search: nejzajímavější objekty tam nikdy nevznikly cíleným hledáním.

3.5 Shrnutí ke zkoušce

- Evoluční strategie pracují s reálnými vektory: hlavním operátorem je mutace, rodičovská selekce probíhá náhodně a environmentální selekce deterministicky, přičemž jedinec = $[\vec{x}, \vec{\sigma}]$.

- U dvojice (μ, λ) vs. $(\mu + \lambda)$ je nutné umět vyjmenovat rozdíly, tedy elitismus, rychlost konvergence, chování v lokálních optimech a vliv na samoadaptaci.
- Velikost kroku v $(1 + 1)$ -ES řídí pravidlo $1/5$: $p > 1/5 \Rightarrow$ zvětši σ , $p < 1/5 \Rightarrow$ zmenši. Jde o řízení adaptivní, nikoli samoadaptivní.
- Samoadaptace zmutuje nejdřív strategický parametr a teprve pak proměnné: $\sigma' = \sigma \exp(\tau' N(0, 1) + \tau N_i(0, 1))$ a poté $x' = x + \sigma' N(0, 1)$, tedy **nejdřív σ , pak x** . Strategický parametr je jeden u izotropní mutace, je jich n u nekorelované a $n(n + 1)/2$ u korelované, kde přibývají úhly natočení.
- Rekombinace se dělí podle počtu rodičů na lokální ($\rho = 2$) a globální ($\rho = \mu$) a podle způsobu míchání na diskretní (dominantní) a intermediární (aritmetickou).
- CMA-ES si nese stav $\theta = (\vec{m}, \sigma, \mathbf{C}, \vec{p}_\sigma, \vec{p}_c)$, vzorkuje z $\mathcal{N}(\vec{m}, \sigma^2 \mathbf{C})$ a kovarianční matici upravuje pomocí evolučních cest rank-1 a rank- μ updatem; velikost kroku řídí délka $\vec{p}_\sigma$. Metoda je invariantní vůči monotónním transformacím fitness a lineárním transformacím prostoru.
- Schéma DE/rand/1/bin vytvoří dárce $\vec{v} = \vec{x}_a + F(\vec{x}_b - \vec{x}_c)$, zkříží ho binomicky s prahem C a pojistkou $j = I_{\text{rand}}$ a potomka nechá soubojit s vlastním rodičem, kterého nahradí jen při zlepšení.
- Parametry DE jsou $F \in [0, 2]$ (typicky 0,5–0,9), $C \in [0, 1]$ a velikost populace $N \approx 10D$.
- Geometrická podstata DE spočívá v tom, že se měřítko i orientace kroků přebírají z aktuálního rozptylu populace, takže adaptace probíhá automaticky a bez strategických parametrů.
- Nejčastější varianty jsou rand/1, rand/2, best/1, best/2 a current-to-best, křížení pak bin nebo exp.
- U zkoušky se žádá předvedení jednoho kroku s konkrétními čísly (mutace $\rightarrow$ křížení $\rightarrow$ selekce).
- Koevoluce znamená kontextovou fitness a dynamickou krajinu. *Kompetitivní* podobu mají úlohy predátor–kořist, host–parazit a Hillisovy řadičské sítě, *kooperativní* pak dekompozice na komponenty, CCGA a schéma moduly + blueprinty.
- Mezi patologie patří cyklení (intransitivita), odpojení, efekt Červené královny, průměrné stabilní stavy, přespecializace a zapomínání. Lékem jsou archivy (hall of fame), sdílená fitness a měření proti pevné sadě.
- Ve věžňově dilematu platí $T > R > P > S$ a $2R > T + S$; D je dominantní strategie a DD Nashovo ekvilibrium, které jako jediné není Pareto-optimální. V iterované verzi se prosadí TFT a úspěšná strategie má čtyři vlastnosti (slušnost, dráždivost, odpouštění, nezávistivost), k nimž se jako pátá přidává srozumitelnost; v zašuměném prostředí se osvědčí Pavlov.
- Diverzitu udržují fitness sharing, crowding, speciace, ostrovní model a nenáhodné páření.
- Na dynamickou fitness se odpovídá diploidí, hypermutací, náhodnou migrací, udržováním diversity a (μ, λ) -ES.
- Otevřená evoluce běží bez pevného cíle a pozná se podle znaků: trvalé novosti, neomezeného růstu složitosti, vzniku nových tříd entit a evoluce evolvability. Systémy Tierra, Avida, Polyworld a ECHO se všechny dříve či později zaseknou. Neomezeně rostoucí složitost kódují generativní (nepřímé) reprezentace: L-systémy, celulární automaty, buněčné kódování a CPPN.
- Novelty search měří řídkost $\rho(x)$ v prostoru chování, a to vůči populaci a permanentnímu archivu, zatímco účelovou funkci vůbec nepoužívá; navazuje na něj quality-diversity a MAP-Elites s mřížkou deskriptorů chování.
- V interaktivní evoluci hraje roli fitness člověk (Biomorphs, Picbreeder, EndlessForms); omezením je malá populace a únava uživatele.

4 Rojové optimalizační algoritmy

Hejno špačků se dokáže srovnaně vyhnout dravci, aniž by mělo velitele, a mravenčí kolonie najde nejkratší cestu ke zdroji potravy, přestože žádný jednotlivý mravenec tu cestu nezná. Z takových pozorování vychází rojová inteligence: nehledá se chytřejší jedinec, ale takové lokální chování, ze kterého se rozumné řešení složí samo.

Rojová inteligence (*swarm intelligence*)

Přístup, kde globálně inteligentní chování **emerguje** z jednoduchých lokálních pravidel mnoha jedinců bez centrálního řízení. Typické rysy jsou čtyři: řízení je decentralizované, jedinci jsou jednodušší, komunikace probíhá nepřímo prostřednictvím prostředí (*stigmergie*) a systém je robustní vůči výpadku jedince, protože žádný jedinec není nenahraditelný. Na rozdíl od EA zde obvykle není křížení, mutace ani „smrt“ jedinců: jedinci místo toho trvale existují, pohybují se a pamatují si.

4.1 Optimalizace rojem částic (PSO)

Particle swarm optimization (Eberhart a Kennedy, 1995) je inspirována hejny ptáků a ryb. Pták v hejnu nemá mapu potravy. Řídí se dvěma věcmi: vzpomínkou na místo, kde se mu dosud dařilo nejlépe, a tím, kam míří hejno kolem něj. Právě tyto dva vlivy algoritmus překládá do vzorce.

Jedinec se proto nejmenuje jedinec, ale *částice*, a je to bod v $\mathbb{R}^n$ s rychlostí. Nepoužívá se křížení ani mutace; namísto nich má algoritmus **paměť**, a to hned dvojí:

- $\vec{p}_i$ ($pBest$) je nejlepší pozice, kterou částice i kdy navštívila, a představuje tedy paměť kognitivní, individuální,
- $\vec{g}$ ($gBest$) je nejlepší pozice nalezená celým rojem, tedy paměť sociální a kolektivní.

Pohyb částice pak vznikne složením tří tendencí: pokračovat setrvačně dosavadním směrem, zatočit ke své nejlepší vzpomínce a zatočit k nejlepšímu místu roje.

Pohybové rovnice PSO

$$\vec{v}_i \leftarrow \underbrace{w \vec{v}_i}_{\text{setrvačnost}} + \underbrace{c_1 r_1 (\vec{p}_i - \vec{x}_i)}_{\text{kognitivní složka}} + \underbrace{c_2 r_2 (\vec{g} - \vec{x}_i)}_{\text{sociální složka}}, \quad \vec{x}_i \leftarrow \vec{x}_i + \vec{v}_i,$$

kde $r_1, r_2 \sim U(0, 1)$ (losují se pro každou složku zvlášť), c_1, c_2 jsou koeficienty učení (v původní verzi $c_1 = c_2 = 2$) a w je *setrvačná váha* (*inertia weight*). Původní verze z roku 1995 setrvačnou váhu ještě nemá, což odpovídá volbě $w = 1$. Doplnili ji až Shi a Eberhart a doporučují nechat ji lineárně klesat, například z 0,9 na 0,4: velké w podporuje exploraaci, malé exploataci.

Algoritmus 7 PSO

```
1: inicializuj pozice  $\vec{x}_i$  a rychlosti  $\vec{v}_i$  náhodně;  $\vec{p}_i \leftarrow \vec{x}_i$ 
2: repeat
3:   for každou částici  $i$  do
4:     vyhodnoť  $f(\vec{x}_i)$ 
5:     if  $f(\vec{x}_i)$  je lepší než  $f(\vec{p}_i)$  then  $\vec{p}_i \leftarrow \vec{x}_i$ 
6:   end if
7: end for
8:  $\vec{g} \leftarrow$  nejlepší z  $\{\vec{p}_i\}$ 
9:   for každou částici  $i$  do
10:    aktualizuj rychlost a pozici dle pohybových rovnic
11:  end for
12: until je dosaženo maxima iterací nebo požadované přesnosti
```

Číselný příklad. Spočítejme jeden krok pro částici v $\mathbb{R}^2$. Ta má pozici $\vec{x} = (1; 2)$ a rychlost $\vec{v} = (0,5; -0,5)$, její vlastní nejlepší pozice je $\vec{p} = (0; 3)$ a nejlepší pozice roje $\vec{g} = (-1; 1)$. Parametry

jsou $w = 0,7$, $c_1 = c_2 = 1,5$ a náhodná čísla vyšla $r_1 = 0,4$, $r_2 = 0,8$.

$$\begin{aligned}\vec{v}' &= 0,7(0,5; -0,5) + 1,5 \cdot 0,4((0; 3) - (1; 2)) + 1,5 \cdot 0,8((-1; 1) - (1; 2)) \\ &= (0,35; -0,35) + 0,6(-1; 1) + 1,2(-2; -1) = (-2,65; -0,95), \\ \vec{x}' &= (1; 2) + (-2,65; -0,95) = (-1,65; 1,05).\end{aligned}$$

Částice tedy neskončila u $\vec{g}$, nýbrž přeletěla za ně. To je pro PSO typické a je to zdroj explorační: roj cíl obkružuje a přitom prohledává jeho okolí. Přestřelení ale nesmí být neomezené, a tak se rychlost buď shora ořezává podmínkou $|v_j| \leq v_{\max}$, nebo se použije *constriction faktor* (Clerc a Kennedy), který celou aktualizaci srazí jedním koeficientem: $\vec{v} \leftarrow \chi[\vec{v} + c_1 r_1(\vec{p} - \vec{x}) + c_2 r_2(\vec{g} - \vec{x})]$, kde $\chi = 2/|2 - \varphi - \sqrt{\varphi^2 - 4\varphi}|$ pro $\varphi = c_1 + c_2 > 4$. Pro obvyklou volbu $c_1 = c_2 = 2,05$ vychází $\chi \approx 0,729$ a tato varianta má zaručenou konvergenci roje.

Topologie sousedství. Sdílet jedno globální $\vec{g}$ znamená, že se každý dozví o každém zlepšení okamžitě, což roj rychle stáhne k jednomu místu. Této topologii se říká *gbest* a odpovídá úplnému grafu. Alternativa *lbest* nechává částici ohlížet se jen na nejlepšího jedince ve svém lokálním okolí:

- **Kruh**, kde každá částice má k sousedů, šíří informaci pomalu. Konvergence je proto pomalejší, zato je lepší explorační a menší riziko uvíznutí.
- Používá se také **Von Neumannova mřížka**, **hvězda** nebo **náhodné grafy**.
- Empiricky platí, že *gbest* je rychlejší na jednoduchých úlohách, kdežto *lbest* je robustnější na úlohách multimodálních.

Varianty a vztah k EA. Existuje i diskrétní a binární PSO, kde se rychlost interpretuje jako pravděpodobnost překlopení bitu přes sigmoidu. Dále se používá PSO s více roji, hybridizace s lokálním prohledáváním nebo *grammatical swarm*. Následující tabulka shrnuje, čím se PSO liší od genetického algoritmu.

	genetický algoritmus	PSO
společné	náhodná inicializace, fitness, stochastické aktualizace	totéž
jedinci	vznikají a zanikají	trvale existují, pohybují se
paměť	jen v populaci	explicitní ($\vec{p}_i$, $\vec{g}$)
operátory	křížení, mutace	žádné, jen pohyb
tok informace	obousměrný mezi rodiči	jednosměrný: od lepších k ostatním

4.2 Optimalizace mravenčí kolonií (ACO)

♦ nad rámec slidů: ve slidech EVA je z rojových algoritmů jen PSO; okruh ale mluví o rojových algoritmech v množném čísle a ACO je jejich kanonický druhý zástupce]

Ant colony optimization (M. Dorigo, 1991) napodobuje způsob, jakým hledají cestu mravenci. Mravenec za sebou pokládá *feromonovou stopu*. Po kratší cestě se vrátí dřív, takže se na ní feromon hromadí rychleji, a silnější stopa přitáhne další mravence: je to kladná zpětná vazba, která krátkou cestu sama zvýrazní. Aby se kolonie nezasekla na první slušné trase, feromon se navíc **odpařuje**, čímž systém postupně zapomíná zastaralou informaci.

Stigmergie

Nepřímá komunikace prostřednictvím změn v prostředí. Mravenci spolu nekomunikují přímo; jediným kanálem je feromon na hranách. Je to obecný princip reaktivních multiagentních architektur.

Ilustrace — Steelsův průzkumník Marsu. Že stigmergie stačí i na netriviální kolektivní úlohu, ukazuje známý myšlenkový příklad. Roj robotů má sbírat vzorky hornin a základna vysílá navigační signál, takže k návratu stačí sledovat gradient. Robot má k dispozici „drobky“, tedy radioaktivní značky sloužící k nepřímé komunikaci. Chování zadává pět pravidel s prioritou. R1: vidíš-li překážku, změň směr. R2: neseš-li vzorek a jsi na základně, polož ho. R3: neseš-li vzorek, jdi po gradientu nahoru. R4: vidíš-li vzorek, seber ho. R5: jinak se pohybuji náhodně. Druhá iterace návrhu přidá stigmergii. R7: neseš-li vzorek a nejsi na základně, polož 2 drobky a jdi po gradientu nahoru. R8: cítíš-li drobek, seber 1 drobek a jdi po gradientu dolů. Tím vznikne efektivní kolektivní sběr z bohatých nalezišť, aniž by roboti měli mapu či vzájemnou komunikaci.

Obecné schéma ACO. Aby se mravenci dali nasadit na optimalizační úlohu, převede se úloha na hledání cesty v ohodnoceném grafu; mravenci jsou pak agenti, kteří skládají řešení po hranách. Jedna iterace má čtyři kroky:

1. Každý mravenec stochasticky zkonstruuje řešení, tedy posloupnost hran.
2. Volitelně se provedou *akce démona* (*daemon actions*), což je centralizovaný krok, typicky lokální prohledávání zlepšující nalezené cesty.
3. Porovnají se kvality nalezených řešení.
4. Aktualizují se feromony na hranách, a to odpařením a uložením nového feromonu.

Volbu následujícího města řídí kompromis mezi tím, co se osvědčilo kolonii, a tím, co vypadá dobře lokálně.

Ant System (AS) — pravidlo přechodu

Mravenec k v městě i zvolí následující město j z množiny dosud nenavštívených $\mathcal{N}_i^k$ s pravděpodobností

$$p_{ij}^k = \frac{[\tau_{ij}]^\alpha [\eta_{ij}]^\beta}{\sum_{l \in \mathcal{N}_i^k} [\tau_{il}]^\alpha [\eta_{il}]^\beta}, \quad \eta_{ij} = \frac{1}{d_{ij}},$$

kde τ_{ij} je množství feromonu na hraně, η_{ij} heuristická „viditelnost“, tedy převrácená vzdálenost, a α, β váhy obou vlivů (typicky $\alpha = 1, \beta \in [2, 5]$).

Ant System — aktualizace feromonu

$$\tau_{ij} \leftarrow (1 - \rho) \tau_{ij} + \sum_{k=1}^m \Delta \tau_{ij}^k, \quad \Delta \tau_{ij}^k = \begin{cases} Q/L_k & \text{prošel-li mravenec } k \text{ hranou } (i, j), \\ 0 & \text{jinak,} \end{cases}$$

kde $\rho \in (0, 1)$ je míra odpařování a L_k délka trasy mravence k . Příspěvek je nepřímo úměrný délce trasy, takže kratší trasa uloží více feromonu.

Číselný příklad (pravidlo přechodu). Mravenec stojí ve městě i a může jít do měst A, B, C se vzdálenostmi $d = (5; 10; 2)$ a feromony $\tau = (2; 4; 1)$, přičemž $\alpha = 1$ a $\beta = 2$. Viditelnosti jsou převrácené vzdálenosti, tedy $\eta = (0,2; 0,1; 0,5)$, a nenormované váhy vyjdou takto:

$$w_A = 2 \cdot 0,2^2 = 0,08, \quad w_B = 4 \cdot 0,1^2 = 0,04, \quad w_C = 1 \cdot 0,5^2 = 0,25, \quad \sum = 0,37.$$

Po vydělení součtem dostaneme $p_A = 0,216$, $p_B = 0,108$ a $p_C = 0,676$. Přestože hrana do B nese nejvíce feromonu, rozhodne krátká vzdálenost do C , protože se do váhy promítne umocněná na $\beta = 2$.

Číselný příklad (aktualizace). Necht' $\tau_{ij} = 2$, $\rho = 0,5$, $Q = 1$ a hranou prošli dva mravenci s délkami tras $L_1 = 20$ a $L_2 = 25$. Nejprve se polovina feromonu odpaří, pak oba mravenci přispějí:

$$\tau_{ij} \leftarrow 0,5 \cdot 2 + \left(\frac{1}{20} + \frac{1}{25}\right) = 1 + 0,09 = 1,09.$$

ACS (Ant Colony System). Základní Ant System je spíše ilustrativní; teprve ACS je vylepšení konkurenceschopné se specializovanými řešiči TSP. Inspiruje se Q-learningem a mění tři věci:

- **Globálně aktualizuje jen nejlepší řešení** (elitismus), takže feromon přidává pouze globálně, případně iteračně nejlepší mravenec.
- **Lokální aktualizace** sníží feromon na hraně pokaždé, když po ní někdo projde ($\tau \leftarrow (1 - \xi)\tau + \xi\tau_0$). Hrana se tím stává méně atraktivní pro ostatní mravence a roste rozmanitost tras.
- **Pseudonáhodné pravidlo výběru** rozhoduje mezi exploatací a explorací losem: s pravděpodobností q_0 se zvolí deterministicky nejlepší hrana podle $\tau_{il}\eta_{il}^\beta$, jinak se použije standardní pravidlo AS.

Další varianty. V *Elitist AS* přispívá nejlepší trasa navíc oproti ostatním. *Rank-based AS* nechá přispět N nejlepších mravenců a váhu příspěvku určí podle pořadí. Algoritmus **MAX-MIN AS** drží feromon v intervalu $[\tau_{\min}, \tau_{\max}]$, aktualizuje jen podle nejlepšího řešení a inicializuje na $\tau_{\max}$, čímž brání předčasné stagnaci. Existuje i ACO pro spojitou optimalizaci, kde se prostor diskretizuje na podintervaly a feromon vede hledání do lepších podprostorů.

Vlastnosti a aplikace. ACO je decentralizované, řešení v něm emerguje a komunikace je nepřímá, takže má blízko k reaktivním architekturám agentů, jmenovitě k Brooksově subsumpční architektuře. Velkou předností je schopnost **optimalizovat v dynamickém prostředí**: změní-li se graf, feromony se změně samy přizpůsobí. Používá se na TSP, směřování vozidel, směřování v sítích, sekvenování DNA, skládání proteinů, detekci hran v obraze a rozvrhování.

4.3 Další rojové algoritmy

[♦ nad rámec slidů]

Rojových metod vznikly desítky a každá si bere metaforu od jiného živočicha. Za pozornost stojí zejména tyto:

- **Umělá včelí kolonie** (*artificial bee colony*, ABC; Karaboga 2005) chápe populaci zdrojů potravy jako populaci řešení a rozděluje včely do tří rolí. *Dělnice* prohledávají okolí svého zdroje. *Pozorovatelky* si zdroj vyberou ruletou podle kvality a teprve pak prohledávají jeho okolí. *Průzkumnice* řeší stagnaci: zdroj, který se *limit* pokusů nezlepšil, se opustí a nahradí náhodným, což je vestavěný restart.
- **Firefly algorithm** (Yang, 2008) pracuje se světluškami, jejichž jasnost je úměrná fitness a se vzdáleností klesá podle $I(r) = I_0 e^{-\gamma r^2}$. Méně jasná světluška se posouvá směrem k jasnější: $\vec{x}_i \leftarrow \vec{x}_i + \beta_0 e^{-\gamma r_{ij}^2} (\vec{x}_j - \vec{x}_i) + \alpha \vec{e}$. Protože přitažlivost má omezený dosah, roj se přirozeně dělí na podskupiny, a algoritmus se proto hodí na multimodální úlohy.
- **Bat algorithm, cuckoo search, grey wolf optimizer** a desítky dalších „metaforických“ algoritmů se objevují stále. Sklízají ovšem kritiku, že mnohé z nich jsou jen přejmenované varianty PSO či ES a přinášejí spíše terminologický zmatek než nové principy.

4.4 Shrnutí ke zkoušce

- Jádrem PSO jsou pohybové rovnice $\vec{v} \leftarrow w\vec{v} + c_1 r_1 (\vec{p} - \vec{x}) + c_2 r_2 (\vec{g} - \vec{x})$ a $\vec{x} \leftarrow \vec{x} + \vec{v}$, kde se skládá setrvačnost w (klesá $0,9 \rightarrow 0,4$) se složkou kognitivní a sociální při $c_1 = c_2 = 2$. Přestřelení cíle se krotí omezením $v_{\max}$ nebo constriction faktorem $\chi \approx 0,729$ a roj může být propojen topologií gbest, nebo lbest (kruh, mřížka).
- Proti GA se PSO liší tím, že nemá žádné genetické operátory, částice si pamatují $\vec{p}_i$ a $\vec{g}$ a informace teče jednosměrně od lepších k horším.
- V ACO je pravděpodobnost přechodu úměrná $p_{ij} \propto \tau_{ij}^\alpha \eta_{ij}^\beta$ s viditelností $\eta = 1/d$ a feromon se aktualizuje jako $\tau \leftarrow (1 - \rho)\tau + \sum_k Q/L_k$, tedy odpařováním a uložením úměrným kvalitě trasy.

- Vyplatí se umět odlišit původní AS, kde aktualizují všichni mravenci, od ACS s globálním updatem jen podle nejlepšího, lokálním updatem při průchodu a pseudonáhodným pravidlem s parametrem q_0 , a od MAX–MIN AS, který drží meze feromonu. Akce démona jsou centralizovaný krok, prakticky lokální prohledávání.
- Z dalších metod stojí za zapamatování ABC s rolemi dělnic, pozorovatelek a průzkumnic, kde limit nezlepšení funguje jako restart, a firefly algorithm, v němž přitažlivost klesá exponenciálně se vzdáleností.
- Klíčová slova celé kapitoly jsou stigmergie, emergence a decentralizace; mechanismus stojí na kladné zpětné vazbě doplněné odpařováním a díky tomu se metody hodí i do dynamického prostředí.

5 Lokální prohledávání a memetické algoritmy

5.1 Lokální prohledávání a horolezecký algoritmus

Lokální metody pracují s jediným řešením, které postupně vylepšují drobnými změnami. Aby to bylo možné, musíme nejdřív říct, které změny jsou ještě drobné. K tomu slouží pojem okolí.

Okolí a lokální prohledávání

Pro prostor řešení X definujeme *okolí* $N(x) \subseteq X$ jako množinu řešení, do nichž se z x dostaneme jedinou elementární změnou; podle úlohy to bývá překlopení bitu, prohození dvou měst nebo krok 2-opt. *Lokální prohledávání* pak iterativně přechází z aktuálního řešení do některého souseda podle zvoleného pravidla. *Lokální optimum* vzhledem k N je takové x , že žádný jeho soused není lepší. Tato vlastnost tedy závisí na definici okolí: co je lokálním optimumem v jednom okolí, nemusí jím být v jiném!

Nejjednodušší lokální metodou je horolezecký algoritmus, který se od nejbližších příbuzných liší jen tím, jak souseda vybírá a kdy jej přijme.

Horolezecký algoritmus (*hill climbing*)

- 1. Steepest ascent (nejstrmější stoupání):** algoritmus projde *celé* okolí a přejde do nejlepšího souseda, je-li lepší než x ; jinak skončí.
- 2. First-improvement (první zlepšení):** okolí se prochází jen do prvního nalezeného lepšího souseda, do něhož se rovnou přejde. Je to levnější a často stejně dobré.
- 3. Stochastic hill climbing:** vybere se náhodný soused a přijme se, je-li lepší; v jiné variantě se přijímá s pravděpodobností úměrnou zlepšení.
- 4. Random-restart hill climbing:** po uvíznutí se začne z nové náhodné pozice a nejlepší dosud nalezené řešení si algoritmus pamatuje. S rostoucím počtem restartů konverguje pravděpodobnost nalezení globálního optima k 1.

Slabiny má horolezec tři a všechny plynou z toho, že se dívá jen o krok dopředu. Uvízne v lokálním optimu, zastaví se na *plošinách* (plateau, kde jsou všichni sousedé stejní a pomáhají *sideways moves*) a má potíže na hřebenech. Proti tomu stojí jeho přednosti: je jednoduchý, vystačí s malou pamětí a je rychlý. Právě proto se hodí jako vylepšovací procedura uvnitř evolučního algoritmu.

5.2 Simulované žíhání

Nejpřímočařejší způsob, jak horolezce z lokálního optima vyvést, je dovolit mu občas krok dolů. Otázka pak zní, jak často a jak hluboko.

Simulované žíhání (*simulated annealing*, SA)

Metodu navrhli Kirkpatrick, Gelatt a Vecchi (1983) na základě fyzikální analogie s pomalým chladnutím kovu, při němž atomy zaujmou uspořádání s minimální energií. Algoritmus je horolezec, který **připouští i zhoršující kroky**, a to s pravděpodobností klesající s velikostí zhoršení a s teplotou.

Konkrétní podobu té pravděpodobnosti dává Metropolisovo kritérium.

Metropolisovo kritérium

Pro minimalizaci, kandidáta $y \in N(x)$ a $\Delta = f(y) - f(x)$ platí:

$$P(\text{přijmout } y) = \begin{cases} 1, & \Delta \leq 0 \quad (\text{zlepšení}), \\ e^{-\Delta/T}, & \Delta > 0 \quad (\text{zhoršení}), \end{cases}$$

kde $T > 0$ je teplota.

Číselný příklad. Mějme zhoršení $\Delta = 2$. Při $T = 1$ vyjde $P = e^{-2} = 0,135$, při $T = 0,5$ klesne na $P = e^{-4} = 0,018$ a při $T = 5$ naopak stoupne na $P = e^{-0,4} = 0,67$. Na začátku běhu, kdy je horko, se tedy algoritmus chová téměř jako náhodná procházka; na konci, kdy je chladno, jako čistý horolezec.

Algoritmus 8 Simulované žíhání

```
1:  $x \leftarrow$  počáteční řešení;  $T \leftarrow T_0$ ;  $x^* \leftarrow x$ 
2: while  $T > T_{\min}$  a není splněna ukončovací podmínka do
3:   for  $k$  krát (délka Markovova řetězce při dané teplotě) do
4:     vyber náhodně  $y \in N(x)$ ;  $\Delta \leftarrow f(y) - f(x)$ 
5:     if  $\Delta \leq 0$  nebo  $\text{rand}() < e^{-\Delta/T}$  then  $x \leftarrow y$ 
6:     end if
7:     if  $f(x) < f(x^*)$  then  $x^* \leftarrow x$ 
8:     end if
9:   end for
10:   $T \leftarrow \text{cool}(T)$ 
11: end while
```

Výstup: x^*

O tom, jak rychle teplota klesá, rozhoduje rozvrh chlazení.

Rozvrh chlazení (*cooling schedule*).

- *Geometrický*: teplota se v každém kroku vynásobí konstantou, tedy $T_{k+1} = \alpha T_k$ pro $\alpha \in [0,8; 0,99]$. V praxi je to nejběžnější volba.
- *Lineární*: teplota klesá o pevný krok podle $T_k = T_0 - \beta k$.
- *Logaritmický*: $T_k = c / \log(k + 1)$. Je to jediný rozvrh, pro který je dokázána konvergence.
- Adaptivní rozvrhy se řídí průběhem běhu; patří k nim *reheating*, kdy se při stagnaci teplota znovu zvýší.

Věta 5.1 (Konvergence SA). *Při logaritmickém rozvrhu $T_k \geq c / \log(k + 1)$ s dostatečně velkou konstantou c (souvisí s výškou nejhlubší „bariéry“ v krajině) konverguje simulované žíhání k množině globálních optim s pravděpodobností 1.*

Prakticky je ovšem takový rozvrh nepoužitelně pomalý. Simulované žíhání je proto teoreticky uspokojivé, ale v praxi se sahá po rychlejších heuristických rozvrzích.

Formálně je SA nehomogenní Markovův řetězec. Při pevné T konverguje k Boltzmannovu rozdělení $\pi(x) \propto e^{-f(x)/T}$, které se pro $T \rightarrow 0$ koncentruje na globálních optimech. Tatáž myšlenka se objevuje

i v evolučních algoritmech: *Boltzmannův turnaj* (odst. 1.1.5) ji přenáší do selekce.

5.3 Tabu prohledávání

Simulované žíhání se z lokálního optima dostává náhodou. Tabu prohledávání volí opačnou cestu: cíleně si zakáže tahy, které by je vrátilo tam, odkud právě přišlo.

Tabu search (Glover, 1986)

Lokální prohledávání doplněné o *krátkodobou paměť*. Uchovává se *tabu list* nedávno provedených tahů, což je efektivnější než ukládat celá řešení, a tyto tahy jsou po dobu zvanou *tabu tenure* zakázány. V každém kroku algoritmus přejde do **nejlepšího nezakázaného souseda**, a to i tehdy, je-li horší než aktuální řešení. Právě tím se dostane z lokálního optima a zároveň necyklí.

- **Tabu tenure** T může být statická, tedy pevný počet iterací, nebo dynamická, kdy se losuje z nějakého rozsahu.
- **Aspirační kritérium** dovoluje tabu porušit, vede-li tah k řešení lepšímu než dosud nejlepší nalezené. Bez něj by tabu mohlo blokovat postup.
- **Střednědobá paměť** jsou pravidla, která hledání směřují do slibných oblastí, tedy zajišťují intenzifikaci.
- **Dlouhodobá paměť**, jinak též frekvenční, je počítadlo toho, jak často byl který prvek řešení použit. Často navštěvované konfigurace se penalizují, což slouží k diverzifikaci, případně se restartuje z málo navštívené oblasti.

Tabu search se používá na TSP (výměny hran, *ejection chains*), na rozvozní úlohy, na sekvenování DNA a na rozvrhování. Mluví pro něj únik z lokálních optim, absence cyklení a to, že funguje v diskrétních i spojitých doménách. Proti němu stojí velký počet parametrů a vyšší výpočetní náročnost daná tím, že se prochází celé okolí.

5.4 Scatter search

Scatter search je populační metaheuristika zaměřená na **diverzitu**. Udrží malou *referenční množinu* R o řádově desítkách jedinců a vybírá do ní podle kombinace kvality a vzájemné vzdálenosti: u nově přidávaného jedince se maximalizuje jeho minimální vzdálenost od zbytku R .

Cyklus má pět kroků. Začíná (1) *diverzifikací*, tedy chytroou inicializací. Pak přijde (2) *generování podmnožin*, typicky všech dvojic z R , a jejich (3) *kombinace* aritmetickým či permutačním křížením. Výsledek projde (4) *vylepšením* lokálním prohledáváním, mutací nebo tabu search a nakonec se (5) *aktualizuje* R .

Aktualizovat referenční množinu lze třemi způsoby. *Inkrementálně*, kdy za generaci přibude jeden či několik jedinců; *úplnou obnovou* každou generaci; nebo *průběžně* už ve vnitřním cyklu, kdy se noví jedinci do R zařazují okamžitě a rekombinuje se rovnou s nimi. Metoda se hodí pro úlohy s velmi drahou fitness, například pro black-box optimalizátor OptQuest.

5.5 Memetické algoritmy

Všechny dosavadní lokální metody mají společné to, že jeden běh nijak netěží ze zkušenosti běhu předchozího. Memetické algoritmy tuhle mezeru zaplňují tím, že lokální prohledávání zasadí dovnitř evoluce.

Mem a memetický algoritmus

Mem (Dawkins, 1976) je jednotka kulturní informace, která se šíří napodobováním; bývá popsán jako „gen kultury“. *Memetický algoritmus* (Moscato, 1989) je hybridní metoda: **evoluční algoritmus s vnořeným lokálním prohledáváním**, které vylepšuje jednotlivé jedince, a vykládá se proto jako „individuální učení“. Setkáme se i se synonymy hybridní EA, genetické lokální

Algoritmus 9 Memetický algoritmus

```

1: inicializuj populaci
2: while není splněna ukončovací podmínka do
3:   ohodnoť jedince v populaci
4:   vytvoř novou populaci stochastickými operátory (selekce, křížení, mutace)
5:   vyber podmnožinu  $O$  jedinců, kteří projdou vylepšením
6:   for každého  $x \in O$  do
7:     proved' individuální učení  $x$  pomocí memu (lokálního prohledávače), s pravděpodobností
        $p$  po dobu  $t$ 
8:     výsledek zpracuj lamarckovsky nebo baldwinovsky
9:   end for
10: end while

```

Podstatou metody je lokální prohledávání vnořené do evolučního cyklu. Genetické operátory přenášejí informaci *mezi* jednotlivými běhy lokálního prohledávače, takže celkové hledání je mnohem efektivnější než opakovaný restart samotné lokální metody (Krasnogor & Smith). V roli memu může vystupovat hill climbing, simulované žíhání, tabu search, kombinatorická heuristika jako 2-opt, ale i gradientní metoda, typicky zpětné šíření chyby při učení neuronové sítě.

Lamarckismus vs. baldwinismus

Co udělat s vylepšeným jedincem $x' = \text{LS}(x)$?

- **Lamarckovsky:** do populace se vloží x' i s jeho fitness, takže *genotyp se změní podle fenotypu*. Z darwinistického hlediska je to „nekošer“, prakticky je to ale rychlejší.
- **Baldwinovsky:** jedinci x se přiřadí *fitness* vylepšeného x' , ale genotyp x zůstane *nezměněn*. To je darwinisticky korektní a fitness se pak interpretuje jako „potenciál“ jedince. V tom spočívá *Baldwinův efekt*: schopnost učit se jedince zvýhodňuje a evoluce se totéž posléze „doučí“ sama, což se označuje jako genetická asimilace.

Lamarckismus konverguje rychleji, zato více snižuje diverzitu a může zkreslit krajinu. Baldwinismus diverzitu zachovává a fitness krajinu vyhlazuje, je však pomalejší.

Návrhové otázky. Při návrhu memetického algoritmu se rozhoduje o třech věcech. Které jedince vylepšovat: všechny, jen elitu, nebo náhodný podíl p ? Jak dlouho, tedy kolik kroků t ? A jak silně, tedy do jaké hloubky lokální prohledávání pustit? Přílišné lokální prohledávání znamená zbytečné náklady a ztrátu diverzity, příliš malé nepřinese žádný užitek.

Memetické počítání (*memetic computing*). Zobecnění, v němž memy nejsou známy předem, ale jsou samy předmětem hledání.

- **Multi-mem MA:** mem je zakódován *jako součást genotypu* jedince. Dědí se i kříží (potomek zdědí mem lepšího rodiče, nebo se mem vybere náhodně) a používá se k vylepšení právě tohoto jedince.
- **Meta-lamarckovský MA:** memy mají *vlastní populaci* a koevolvuji s jedinci. Při vylepšení se jedinci přiřadí mem, úspěch jedince je pro mem odměnou a nad memy běží druhý evoluční algoritmus.
- **Memetický prostor** je protějškem prostoru fenotypů. Memy v něm lze vybírat heuristikami, nechat je soutěžit podle úspěšnosti, dělit do subpopulací nebo je řídit meta-memetickým algoritmem.

5.6 Ladění a řízení parametrů

S memetickými algoritmy úzce souvisí otázka, odkud se berou hodnoty parametrů. Evoluční algoritmus jich má mnoho: velikost populace, p_C , p_M , velikost turnaje, míru elitismu. Nejsou na sobě

nezávislé a vyčerpávající prohledání všech kombinací není možné. Podle toho, kdy se o nich rozhoduje, se rozlišuje ladění od řízení.

- **Ladění** (*tuning*) určí parametry *před* během. Používá se pokus-omyl, grid search, meta-EA nebo iterované závodění (irace).
- **Řízení** (*control*) mění parametry *během* běhu a má tři podoby:
 - *Pevná strategie* (deterministická) závisí jen na čase, například $\sigma(t) = 1 - 0,9t/T$ nebo pokles p_M o 0,1 % za generaci; její stochastickou variantou je simulované žihání.
 - *Adaptivní strategie* se řídí zpětnou vazbou z běhu, tedy kvalitou řešení, rozptylem fitness či diverzitou. Patří sem pravidlo 1/5, hypermutace při poklesu fitness a adaptivní zjemňování reprezentace, kdy se zvyšuje počet bitů na reálné číslo, klesne-li diverzita měřená Hammingovou vzdáleností nejlepšího a nejhoršího jedince.
 - *Samoadaptivní strategie* dělá z parametrů součást genomu a nechává je evolvovat; tak pracují ES, EP i multi-mem MA.
- Řídit lze i *velikost populace*, byť jen nepřímo: každému novému jedinci se přidělí „délka života“ úměrná jeho fitness a po jejím vypršení jedinec zemře. Lepší jedinci žijí déle, takže velikost populace je odvozená dynamická veličina.

5.7 Shrnutí ke zkoušce

- Horolezecký algoritmus má čtyři varianty: steepest ascent, first improvement, stochastickou a s náhodnými restarty. Naráží na lokální optima, plošiny a hřebeny.
- Simulované žihání přijímá kroky podle Metropolisova kritéria $P = \min(1, e^{-\Delta/T})$. Z rozvrhů chlazení se v praxi používá geometrický, zatímco logaritmický je ten, pro který je dokázána konvergence; pro pevné T řetězec konverguje k Boltzmannovu rozdělení $\propto e^{-f/T}$.
- Tabu search si drží tabu list tahů, jejich tenure, aspirační kritérium a střednědobou i dlouhodobou (frekvenční) paměť. Vždy jde do nejlepšího nezakázaného souseda, i když je horší než aktuální řešení.
- Scatter search udržuje referenční množinu kombinující kvalitu a diverzitu a pracuje s aritmetickými kombinacemi doplněnými o vylepšení.
- Memetický algoritmus je evoluční algoritmus doplněný o lokální prohledávání nad jedinci. **Lamarckismus** mění genotyp a je rychlejší, **baldwinismus** mění jen fitness a zachovává diverzitu. Memetic computing jde dál a umisťuje memy do genomu nebo do koevolvující populace.
- U parametrů se rozlišuje ladění před během od řízení během běhu, přičemž strategie řízení mohou být pevné, adaptivní, nebo samoadaptivní.

6 Aplikace evolučních algoritmů

Poslední věta okruhu vyjmenovává čtyři aplikační oblasti a každá z nich tvoří jeden oddíl této kapitoly. Přestože se úlohy liší, mají jedno společné: o úspěchu v nich rozhoduje **volba reprezentace a operátorů**, nikoli volba „lepšího“ evolučního algoritmu. Právě proto stojí za to procházet aplikace jednu po druhé a sledovat u každé, co je v ní jedincem a jak se s ním smí zacházet.

6.1 Evoluce pravidlových a expertních systémů

6.1.1 Michigan vs. Pittsburgh

Úloha zní: naučit se celou **sadu pravidel** tvaru IF podmínka THEN akce, a to buď z trénovacích dat, nebo z interakce s prostředím. Takový výstup se hodí do dobývání znalostí a do expertních systémů, ale stejně dobře poslouží při řízení robotů a ve zpětnovazebním učení. Zbývá rozhodnout jedinou, zato zásadní věc: co bude jedincem. Odpovědi jsou dvě a vedly ke dvěma základním přístupům, které se liší prakticky ve všem ostatním:

	Michigan (Holland)	Pittsburgh (Smith)
jedinec	jedno pravidlo	celá sada pravidel
řešení	celá populace dohromady	jeden jedinec
fitness	síla/přesnost pravidla (nutné rozdělení odměny)	kvalita celého systému (přímočará)
operátory	standardní, evoluce probíhá jen občas a na části populace	složitě, desítky operátorů nad sadami i jednotlivými pravidly
výhody	on-line učení, malý prostor, přirozené pro RL	jednoznačná fitness, žádný credit assignment
nevýhody	přiřazení zásluh, pravidla musí spolupracovat, ale zároveň soutěží	velcí jedinci, drahé vyhodnocení, riziko bloatu

6.1.2 Learning classifier systems (Michigan)

Michiganskou větev reprezentují *learning classifier systems*, tedy systémy, v nichž se znalost neschovává do jednoho jedince, ale do celé populace.

LCS (*learning classifier system*)

Systém, kde populace jednoduchých pravidel (*klasifikátorů*) tvoří dohromady expertní či řídicí systém. Klasifikátor má levou stranu (podmínku) jako řetězec nad $\{0, 1, *\}$ porovnávaný se vstupem, pravou stranu (akci či klasifikační třídu) a **váhu** / **sílu** odrážející jeho úspěšnost, která slouží jako fitness. Evoluce neprobíhá generačně, ale průběžně a jen na části populace.

Architektura LCS. Klíčové je uvědomit si, že *expertním systémem je celá populace*, nikoli jedinec. Systém pracuje ve smyčce, která má šest kroků:

1. **Detektory** sejmou stav prostředí a převedou jej na vstupní zprávu.
2. Zpráva se uloží do **bufferu zpráv** a porovná se s levými stranami všech pravidel; ta pravidla, která jí vyhoví, tvoří dohromady *match set*.
3. Mezi vyhovujícími pravidly proběhne **aukce/soutěž** o právo jednat, v níž rozhoduje síla pravidla.
4. Vítězné pravidlo zapíše svou pravou stranu buď do bufferu jako vnitřní zprávu, nebo ji přes **efektory** pošle do prostředí jako akci.
5. Prostor vrátí **odměnu**, kterou mechanismus *bucket brigade* rozdělí mezi pravidla, jež k ní vedla.
6. Občas a jen na části populace se spustí **GA**, který pravidla obměňuje.

Problém reaktivity. Čistě reaktivní systém nemá vnitřní paměť, takže se nedokáže rozhodovat podle ničeho jiného než podle okamžitého vjemu. Holland proto zavedl *zprávy (messages)*: pravá strana pravidla může místo akce zapsat vnitřní zprávu do bufferu a levá strana má *receptory*, jimiž ji zachytí. Pravidla se tak dají řetězit za sebe a systém tím získá vnitřní stav, a s ním i výpočetní sílu. Za to ovšem platí novým problémem: jak rozdělit jedinou odměnu mezi celý řetězec pravidel, který k ní vedl.

Bucket brigade (algoritmus „požárního řetězu“)

Mechanismus rozdělení odměny v LCS. Pravidlo, které chce být aplikováno, musí „zaplatit“ část své síly pravidlu, jež ho aktivovalo (jako by kupovalo právo jednat). Odměna z prostředí připadne poslednímu článku řetězu a postupně „proteče“ zpět k pravidlům, která k ní vedla. Prakticky je obtížné vyvážit tuto ekonomiku pravidel, dnes se proto téměř nepoužívá.

ZCS (Wilson, 1994) — „zero-level“ **LCS**. Wilson šel opačnou cestou než Holland a systém radikálně zjednodušil: žádné vnitřní zprávy, žádná složitá redistribuce odměny. Zůstane tohle:

- Populaci pravidel označíme P , vstup přicházející z detektorů je x .

- Do *match setu* M patří ta pravidla, jejichž podmínka odpovídá vstupu x .
- Akce a se z M vybere ruletou podle síly pravidel; pravidla, která prosazují právě zvolenou akci, tvoří *action set* $A \subseteq M$.
- **Posílení:** z pravidel v A se nejdříve odebere podíl β jejich síly do „kbelíku“ B . Jakmile dorazí odměna r , přičte se každému pravidlu v A hodnota $\beta r/|A|$ a pravidlům z předchozího action setu A_{-1} hodnota $\gamma B/|A_{-1}|$. Pravidla z $M \setminus A$, tedy ta, která vstupu odpovídala, ale prosazovala jinou akci, jsou zdaněna.
- **Cover operátor:** pokud pro aktuální vstup neexistuje žádné vyhovující pravidlo, vygeneruje se ad hoc — do podmínky se náhodně přidají hvězdičky a akce se zvolí náhodně.

XCS (Wilson, 1995) — fitness založená na přesnosti. ZCS má tři slabiny. Neevolvuje *úplný* systém pravidel, který by pokrýval všechny situace. Pravidla stojící na začátku řetězců jsou odměňována jen zřídka, a proto vymírají. A vymírají i pravidla, která vedou k malým, zato správně předpovězeným odměnám. Společnou příčinou je, že síla plnila dvě role najednou: byla fitness pro GA a zároveň kritériem výběru akce. Wilsonova myšlenka rozdělila obojí. *Predikce* má odhadovat odměnu, kdežto *evoluce* má preferovat **spolehlivé (přesné)** klasifikátory, tedy ty, jejichž predikce se trefuje, i když slibuje málo.

XCS — klasifikátor jako pětice

(c, a, p, ϵ, F) : podmínka, akce, predikce odměny p , chyba predikce ϵ , fitness F . Přesnost se počítá jako klesající funkce chyby, $\kappa = \alpha (\epsilon/\epsilon_0)^{-\nu}$ (pro $\epsilon > \epsilon_0$, jinak $\kappa = 1$; α, ϵ_0, ν jsou parametry); fitness je *relativní* přesnost v rámci action setu. Predikce a chyba se aktualizují pravidly Q-learningu.

Běh XCS pak vypadá takto. Podle vstupu se sestaví match set M a ten se rozdělí na action sety A_i podle jednotlivých akcí. Pro každou akci se předpoví výplata jako průměr predikcí vážený fitness. Akce se zvolí buď maximem, což je exploatace, nebo ruletou, což je explorace, a provede se. Odměna se rozdělí do action setu předchozího kroku: nejprve se aktualizuje predikce p pomocí $r + \gamma \max_a p_a$, pak chyba ϵ a nakonec fitness F . GA je zde **nikový** — běží pouze uvnitř action setu, nikoli nad celou populací.

Výsledek stojí za to shrnout. XCS propojuje LCS se zpětnovazebním učením, protože roli mechanismu pro přiřazení zásluh přebírá Q-learning. Evoluuje **úplnou a minimální** mapu vstup–výstup, takže výstupem není jen optimální politika, ale i kompaktní a srozumitelný popis chování. A má za sebou řadu praktických aplikací v dobývání znalostí, v řízení i v bioinformatice.

Dnešní rodina LCS. Varianty se dnes rozlišují podle toho, na jakou úlohu míří. *XCS* slouží pro zpětnovazební učení a predikuje odměnu. **UCS** (*sUpervised Classifier System*) je varianta pro učení s učitelem: správná třída je známa, takže se místo predikce odměny rovnou měří přesnost klasifikace. *XCFSF* míří na aproximaci spojitých funkcí a pravá strana pravidla v něm není konstanta, nýbrž lineární model. Podmínky se navíc dnes běžně zapisují nejen nad $\{0, 1, *\}$, ale i intervaly pro reálné atributy. Tím se z LCS stává plnohodnotný nástroj dobývání znalostí, a to s jednou předností navíc: výsledkem je **čitelná sada pravidel**.

6.1.3 Pittsburghský přístup a systém GIL

V pittsburghském pojetí je jedincem celá sada pravidel, tedy rovnou kompletní klasifikační systém. Hodnocení je proto složitější, protože se musí vypořádat s prioritami pravidel, s konflikty mezi nimi a s falešně pozitivními i negativními odpověďmi. Operátorů je celá řada a pracují na několika úrovních. Důraz se přitom klade na bohatou doménovou reprezentaci, tedy na množiny, výčty a intervaly.

Klasickým představitelem je **GIL**. Řeší binární klasifikaci a jedinec v něm klasifikuje implicitně do jedné třídy, takže pravidla vůbec nemají pravou stranu. Struktura jedince je třípatrová: jedinec je *disjunkce komplexů*, komplex je *konjunkce selektorů* (po jednom pro každou proměnnou) a selektor

je *disjunkce hodnot* z domény dané proměnné, zakódovaná bitovou maskou. Například

$$((X=A_1) \wedge (Z=C_3)) \vee ((X=A_2) \wedge (Y=B_2)) \equiv [001|11|0011 \vee 010|10|1111].$$

Každému patru odpovídají vlastní operátory:

- na úrovni *jedince* se pravidla vyměňují, kopírují, generalizují, specializují a mažou, případně se do jedince zahrne pozitivní příklad;
- na úrovni *komplexu* se komplex rozdělí na jednom selektoru, selektor se generalizuje (nahradí se 11...1) nebo specializuje, případně se zahrne negativní příklad;
- na úrovni *selektoru* jde o mutaci $0 \leftrightarrow 1$, rozšíření $0 \rightarrow 1$ a zúžení $1 \rightarrow 0$.

Tyto operátory přímo odpovídají operacím induktivního učení, tedy generalizaci a specializaci. GIL je proto vlastně evolučně řízený induktivní algoritmus.

6.1.4 Připomenutí: metriky klasifikace

Kvalitu naučeného pravidlového systému je čím měřit. Z matice záměn (TP, FP, FN, TN) se počítá přesnost $= (TP + TN)/(TP + TN + FP + FN)$, senzitivita $= TP/(TP + FN)$ a specifita $= TN/(TN + FP)$. Fitness pravidlového systému bývá kombinací přesnosti, pokrytí a jednoduchosti, kterou měří počet pravidel. Jde tedy přirozeně o vícekritériální úlohu, i když se obvykle řeší skalarizací.

6.2 Neuroevoluce

Druhou velkou aplikační oblastí je učení neuronových sítí evolucí. Vymezení pojmu se cituje takto:

Neuroevoluce (*neuroevolution*)

„Neuroevoluce je metoda pro úpravu vah, topologií nebo ansámblů neuronových sítí za účelem naučení konkrétní úlohy. Evoluční výpočty se používají k hledání parametrů sítí maximalizujících fitness, která měří výkon v úloze. Ve srovnání s jinými metodami učení je neuroevoluce velmi obecná — umožňuje učení bez explicitních cílových hodnot, s nediferencovatelnými aktivačními funkcemi a s rekurentními sítěmi.“ (Miikkulainen, 2017)

Evolvovat se dá v podstatě cokoli, co síť určuje: samotné váhy, další parametry jako aktivační funkce či konstanty učení, architektura ve smyslu topologie a propojení, architektura a váhy zároveň, a dokonce i parametry samotného učicího algoritmu. Hlavní doménou neuroevoluce je **zpětnovazební učení** a robotika, tedy úlohy, kde nejsou k dispozici cílové výstupy potřebné pro učení s učitelem.

6.2.1 Evoluce vah

Nejjednodušší je evolvovat váhy pevně dané sítě. Váhy se zakódují do vektoru pevné délky a pustí se na něj reálný GA, ES nebo DE se standardními operátory; nic dalšího není potřeba vymýšlet. Zajímavější jsou vlastnosti takového postupu:

- Pro učení s učitelem je evoluce pomalejší než specializované gradientní metody, tedy než zpětné šíření chyby.
- Výhodou je naopak snadná paralelizace a nezávislost na gradientu: postup funguje i pro nediferencovatelné a rekurentní sítě a dá se použít pro RL.
- Naráží ale na **problém konkurujících konvencí** (*competing conventions*), jinak též permutační problém: tatáž funkce sítě odpovídá mnoha různým genotypům, protože skryté neurony lze libovolně přeuspořádat. Křížení dvou funkčně shodných rodičů proto často vytvoří nefunkčního potomka.
- V posledních letech se zájem o evoluci vah vrátil. Ukazuje se, že je konkurenceschopná i pro velké sítě, pokud se hodnocení provádí na minibatchích.

6.2.2 Evoluce architektury

Jakmile se místo vah evoluuje architektura, změní se povaha fitness. Fitness architektury je totiž odhad výkonu sítě, a aby se dal spočítat, musí se síť postavit, inicializovat a naučit, ať už zpětným šířením, nebo jiným EA. Nejlépe navíc vícekrát. Vyhodnocení fitness je proto drahé a zašuměné.

- **Přímé kódování** (*direct encoding*) popisuje strukturu propojení jako binární matici. Jedinec je pak buď linearizovaná matice, na kterou stačí standardní operátory, nebo matice sama a pracuje se s ní speciálními 2D operátory. Je to jednoduché, ale neškáluje, protože velikost genomu roste kvadraticky.
- **Gramatické kódování** (Kitano, 1990) matici propojení negeneruje přímo: vyrobí ji jednoduchá 2D formální gramatika a evoluuje se *pravidla gramatiky*. Kódování je kompaktní, tedy logaritmické, a umožňuje opakující se motivy. Za to platí tím, že je nepřímé a hůře se ladí.
- **Buněčné kódování** (*cellular encoding*, Gruau) jde ještě dál: jedincem je **program v GP**, který popisuje, jak má síť *vyrůst* z jediné buňky pomocí operací přidej neuron, rozděľ neuron sériově, rozděľ paralelně, přepoj synapsi a změň váhu. Jde tedy o vývojový (developmental) přístup.
- **Simulovaný růst** spojů ve 2D rovině patřil mezi rané pokusy v evoluční robotice, ale neškáloval.

GNARL (Angeline a kol., 1993). Systém staví na evolučním programování nad grafy. Vstupní a výstupní neurony jsou pevné, kdežto skryté neurony a spoje se evoluuji, a to architektura i váhy *současně*. Mutace jsou dvojího druhu: *strukturální* přidá neuron bez spojů, přidá spoj s nulovou váhou, smaže neuron nebo smaže spoj, zatímco *parametrická* je zaujatá mutace váhy. Sílu mutace řídí *teplota*, což je přístup převzatý ze simulovaného žíhání; strukturální změny mají zůstat malé a nepřilíš destruktivní. Rodiči se stává lepší polovina populace a křížení se nepoužívá, jak je v EP zvykem.

6.2.3 NEAT

Nejvlivnější odpovědí na problém konkurujících konvencí je NEAT.

NEAT (*NeuroEvolution of Augmenting Topologies*, K. Stanley, 2002)

Síť je reprezentována jako **seznam hran**; každý gen-hrana obsahuje: vstupní a výstupní neuron, váhu, příznak *enabled/disabled* a **globální inovační číslo** (*innovation number*), které se přiděluje při prvním vzniku dané strukturální novinky.

Celý systém stojí na třech klíčových myšlenkách:

1. **Inovační čísla řeší problém konkurujících konvencí.** Kříží se pouze geny se *shodným inovačním číslem*, tedy geny se společným evolučním původem. Geny, které přebývají u jednoho rodiče (*disjoint* a *excess*), se přeberou od zdatnějšího z rodičů. Křížení tak strukturu nerozbíjí a nevzniká z něj žádné „bezhlavé kuře“.
2. **Speciace (niching) chrání inovace.** Na vektorech inovačních čísel se definuje míra podobnosti

$$\delta = \frac{c_1 E}{N} + \frac{c_2 D}{N} + c_3 \overline{\Delta W},$$

kde E (*excess*) je počet genů, jejichž inovační číslo leží za nejvyšším inovačním číslem druhého genomu, D (*disjoint*) počet genů chybějících u druhého rodiče *uvnitř* společného rozsahu inovačních čísel, $\overline{\Delta W}$ průměrný rozdíl vah genů se *shodným* inovačním číslem a N velikost většího genomu, která slouží k normalizaci. Síť, jejichž δ je pod prahem, tvoří jeden *druh* a fitness se počítá jako sdílená, tedy relativní v rámci druhu. Nová topologie tak dostane čas „doladit váhy“ dříve, než ji vytlačí zavedená řešení. Bez toho by strukturální novinka, která fitness zpočátku zhoršuje, okamžitě vymřela.

3. **Postupné narůstání složitosti** (*complexification*) znamená, že populace startuje z minimálních sítí, které mají jen vstupy a výstupy, a roste teprve mutacemi *přidej spoj* a *přidej neuron*. Druhá

z nich rozdělí existující spoj: původní se zakáže a vzniknou dva nové. Prohledávání tedy začíná v prostoru nejnižší dimenze a zvyšuje ji jen tehdy, když se to vyplatí.

6.2.4 HyperNEAT a CPPN

◆ nad rámec slidů: slidy zmiňují jednou větou, CPPN vůbec]

HyperNEAT rozšiřuje NEAT o dvě myšlenky. První je *substrát*: váhy sítě se nekódují jako vektor čísel, ale *generuje je funkce* $S : \mathbb{R}^4 \rightarrow \mathbb{R}$, která dostane souřadnice dvou neuronů v rovině a vrátí váhu jejich spojení. Odtud pochází předpona „hyper“. Druhou myšlenkou je *CPPN* (*compositional pattern-producing network*), což je způsob, jak funkci S zapsat: je reprezentovaná sítí ve tvaru DAG, jejíž uzly počítají různé funkce ze slovníku, jako je sigmoida, gaussovka, sinus či absolutní hodnota. Tuto CPPN pak evoluuje samotný NEAT. Přínos je v tom, že reprezentace vah funkcí umí vyjádřit **symetrie a opakující se motivy** a vygenerovat sítě s milióny spojů z malého genomu.

6.2.5 Neuroevoluce v éře hlubokých sítí

◆ nad rámec slidů]

Hledání architektury hlubokých sítí dostalo vlastní jméno: **NAS** (*neural architecture search*). Popisuje se třemi osami. *Prohledávaný prostor* může být posloupnost vrstev, DAG opakovaných buněk, nebo podarchitektura jedné velké „super sítě“. *Prohledávací strategií* bývají evoluční algoritmy, náhodné prohledávání jako překvapivě silná baseline, bayesovská optimalizace, zpětnovazební učení nebo gradientní DARTS. *Odhad výkonu* se zlevňuje zkráceným učením a sdílením vah. Z konkrétních systémů stojí za zapamatování **ENAS**, který sdílí váhy mezi kandidáty, **DeepNEAT**, kde je uzlem chromozomu celá vrstva, a **CoDeepNEAT**, který koevoluuje populaci modulů a populaci „blueprintů“. Samostatnou linií je **ES pro RL** (OpenAI 2017): jednoduchá evoluční strategie *pouze s mutacemi* nad milióny vah, kterou lze triviálně paralelizovat, protože uzly si mezi sebou vyměňují jen semínka generátoru náhodných čísel.

6.3 Kombinatorické úlohy a práce s omezeními

Na NP-těžké kombinatorické úlohy jsou evoluční algoritmy oblíbeným nástrojem. Optimum sice nezaručí, zato v rozumném čase poskytnou dobré řešení a snadno se přizpůsobí dodatečným omezením reálné úlohy, i když jsou tato omezení sebeošklivější.

6.3.1 Problém batohu

Zadání. Batoh má kapacitu $C_{\max}$ a k dispozici je N předmětů, z nichž každý má svou cenu v_i a svůj objem c_i . Úkolem je maximalizovat $\sum_i x_i v_i$ za podmínky $\sum_i x_i c_i \leq C_{\max}$, kde $x_i \in \{0, 1\}$ vyjadřuje, jestli předmět bereme, nebo ne.

Kódování je triviální. Jedincem je bitová mapa, takže třeba 0110010 znamená „vezmi předměty 2, 3 a 6“, a fungují na ni standardní operátory, tedy křížení a bit-flip mutace. **Problém je fitness**, protože jedinci mohou porušovat kapacitu. Stojí tu proti sobě dvě protichůdná kritéria: maximalizovat $\sum v_i$ a zároveň splnit $\sum c_i \leq C_{\max}$.

6.3.2 Tři obecné strategie pro omezení

Batoh je tu jen záminkou: potíží s nepřipustnými jedinci má každá úloha s omezeními a odpovědi na ni jsou tři. Liší se tím, kde se omezení uplatní — zda až ve fitness, po vygenerování jedince, nebo už v samotném kódování.

Práce s omezeními v EA

- 1. Penalizace** (*penalty*): nepřipustná řešení zůstávají v populaci, ale jejich fitness se sníží, např. $f'(x) = \sum_i x_i v_i - \alpha \max(0, \sum_i x_i c_i - C_{\max})$. Volba α je zásadní: příliš malá $\Rightarrow$ populace se plní nepřipustnými řešeními, příliš velká $\Rightarrow$ ztrácí se možnost projít nepřipustnou oblastí ke slibnému řešení. Používá se i dynamická (α roste v čase) či adaptivní penalizace.
- 2. Oprava** (*repair*): nepřipustné řešení se opraví (u batohu: vyhazuj předměty s nejhorším poměrem v_i/c_i , dokud se vejde). Opravu lze provést jen pro výpočet fitness (baldwinovsky) nebo trvale zapsat do genotypu (lamarckovsky).
- 3. Dekodér** (*decoder*) / uzavřené operátory: kódování se navrhne tak, že *každý* genotyp reprezentuje přípustné řešení. U batohu: bit 1 znamená „přidej předmět, pokud se ještě vejde“, jinak jej přeskoč. Elegantní a účinné; nevýhodou je nejednoznačnost mapování (více genotypů dává tentýž fenotyp) a jeho horší lokalita.

Nabízí se ještě čtvrtá možnost: chápat porušení omezení jako **druhé kritérium** a sáhnout po vícekritériálním EA (odst. 6.4). Výsledkem pak není jediné řešení, ale celá Pareto fronta kompromisů mezi kvalitou řešení a mírou porušení omezení.

6.3.3 Problém obchodního cestujícího

Úloha TSP zní jednoduše: pro N měst najdi nejkratší okružní jízdu. Fitness je zde triviální, protože jí je délka cesty, a o nastavení evolučního algoritmu se tentokrát mluvit nemusí. **Problémem je kódování a zejména křížení**, protože z dvojice platných cest se ledabylým operátorem snadno stane cesta, která žádnou cestou není.

Reprezentace.

- Sousednostní** (*adjacency*) reprezentace má na pozici i město j právě tehdy, vede-li hrana $i \rightarrow j$. Například (248397156) odpovídá cestě 1–2–4–3–8–5–9–6–7. Každá cesta má jedinou reprezentaci, ale ne každý seznam je platná cesta a klasické křížení nefunguje. Zajímavé je, že schémata tu dávají smysl: ($*3 * \dots$) jsou třeba všechny cesty s hranou 2–3. Přesto platí: *nepoužívat*.
- Ordinální** (*buffer*) reprezentace si udržuje seznam měst, gen i je *pozice* města v aktuálním seznamu a použité město se ze seznamu vyjme. Cesta 1–2–4–3–8–5–9–6–7 se tedy pro seznam (123456789) zakóduje jako (112141311). Motivací bylo umožnit standardní jednobodové křížení. To sice produkuje platné cesty, jenže výsledek nemá s rodiči nic společného. *Také nepoužívat*.
- Cestová (permutační)** reprezentace popisuje cestu prostě pořadím měst, takže cesta 5–1–7–8–9–4–6–2–3 se zapíše jako (517894623). Je nejpřirozenější, ale vyžaduje speciální operátory PMX, OX, CX a ER (odst. 1.3).
- Maticová** reprezentace používá binární matici, kde 1 na pozici (i, j) znamená buď hranu $i \rightarrow j$, nebo, což je častější, že i předchází j . Operátory jsou opět speciální: *konjunkce* udělá bitové AND obou rodičů a náhodně doplní chybějící hrany, *disjunkce* rozdělí matici na kvadranty, dva z nich zahodí, odstraní spory a zbytek doplní náhodně.

Inicializace. Populaci lze naplnit náhodnými permutacemi, nebo sáhnout po konstrukčních heuristikách. *Nejbližší soused* začne náhodným městem a vždy jde do nejbližšího nenavštíveného. *Vkládání hran* k částečné cestě T vybere nejbližší město c mimo T , najde v T hranu $k-j$ minimalizující $d(k, c) + d(c, j) - d(k, j)$ a tuto hranu nahradí dvojicí $k-c$, $c-j$.

Mutace. Nejdůležitější je inverze úseku, protože odpovídá 2-opt kroku. Dále se používá vložení města jinam, posun podcesty, prohození dvou měst, prohození podcest a heuristické mutace 2-opt a 3-opt, které vezmou dvě hrany a přepojí čtyři města jinak. Kombinace ES či GA s „chytrými mutacemi“ typu 2-opt je v praxi velmi účinná; de facto jde o memetický algoritmus.

6.3.4 SAT

♦ nad rámec slidů: slidy řeší batoh, TSP a rozvrhování; SAT je z Michalewicze]

Zadání. Je dána formule $f : \mathbb{B}^n \rightarrow \mathbb{B}$ v CNF a hledá se ohodnocení x , pro které platí $f(x) = 1$. Zatímco 2-SAT je v P, 3-SAT a všechno nad ním je NP-úplné.

Fitness. Samotné f je nepoužitelné, protože nabývá jen hodnot 0 a 1 a krajina tak vypadá jako „jehla v kupce sena“. Standardem je proto **počet splněných klauzulí**. Zlepšit se dá vážením problematických klauzulí a *zjemňující funkcí*, tedy regularizačním členem, který rozliší jedince se stejným počtem splněných klauzulí a pomůže tím z plošin.

Reprezentace bývá přímo bitový řetězec. Zajímavá je *reálná* varianta: proměnná x_j se nahradí reálným y_j (pravda = 1, nepravda = -1), literály se přepíší jako $x \mapsto (1-y)^2$ a $\neg x \mapsto (1+y)^2$ a výraz se *minimalizuje*. Hodnota literálu je tedy *penalta*, a proto se **disjunkce** (klauzule) přepíše jako **součin** a **konjunkce** (formule) jako **součet**. Klasickou lokální heuristikou je **WalkSAT** a kombinace EA + WSAT je typické memetické řešení.

6.3.5 Rozvrhování a plánování výroby

Školní rozvrh. Jedincem je celý rozvrh zapsaný jako matice, jejíž řádky jsou učitelé, sloupce třídy a hodnoty kódy předmětů. Mutace míchá předměty a křížení vyměňuje mezi jedinci lepší řádky. Fitness kombinuje kvalitu jednotlivých řádků s měkkými kritérii, jako jsou okna a rovnoměrnost. **Tvrdá omezení**, například že učitel nemůže učit dvě třídy naráz, je nutné respektovat *přímo v operátorech*. Jinak vzniká příliš mnoho nepřipustných jedinců.

Job shop scheduling. Výrobky se skládají z dílů, pro každý díl existuje více plánů výroby na strojích a přestavení stroje stojí čas; fitness je celkový čas výroby (*makespan*). Kritické je i zde kódování. *Permutační* jedinec nese jen pořadí výrobků a zbytek dourčí dekodér, což dovolí použít křížení z TSP, ale ukazuje se to jako málo efektivní, protože složitou část práce odvede dekodér. *Přímé* kódování celého plánu je výkonnější, zato vyžaduje specializované operátory.

6.4 Vícekriteriální optimalizace

6.4.1 Dominance a Pareto fronta

Místo jediné fitness teď máme celý vektor kritérií $f_1, \dots, f_k$; pro jednoduchost budeme všechna minimalizovat. Taková kritéria si obvykle odporují: levnější řešení bývá horší kvality, přesnější model bývá větší. Jakmile jsou kritéria v konfliktu, přestává existovat jediné nejlepší řešení, protože zlepšení v jednom kritériu se platí zhoršením v jiném. Potřebujeme tedy jiné vymezení toho, co znamená „lepší“, než je porovnání dvou čísel.

Dominance

Pro dvojici jedinců x, y zavádíme tři vztahy:

- Jedinec x **slabě dominuje** y ($x \preceq y$), právě když $f_i(x) \leq f_i(y)$ pro všechna $i = 1, \dots, k$, tedy není-li x v žádném kritériu horší.
- Jedinec x **dominuje** y ($x \prec y$), právě když $x \preceq y$ a navíc existuje j s $f_j(x) < f_j(y)$: v ničem není horší a alespoň v jednom kritériu je opravdu lepší.
- Jedinci x a y jsou **neporovnatelné**, pokud neplatí ani $x \prec y$, ani $y \prec x$; každý z nich je pak lepší v něčem jiném.

Relace $\prec$ je *ostré* (striktní) částečné uspořádání: je ireflexivní a tranzitivní, ale nikoli úplné. Právě z té neúplnosti plyne, že optimum obecně není jediný bod, nýbrž celá množina navzájem neporovnatelných řešení.

Pareto optimalita a Pareto fronta

Řešení x^* je *Pareto-optimální*, není-li dominováno žádným přípustným řešením. Množina všech Pareto-optimálních řešení v prostoru proměnných se nazývá *Pareto množina* a její obraz v prostoru kritérií je **Pareto fronta**. Cílem vícekritériálních EA je tuto frontu populací *aproximovat*.

Od algoritmu se proto chtějí dvě věci najednou. **Konvergence** znamená dostat populaci co nejbližší ke skutečné frontě, kdežto **diverzita** žádá pokrýt frontu rovnoměrně v celém jejím rozsahu. Právě proto se na tuhle úlohu evoluční algoritmy hodí lépe než cokoli jiného: pracují s populací, a mohou tedy celou aproximaci fronty vrátit v jediném běhu.

Příklad. Uvažujme body $A(1, 10)$, $B(2, 7)$, $C(4, 5)$, $D(8, 2)$ a $E(5, 8)$ a minimalizujme obě kritéria. Bod E je dominován bodem C , protože platí $4 \leq 5$ i $5 \leq 8$ a alespoň jedna z nerovností je ostrá; E tedy do fronty nepatří. Zbylé body A, B, C, D jsou navzájem neporovnatelné a tvoří Pareto frontu.

6.4.2 Skalarizace — jednoduchá řešení

Nejsnazší způsob, jak si s více kritérii poradit, je převést je zpátky na jediné číslo a pustit na úlohu obyčejný jednokritériální algoritmus. Podob takového převodu je několik a liší se hlavně tím, na které části fronty vůbec dosáhnou.

- **Lineární skalarizace** sečte kritéria s vahami, $f = \sum_i w_i f_i$, a výsledek předá standardnímu jednokritériálnímu EA (v kontextu MOEA se označuje jako SOEA). Slabiny má tři: neumíme volit váhy, jeden běh dá jediný bod fronty a zásadně **nekonvexní části fronty nelze lineární skalarizací nikdy nalézt**.
- **ε -constraint** minimalizuje jen kritérium f_1 a zbylá kritéria převede na omezení $f_i \leq \varepsilon_i$ pro $i \geq 2$. Na nekonvexní části fronty už dosáhne, zaplatí se za to ale volbou konstant ε_i a tím, že se řeší úloha s omezeními.
- **Čebyševova (Chebyshev) skalarizace** minimalizuje $\max_i \lambda_i |f_i(x) - z_i^*|$, kde z^* je referenční (ideální) bod. Na rozdíl od lineární varianty umí dosáhnout na libovolný bod fronty včetně nekonvexních částí.

6.4.3 Historické algoritmy

VEGA (Schaffer, 1985) patří mezi vůbec první MOEA. Populace N jedinců se pro každé kritérium f_i zvlášť seřadí a vybere se z ní N/k jedinců nejlepších podle f_i ; takto vzniklé podmnožiny se pak sloučí dohromady, zkříží a zmutují. Slabina je z toho vidět hned: každá podpopulace táhne evoluci ke svému vlastnímu extrému, takže se špatně udržuje diverzita a celek konverguje ke krajním bodům optimálním pro jednotlivá kritéria, zatímco „střed“ fronty se ztrácí.

NSGA (1994) poprvé použil dominanci přímo jako fitness. Populace se rozdělí do front: F_1 jsou nedominovaní jedinci z P , F_2 nedominovaní z $P \setminus F_1$, F_3 nedominovaní z $P \setminus (F_1 \cup F_2)$ a tak dál. Jedinci z první fronty dostanou „fiktivní“ fitness, kterou algoritmus vydělí *nikovým faktorem* $\sum_j sh(i, j)$, kde $sh(i, j) = 1 - (d(i, j)/d_{\text{share}})^2$ pro $d(i, j) < d_{\text{share}}$ a 0 jinak. Druhá fronta dostane fitness menší, než má nejhorší jedinec první fronty, opět dělenou nikovým faktorem, a stejně se pokračuje dál. Nevýhody jsou tři: je nutné nastavit d_{share} , chybí elitismus a výpočetní složitost $O(kN^3)$ je vysoká.

6.4.4 NSGA-II

Právě tyto nedostatky odstraňuje nástupce, který se dodnes používá jako standardní srovnávací algoritmus vícekritériální optimalizace.

NSGA-II (Deb a kol., 2000)

Opravuje nedostatky NSGA a stojí na třech pilířích. **(1) Rychlé nedominované třídění** rozdělí populaci do front se složitostí $O(kN^2)$. **(2) Crowding distance** nahradí parametr d_{share} , takže odpadá jeho ruční nastavování. **(3) Elitismus** spočívá v tom, že se rodiče i potomci spojí dohromady, setřídí a novou populaci tvoří lepší polovina.

Crowding distance

Počítá se pro každou frontu zvlášť. Pro každé kritérium m se jedinci fronty seřadí podle f_m , krajním jedincům se přiřadí ∞ a ostatním se přičte normalizovaná vzdálenost jejich sousedů:

$$d_i = \sum_{m=1}^k \frac{f_m(i+1) - f_m(i-1)}{f_m^{\max} - f_m^{\min}}.$$

Výsledek vyjadřuje „obvod kvádrů“ tvořeného nejbližšími sousedy, takže velká hodnota znamená řídce obsazenou oblast, kterou chceme zachovat.

Crowded comparison operator

Jedinec i je lepší než j ve dvou případech: buď i leží v nižší frontě ($\text{rank}_i < \text{rank}_j$), nebo se fronty shodují a i má větší crowding distance. Žádná skalární fitness se tedy vůbec nepočítá, porovnávají se jen tato dvě čísla, typicky v binárním turnaji.

Algoritmus 10 NSGA-II (jedna generace)

- 1: $R_t \leftarrow P_t \cup Q_t$ ▷ rodiče a potomci, celkem $2N$ jedinců
 - 2: $(F_1, F_2, \dots) \leftarrow$ nedominované třídění R_t
 - 3: $P_{t+1} \leftarrow \emptyset$; $i \leftarrow 1$
 - 4: **while** $|P_{t+1}| + |F_i| \leq N$ **do**
 - 5: spočítej crowding distance ve F_i ; $P_{t+1} \leftarrow P_{t+1} \cup F_i$; $i \leftarrow i + 1$
 - 6: **end while**
 - 7: seřaď F_i sestupně podle crowding distance a doplň P_{t+1} na N jedinců
 - 8: $Q_{t+1} \leftarrow$ turnajová selekce (crowded comparison) + křížení (SBX) + mutace
-

Číselný příklad crowding distance. Vraťme se k frontě $A(1, 10)$, $B(2, 7)$, $C(4, 5)$, $D(8, 2)$. První kritérium se pohybuje v rozsahu $f_1 \in [1, 8]$, tedy s rozpětím 7, druhé v rozsahu $f_2 \in [2, 10]$ s rozpětím 8. Krajní body A a D dostanou ∞ , protože jsou krajní v obou kritériích. Pro vnitřní body vyjde

$$d_B = \frac{f_1(C) - f_1(A)}{7} + \frac{f_2(A) - f_2(C)}{8} = \frac{4 - 1}{7} + \frac{10 - 5}{8} = 0,43 + 0,63 = 1,05,$$

$$d_C = \frac{f_1(D) - f_1(B)}{7} + \frac{f_2(B) - f_2(D)}{8} = \frac{8 - 2}{7} + \frac{7 - 2}{8} = 0,86 + 0,63 = 1,48.$$

Musí-li algoritmus vybrat jednoho z dvojice B , C , zvítězí C , protože leží v řidčeji obsazené části fronty.

NSGA-III. Tato varianta míří na úlohy s mnoha kritérii ($k \geq 4$, *many-objective*), kde crowding distance selhává, protože při tolika kritériích je nedominované téměř všechno. Crowding distance je zde nahrazena množinou rovnoměrně rozmístěných **referenčních bodů**, jejichž počet zhruba odpovídá velikosti populace a pro p dělení vychází $N = \binom{p+k-1}{p}$. Po nedominovaném třídění se přednostně doplňují ty referenční směry, které mají nejméně přiřazených řešení; nemá-li nějaký směr řešení žádné, přežije jedinec s nejmenší kolmou vzdáleností od příslušného referenčního vektoru.

6.4.5 Další významné algoritmy

♦ nad rámec slidů: slidy končí u NSGA-II]

SPEA2 si vedle populace udržuje externí **archiv** nedominovaných řešení. Fitness jedince je součet *sil* všech jedinců, kteří ho dominují, přičemž síla jedince je počet jedinců, které sám dominuje; k tomu se přičítá hustota odvozená ze vzdálenosti k σ -tému nejbližšímu sousedovi. **MOEA/D** rozloží úlohu na μ *skalárních* podúloh s rovnoměrně rozloženými váhovými vektory, které se na jedno číslo převádějí Čebyševovou skalarizací. Každý jedinec odpovídá jedné podúloze a spolupracuje jen se svým sousedstvím, odkud plyne nízká složitost a dobrá škálovatelnost. **SMS-EMOA** vybírá přímo podle příspěvku jedince k *hypervolume*; teoreticky je nejlépe podložený, ale s počtem kritérií exponenciálně zdražuje. **NSGA-III** už padl výše: nahrazuje crowding distance rovnoměrně rozmístěnými **referenčními body** a cílí na úlohy s mnoha kritérii ($k \geq 4$), kde crowding distance selhává.

6.4.6 Metriky kvality aproximace

♦ nad rámec slidů] Když algoritmus vrací celou aproximaci fronty místo jediného čísla, není samo sebou, jak dva běhy porovnat. K tomu slouží následující indikátory.

Hypervolume (*S*-metrika)

Objem oblasti dominované nalezenou frontou a ohraničené *referenčním bodem* r , který se volí tak, aby byl dominován všemi řešeními. Je to jediná běžná metrika, která je *striktně monotónní vůči dominanci* a nevyžaduje znalost skutečné Pareto fronty.

Příklad: mějme frontu $\{(2, 7), (4, 5), (8, 2)\}$ a $r = (10, 10)$. Objem sečteme po svislých pruzích: bod $(8, 2)$ dá $2 \cdot 8 = 16$, bod $(4, 5)$ dá $4 \cdot 5 = 20$ a bod $(2, 7)$ dá $2 \cdot 3 = 6$, celkem tedy $HV = 42$. Přidání dominovaného bodu hodnotu nezmění.

Používají se i další indikátory. *GD* je průměrná vzdálenost nalezených bodů od skutečné fronty, a měří tedy konvergenci; *IGD* počítá vzdálenost opačným směrem a postihne kromě konvergence i pokrytí fronty; *spread* hodnotí rovnoměrnost rozložení. GD i IGD ovšem vyžadují znalost skutečné fronty.

6.4.7 Souhrnné srovnání MOEA

Následující tabulka staví vedle sebe to, čím se jednotlivé algoritmy liší v selekci a v udržování diverzity.

algoritmus	princip selekce	diverzita	poznámka
VEGA	podpopulace podle f_i	žádná	konverguje ke krajům
NSGA	fronty + fiktivní fitness	fitness sharing (d_{share})	bez elitismu, $O(kN^3)$
NSGA-II	fronty + crowding	crowding distance	elitismus, $O(kN^2)$, standard
NSGA-III	fronty + referenční body	referenční směry	many-objective
SPEA2	síla dominujících	σ -tý souseď, truncation	externí archiv
MOEA/D	skalarizované podúlohy	rovnoměrné váhové vektory	sousedství, škáluje
SMS-EMOA	příspěvek k hypervolume	implicitní	teoreticky nejlépe podložené, drahé

6.5 Shrnutí ke zkoušce

- V **michiganském** přístupu je jedinec jedno pravidlo a řešením je celá populace, takže se musí řešit rozdělení odměny mezi pravidla (credit assignment). V **pittsburghském** přístupu je jedinec celá sada pravidel, fitness se proto spočítá přímočaře, zato jsou složité operátory (GIL).
- Hollandův LCS má podmínky zapsané nad abecedou $\{0, 1, *\}$, používá vnitřní zprávy a receptory a odměnu rozděluje mechanismem bucket brigade, v němž pravidlo platí svému předchůdci za právo jednat.

- ZCS se obejde bez vnitřních zpráv: z množiny odpovídajících pravidel (match set M) se ruletou vybere akce a tím vznikne action set A , posiluje se A i A_{-1} a pravidla z $M \setminus A$ platí daň. Chybí-li pro vstup vhodné pravidlo, zasáhne cover operátor.
- XCS pracuje s klasifikátorem (c, a, p, ϵ, F) a jeho hlavní myšlenkou je **fitness podle přesnosti predikce** $\kappa = \alpha(\epsilon/\epsilon_0)^{-\nu}$; predikce se aktualizuje Q-learningem a nikový GA běží v action setu. Výsledkem je úplná a minimální mapa vstup-výstup. Varianty jsou UCS pro učení s učitelem a XCSF pro aproximaci funkcí.
- Je dobré umět vysvětlit, proč přesnost jako fitness řeší nedostatky „strength-based“ systémů.
- Evolvovat se dají váhy, topologie, obojí zároveň, nebo hyperparametry učení. Výhodou EA je, že se obejdou bez gradientu, zvládnou rekurentní sítě i RL a dobře se paralelizují.
- Problém konkurujících konvencí (permutační problém) znamená, že křížení sítí je bez zvláštních opatření destruktivní.
- Architekturu lze kódovat přímo maticí, gramaticky (Kitano), nebo buněčně (Gruau, kde genotypem je GP program pro růst sítě). GNARL je EP nad grafy se strukturálními i parametrickými mutacemi, jejichž míru řídí teplota.
- NEAT reprezentuje síť seznamem hran opatřených **inovačními čísly**, díky nimž se kříží jen odpovídající si geny. **Speciace** podle podobnosti genomů se sdílenou fitness chrání nově vzniklé topologie a **complexification** znamená, že se začíná od minimální sítě.
- HyperNEAT používá substrát $S : \mathbb{R}^4 \rightarrow \mathbb{R}$, který generuje váhy ze souřadnic neuronů; ten je reprezentovaný pomocí CPPN, tedy DAG s různými aktivačními funkcemi, a evoluuje se algoritmem NEAT. Díky tomu zachycuje symetrie a regularity.
- U NAS se rozlišuje prostor architektur (lineární, DAG buněk, super síť), vyhledávací strategie (EA, RL, bayesovská, gradientní) a způsob odhadu výkonu, kde pomáhá sdílení vah (ENAS). Sem patří i DeepNEAT a CoDeepNEAT s moduly a blueprints a ES pro deep RL.
- Batoch se kóduje bitovou mapou, operátory jsou triviální a celý problém je v omezení kapacity. Nabízejí se tři strategie: **penalizace**, u níž je nutné zvolit váhu α , **oprava** jedince (lamarckovsky, nebo baldwinovsky) a **dekodér** typu „přidej, pokud se vejde“, po němž je každý genotyp přípustný; čtvrtou možností je vícekritériální formulace.
- U TSP je fitness triviální a problém je v kódování. Sousednostní ani ordinální reprezentace se nepoužívají, osvědčila se permutační s operátory PMX, OX, CX a ER, případně maticová. Inicializovat se dá nejbližším sousedem nebo vkládáním hran, mutovat inverzí, což odpovídá kroku 2-opt.
- U SAT je fitness počet splněných klauzulí, samotné f ovšem nestačí, a proto se přidává vážení klauzulí a zjemňující funkce; reprezentace může být bitová, reálná, klauzulová i cestová a jako lokální heuristika slouží WalkSAT. U reálné reprezentace $(x \mapsto (1 - y)^2, \neg x \mapsto (1 + y)^2, \text{minimalizace})$ pozor na to, že hodnoty jsou *penalty*, takže disjunkce (klauzule) se vyhodnotí jako **součin** a konjunkce (formule) jako **součet**.
- Rozvrhování se kóduje přímo maticí, tvrdá omezení se řeší už v operátorech a měkká až ve fitness; u job shopu stojí proti sobě permutace s dekodérem a přímé kódování.
- Obecně platí, že u kombinatorických úloh rozhoduje kódování a operátory respektující strukturu úlohy a že hybridizace s lokální heuristikou (2-opt, WSAT) je téměř vždy výhodná.
- Dominance $x \prec y$ znamená, že x není v žádném kritériu horší a alespoň v jednom je lepší; Pareto množina leží v prostoru proměnných, Pareto fronta v prostoru kritérií a dvojím cílem algoritmu je konvergence a diverzita.
- Ze skalarizací je nejjednodušší lineární, ta ale nenajde nekonvexní části fronty; dále se používá ϵ -constraint a Čebyševova skalarizace.
- NSGA-II třídí populaci nedominovaným tříděním do front, uvnitř fronty počítá crowding distance $d_i = \sum_m (f_m(i+1) - f_m(i-1)) / (f_m^{\max} - f_m^{\min})$ s ∞ pro krajní body a jedince porovnává operátorem crowded comparison (vyhrává nižší fronta, při shodě větší crowding distance). Elitismus zajišťuje spojení $P_t \cup Q_t$. Crowding distance je třeba umět spočítat na příkladu.
- SPEA2 si udržuje archiv a fitness staví na síle dominujících jedinců a na hustotě přes σ -tého souseda, MOEA/D používá Čebyševovu skalarizaci s váhovými vektory a sousedstvím, SMS-EMOA vybírá podle příspěvku k hypervolume a NSGA-III nasazuje referenční body na many-

objective úlohy.

- Hypervolume je objem dominovaný frontou vůči referenčnímu bodu a jako jediná běžná metrika je monotónní vůči dominanci a obejde se bez znalosti skutečné fronty. Vedle něj se používají GD, IGD, spread a ε -indikátor.

Souhrnné srovnání a typické zkouškové otázky

Jeden pohled na všechny algoritmy

Když jsou všechny metody probrané, vyplatí se podívat se na ně jedním společným pohledem. Slidy EVA II tuto myšlenku formulují až u CMA-ES, platí ale pro celý okruh: probírané algoritmy dělají v jádru jednu a tutéž věc. U zkoušky je to nejlepší možné zahájení kterékoli odpovědi, protože rovnou ukáže, že jednotlivé metody nejsou nesouvislá sbírka triků.

Stochastické black-box hledání

Black-box optimalizace znamená, že minimalizujeme funkci $f : \mathbb{R}^n \rightarrow \mathbb{R}$, přičemž jedinou dostupnou informací o f jsou *její hodnoty ve zvolených bodech*. Gradient k dispozici není a analytický popis také ne. Cenou hledání je proto *počet vyhodnocení f* .

Všechny probírané metody mají pak tentýž skelet. Algoritmus si udržuje **rozdělení pravděpodobnosti** $p_\theta(x)$ nad prostorem řešení a pořád dokola opakuje tři kroky:

$$\text{navzorkuj } \{x_i\}_{i=1}^n \sim p_\theta \longrightarrow \text{vyhodnoť } \{(x_i, f(x_i))\} \longrightarrow \theta \leftarrow h(\theta, D).$$

Parametr θ je *jediná* informace, která se přenáší mezi iteracemi. Kóduje všechno, co se algoritmus dosud naučil, a určuje, kde bude hledat dál. Jednotlivé metody se pak liší jen ve dvou věcech: *co je θ* a jak zní aktualizací heuristika h .

algoritmus	co je θ
EA (GA, GP, ...)	rodičovská populace — rozdělení je empirické, dané vzorkem; aktualizace = selekce + variace
ES	gaussíán: střed a šířka $\vec{\sigma}$ (u samoadaptace nese σ každý jedinec sám)
CMA-ES	$(\vec{m}, \sigma, \mathbf{C}, \vec{p}_\sigma, \vec{p}_c)$ — plná kovariance + evoluční cesty
EDA	parametry explicitního modelu: marginály (UMDA, PBIL), párové závislosti (MIMIC), bayesovská síť (BOA)
PSO	pozice a rychlosti částic + paměť $\vec{p}_i, \vec{g}$
ACO	feromonová matice τ — rozdělení nad <i>konstrukcemi</i> , ne nad body
SA	aktuální bod + teplota T
tabu search	aktuální bod + seznam zakázaných tahů

Z téhle perspektivy přestávají být některé metody překvapením. EDA (§2.2.6) je jen důsledné domyšlení celé myšlenky: když je populace stejně jen implicitní model rozdělení, není důvod ho nemodelovat rovnou explicitně. A naopak, poznání, že genetický algoritmus není nic jiného než transformace rozdělení, je přesně to, co formalizuje Voseho model (§2.2.2).

Mapa oboru

Celý okruh se dá rozdělit do čtyř skupin podle toho, odkud metoda bere svou inspiraci a co je jejím předmětem.

- **Evoluční algoritmy** stojí na populaci, fitness, křížení, mutaci a selekci. Patří sem genetické algoritmy ve všech svých kódováních (binární, celočíselné, permutační, reálné), evoluční strategie, diferenciální evuce, evoluční programování, genetické programování se svými variantami (stromové, lineární, kartézské, gramatická evuce), learning classifier systems a algoritmy odhadu rozdělení (EDA).

- **Přírodou inspirované metody bez evoluce** čerpají předlohu z přírody, evoluční schéma ale nepoužívají. Jsou to rojové algoritmy PSO, ACO, ABC a firefly.
- **Nebiologicky inspirované metaheuristiky** berou inspiraci odjinud než z živé přírody. Řadí se mezi ně tabu search, scatter search, novelty search, simulované žíhání a memetické algoritmy.
- **Aplikační oblasti** si vynucují vlastní reprezentace a operátory, a proto se probírají samostatně. Patří mezi ně kombinatorická optimalizace, neuroevoluce, zpětnovazební učení, vícekritériální optimalizace a AutoML/NAS.

Souhrnná srovnávací tabulka algoritmů

Následující tabulka staví algoritmy vedle sebe podle čtyř hledisek: čím reprezentují řešení, jak vytvářejí nové kandidáty, jak vybírají mezi nimi a co se u každého z nich nejčastěji chce slyšet.

algoritmus	reprezentace	variace	selekce	co si zapamatovat
SGA	binární řetězec	1-bod. křížení, bit-flip	ruleta, generační	teorie schémat, BBH
ES	$\mathbb{R}^n + \sigma$	gaussovská mutace, rekombinace	náhodní rodiče, $(\mu, \lambda)/(\mu+\lambda)$	pravidlo 1/5, samo-adaptace
CMA-ES	$\mathbb{R}^n$, model $\mathcal{N}(m, \sigma^2 C)$	vzorkování z rozdělení	pořadová, μ nejlepších	evoluční cesty, rank-1/rank- μ
DE	$\mathbb{R}^n$	$\vec{x}_a + F(\vec{x}_b - \vec{x}_c)$, binom. křížení	souboj rodič-potomek	měřítko z populace
EP	doménová (automat)	jen mutace	turnaj rodič+potomci	genotyp = fenotyp
GP	syntaktický strom	výměna podstromů, mutace	turnaj	bloat, ADF
PSO	$\mathbb{R}^n +$ rychlost	polybové rovnice	žádná (jen paměť)	w , c_1 , c_2 , gbest/l-best
ACO	konstrukce cesty	feromon + heuristika	implicitní	$\tau^\alpha \eta^\beta$, odpařování
SA	jedno řešení	soused	Metropolis	rozvrh chlazení
MA	libovolná	EA operátory + lokální hledání	libovolná	Lamarck vs. Baldwin
NSGA-II	libovolná	SBX + polynom. mutace	fronty + crowding	elitismus, crowding distance
NEAT	seznam hran	mutace struktury, křížení dle ID	sdílená fitness v druhu	inovační čísla, speciace

Průřezová témata — co se ptá napříč otázkami

Některá témata nepatří k jedné metodě, ale prostupují celým okruhem. Právě na nich se pozná, jestli student vidí souvislosti, a proto se na ně zkoušející ptají nejraději.

- **Explorace vs. exploatace:** u každého algoritmu se dá ukázat, čím se rovnováha mezi nimi řídí. U GA je to napětí mezi křížením a selekčním tlakem, u ES velikost σ a pravidlo 1/5, u simulovaného žíhání teplota, u PSO váha w a topologie, u ACO parametr q_0 a odpařování, u tabu search samotný tabu list. Novelty search je krajní případ, protože stojí jen na exploraci.
- **Udržení diversity** se řeší celou sadou nástrojů: fitness sharing, crowding, speciace, ostrovní model, nenáhodné páření, restart a hypermutace.
- **Adaptace parametrů** se probírá jako dvojice ladění versus řízení a jako trojice pevná, adaptivní a samoadaptivní strategie. U zkoušky se pak chce vědět, kde se která z nich v kterém algoritmu vyskytuje.
- **Práce s omezeními** má několik standardních řešení: penalizaci, opravu, dekodér, uzavřené operátory a vícekritériální formulaci.
- **Lamarckismus vs. baldwinismus** se objevuje u memetických algoritmů, při opravě jedinců i v neuroevoluci s doučováním vah.

- **Teoretické zázemí** okruhu tvoří věta o schématech i její kritika, Voseho model, Markovovy řetězce a konvergence elitistického GA, No Free Lunch a konvergence SA.

Typické zkouškové otázky

Následující otázky se u zkoušky objevují opakovaně; jsou to doslovná zadání, ne parafráze.

1. Popište obecné schéma evolučního algoritmu a vyjmenujte typy selekce včetně jejich vlastností. Spočtete ruletovou selekci pro danou populaci.
2. Formulujte a dokažte větu o schématech. Co je řád a definující délka? Jaké jsou slabiny věty?
3. Vysvětlete hypotézu stavebních bloků a implicitní paralelismus. Co je deceptivní úloha?
4. Popište Voseho model SGA: co jsou vektory p a s , co dělá operátor $\mathcal{F}$, co $\mathcal{M}$, jaké mají pevné body? Jak vypadá model konečné populace jako Markovův řetězec?
5. Co jsou algoritmy odhadu rozdělení (EDA)? Jaký je jejich vztah k Voseho pohledu na GA a jak se liší UMDA/PBIL od BOA?
6. Jaký je rozdíl mezi (μ, λ) a $(\mu + \lambda)$ -ES? Vysvětlete pravidlo $1/5$ a samoadaptaci σ . Jak funguje CMA-ES?
7. Popište jeden krok diferenciální evoluce včetně vzorců a číselného příkladu. Jaké jsou varianty mutace?
8. Jak funguje stromové genetické programování? Co je bloat a jak se proti němu bojuje? K čemu slouží ADF?
9. Vysvětlete gramatickou evoluci a kartézské GP — jaký je genotyp a fenotyp?
10. Co je koevoluce, jaké má typy a jaké patologie? Jak se hodnotí fitness?
11. Co je otevřená evoluce, jaké má znaky a proč se umělé systémy „zasekávají“? Jak funguje novelty search a proč se u deceptivních úloh vyplatí zahodit cíl?
12. Vysvětlete iterované věžňovo dilema, strategii TFT a vlastnosti úspěšných strategií.
13. Napište rovnice PSO a vysvětlete význam jednotlivých členů. Jaký je rozdíl mezi gbest a lbest topologií?
14. Popište ACO: pravidlo přechodu, aktualizaci feromonu, rozdíl mezi AS a ACS.
15. Vysvětlete simulované žíhání, Metropolisovo kritérium a rozvrh chlazení. Kdy konverguje?
16. Co je memetický algoritmus? Vysvětlete rozdíl mezi lamarckovským a baldwinovským učením.
17. Porovnejte michiganský a pittsburghský přístup. Jak funguje XCS a proč je fitness založena na přesnosti?
18. Popište NEAT: k čemu slouží inovační čísla a speciace? Co přidává HyperNEAT?
19. Jak se řeší TSP evolučním algoritmem? Popište PMX, OX, CX a ER.
20. Jak se v EA pracuje s omezeními (na příkladu problému batohu)?
21. Definujte dominanci a Pareto frontu. Popište NSGA-II a spočtete crowding distance na příkladu. Co je hypervolume?

Závěrečné shrnutí

- Evoluční a rojové algoritmy jsou **robustní metaheuristiky** pro úlohy, na kterých selhávají exaktní a gradientní metody. Typicky jde o úlohy black-box, nediferencovatelné, diskrétní, strukturované, vícekritériální a dynamické.
- Jejich síla neleží v jednom „nejlepším“ algoritmu, protože takový podle No Free Lunch neexistuje. Leží ve **schopnosti přizpůsobit reprezentaci a operátory dané doméně** a v možnosti **hybridizace** s lokálními heuristikami a doménovými znalostmi.
- Teorie, tedy schémata, Voseho model, konvergenční věty a analýza doby běhu, poskytuje intuici a asymptotické záruky. Praktický návrh přesto zůstává z velké části experimentální disciplínou.